ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN

Môn: NGÔN NGỮ LẬP TRÌNH C#

Tên đề tài:Lập Trình WinForm Nâng Cao

Giảng viên hướng dẫn: Đỗ Như Tài

Sinh viên thực hiện: Huỳnh Công Tính

MSSV: 3124411315

Lớp: DCT124C6

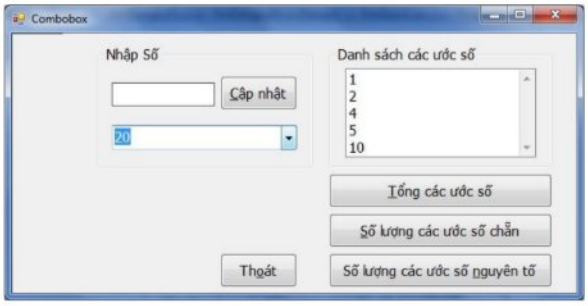
TP. HỒ CHÍ MINH, THÁNG 9 NĂM 2026

THỰC HÀNH 5C: LẬP TRÌNH WINDOW FORM NÂNG CAO (1)

Bài Tập Cơ Bản - Bài 1:

2. Bài tập tại lớp

Bài 1. Thiết kế giao diện sau:



Yêu cầu:

- Nhập số vào textbox Nhập số (có kiểm tra dữ liệu nhập).
- btnCapNhat: thêm số vừa nhập vào combobox, đồng thời xóa dữ liệu trong textbox Nhập số, đặt con trỏ lại textbox (có kiểm tra số vừa nhập đã tồn tại trong combobox chọa trước khi thêm).
- Khi ngọời dùng chọn một số trong combobox thì danh sách các ước số của số này sẽ hiển thị vào listbox bên phải tương ứng.
- Khi nhấn các button Tính: "Tổng các ước số", "Số lượng các ước số chẵn", "Số lượng các ước số nguyên tố" thì hiển thị thông tin tương ứng vào MessageBox dựa vào các ước số trên listbox.
- btnThoat: thoát khỏi ứng dụng có xác nhận từ ngọời dùng.
- Tạo HotKey cho các button nhọ trên giao diện

1. Yêu cầu bài tập

Thiết kế chương trình cho phép người dùng nhập số vào TextBox và thêm số vào ComboBox bằng nút Cập nhật. Trước khi thêm, chương trình phải kiểm tra dữ liệu nhập và kiểm tra số đã tồn tại trong ComboBox hay chưa.

Khi người dùng chọn một số trong ComboBox, chương trình hiển thị danh sách các ước số của số đó vào ListBox. Các nút xử lý thực hiện tính tổng các ước số, đếm số lượng ước chẵn và đếm số lượng ước nguyên tố. Kết quả được thông báo bằng MessageBox.

Chương trình có phím tắt cho các nút và hỏi xác nhận trước khi đóng Form.

2. Cách tổ chức chương trình

Bài tập được đặt trong project Bai01_UocSo, thuộc thư mục BaiTapWinform\TH5c.

File Program.cs chứa điểm bắt đầu của chương trình và gọi Form frmUocSo. File frmUocSo.cs chứa phần tạo giao diện, xử lý sự kiện và các phương thức tính toán.

Giao diện được tạo bằng code để có thể chỉnh sửa trực tiếp trong VS Code. Dữ liệu chỉ được lưu trong bộ nhớ khi chương trình đang chạy.

3. Thiết kế giao diện

Chương trình sử dụng TextBox để nhập số, ComboBox để lưu các số đã nhập và ListBox để hiển thị danh sách ước số.

Các nút trên Form gồm Cập nhật, Tổng các ước số, Số lượng các ước số chẵn, Số lượng các ước số nguyên tố và Thoát.

ComboBox được đặt thuộc tính DropDownStyle là DropDownList. Vì vậy, người dùng chỉ chọn những số đã được kiểm tra và thêm vào danh sách.

4. Kiểm tra dữ liệu nhập

Chương trình nhận số nguyên dương trong phạm vi của kiểu int, từ 1 đến 2147483647.

Sự kiện KeyPress của TextBox chặn các ký tự khác chữ số khi người dùng gõ. Các ký tự điều khiển như Backspace vẫn được cho phép để người dùng sửa nội dung.

Khi nhấn Cập nhật, chương trình tiếp tục kiểm tra bằng int.TryParse. Bước kiểm tra này xử lý cả trường hợp dán dữ liệu không hợp lệ, ô nhập trống và số vượt giới hạn của kiểu int.

Số 0 không được chấp nhận vì có vô số ước nguyên dương. Các số âm cũng không thuộc phạm vi xử lý của bài giải.

5. Thêm số vào ComboBox

Sau khi dữ liệu hợp lệ, chương trình sử dụng Items.Contains để kiểm tra số đã tồn tại trong ComboBox hay chưa. Nếu trùng, chương trình thông báo và không thêm số đó.

ComboBox lưu các giá trị dưới dạng int. Do đó, dữ liệu nhập “20” và “020” đều được chuyển thành số 20 và được xem là trùng nhau.

Nếu số chưa tồn tại, chương trình thêm số vào ComboBox, chọn số vừa thêm, xóa nội dung TextBox và đưa con trỏ trở lại ô nhập.

6. Tìm và hiển thị các ước số

Khi lựa chọn trong ComboBox thay đổi, sự kiện SelectedIndexChanged được thực hiện. Chương trình xóa danh sách ước cũ và tìm các ước của số mới chọn.

Với số nguyên dương n, nếu n chia hết cho i thì i và n/i đều là ước của n. Chương trình dựa vào tính chất này để tìm theo cặp ước và chỉ duyệt đến căn bậc hai của n.

Điều kiện vòng lặp được viết là i <= n / i để tránh phép nhân i * i có thể gây tràn số. Nếu i bằng n/i, chương trình chỉ thêm một lần để tránh trùng ước của số chính phương.

Sau khi tìm xong, danh sách được sắp xếp tăng dần rồi đưa vào ListBox. Các ước được hiển thị bao gồm 1 và chính số đang xét.

7. Tính tổng và đếm các loại ước

Nút Tổng các ước số duyệt các phần tử trong ListBox và cộng vào biến tổng. Biến tổng sử dụng kiểu long để hạn chế tràn số khi cộng nhiều giá trị.

Nút Số lượng các ước số chẵn kiểm tra từng ước bằng điều kiện chia dư cho 2 bằng 0. Nếu thỏa mãn, biến đếm tăng thêm một đơn vị.

Nút Số lượng các ước số nguyên tố gọi phương thức LaSoNguyenTo cho từng phần tử. Phương thức này loại các số nhỏ hơn 2, sau đó kiểm tra xem số có chia hết cho một số từ 2 đến căn bậc hai của nó hay không. Nếu có thì số đó không phải số nguyên tố.

Các chức năng tính toán đều kiểm tra người dùng đã chọn số hay chưa. Nếu chưa chọn, chương trình hiển thị thông báo thay vì đưa ra kết quả không phù hợp.

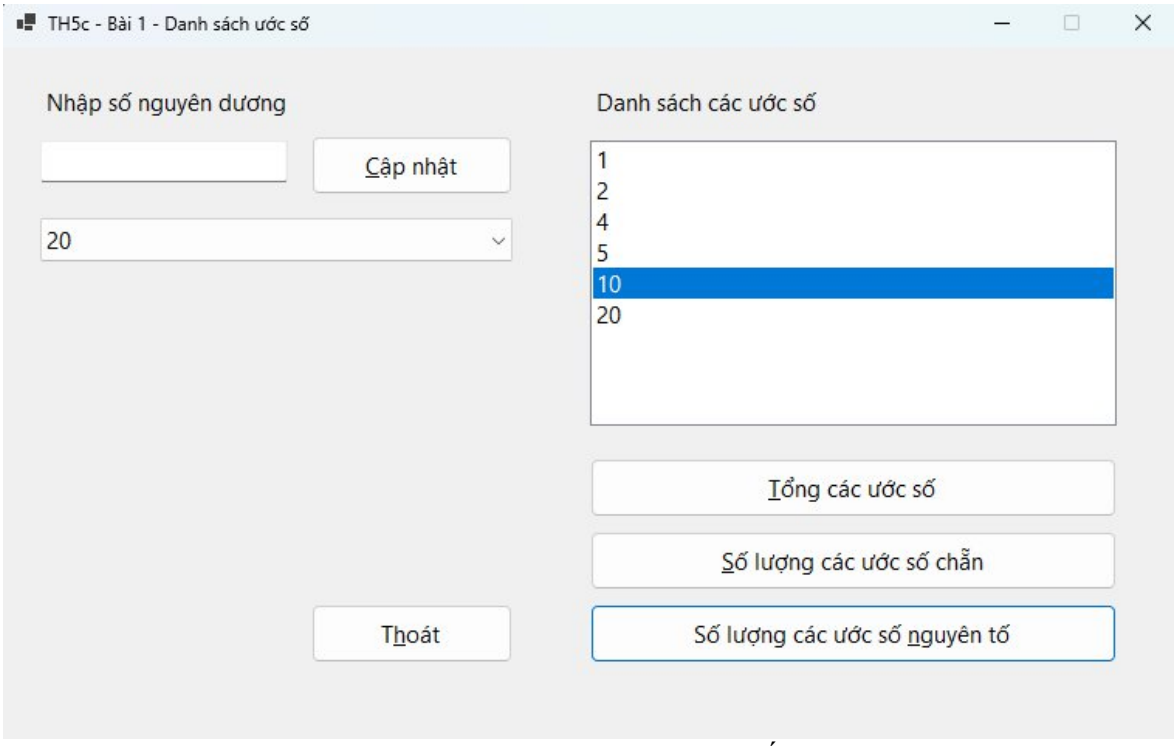
8. Phím tắt và xác nhận đóng Form

Phím tắt được tạo bằng cách đặt dấu & trước một ký tự trong nội dung của Button. Người dùng nhấn Alt kết hợp với ký tự đó để thực hiện chức năng.

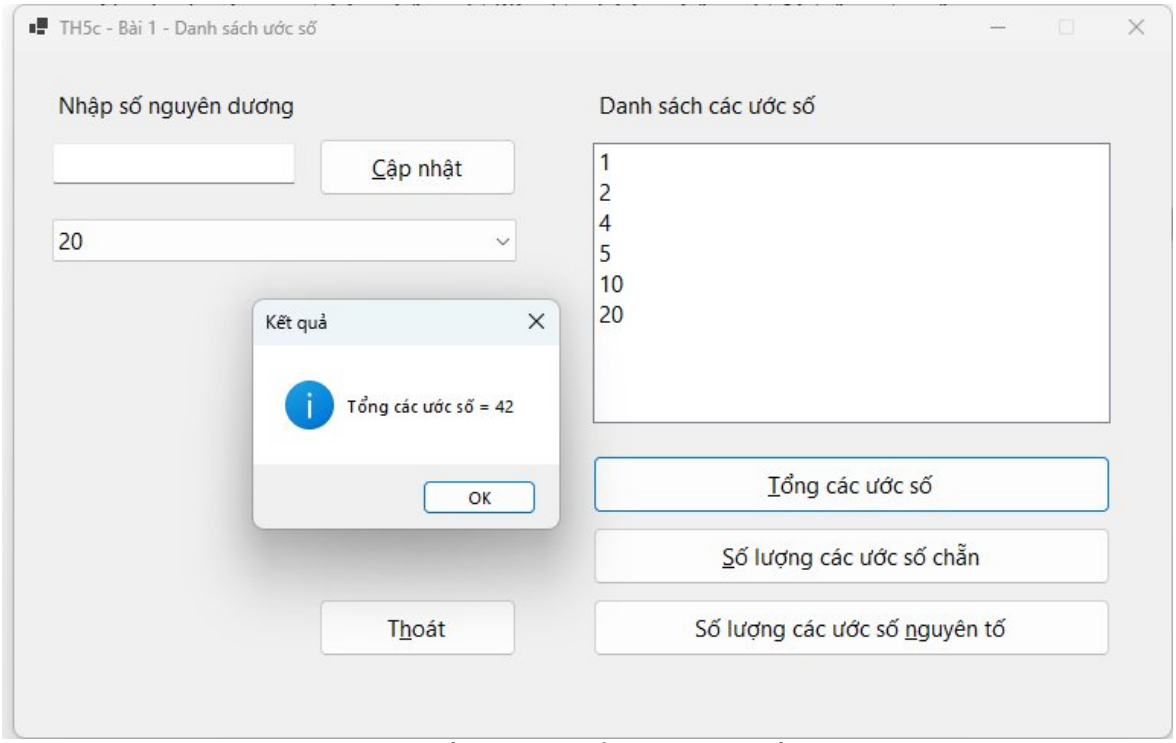
Các phím tắt gồm Alt + C để cập nhật, Alt + T để tính tổng, Alt + S để đếm ước chẵn, Alt + N để đếm ước nguyên tố và Alt + H để thoát. Ngoài ra, thuộc tính AcceptButton cho phép nhấn Enter để cập nhật số.

Sự kiện FormClosing hiển thị MessageBox xác nhận. Nếu chọn Yes, Form được đóng. Nếu chọn No, chương trình đặt e.Cancel = true để hủy thao tác đóng. Cách xử lý này áp dụng cả khi nhấn nút Thoát và khi nhấn nút X trên cửa sổ.

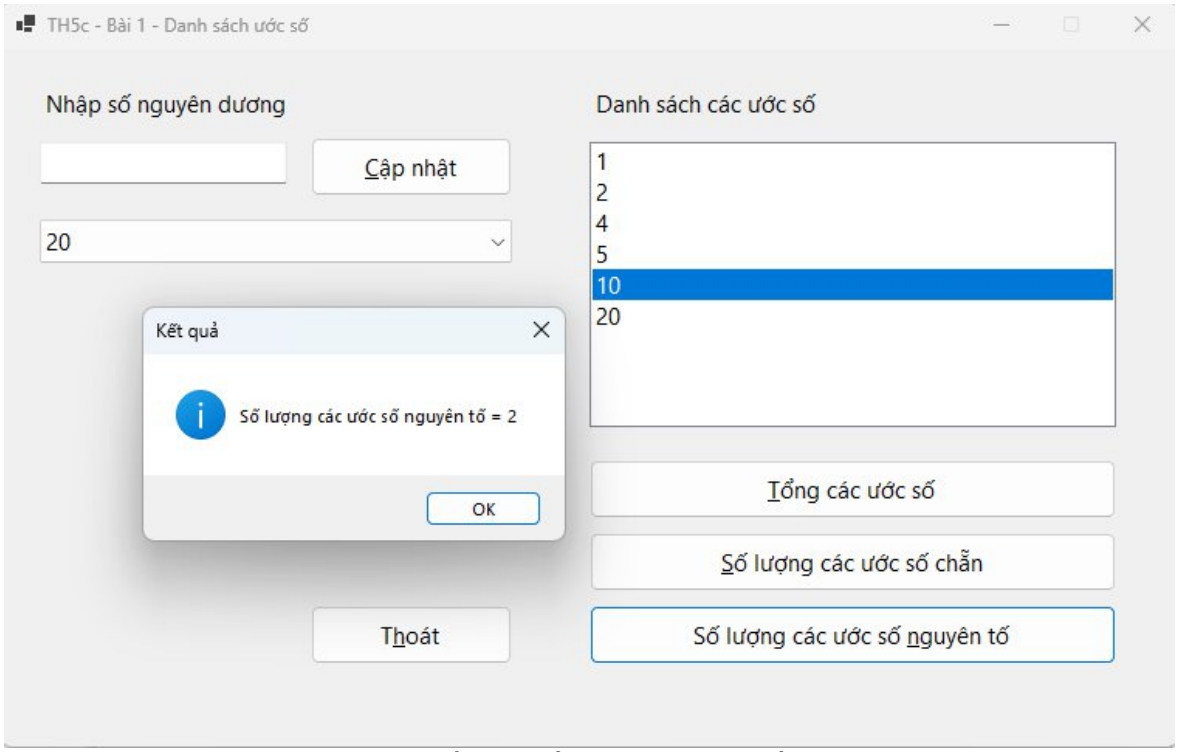
9. Kiểm thử chương trình



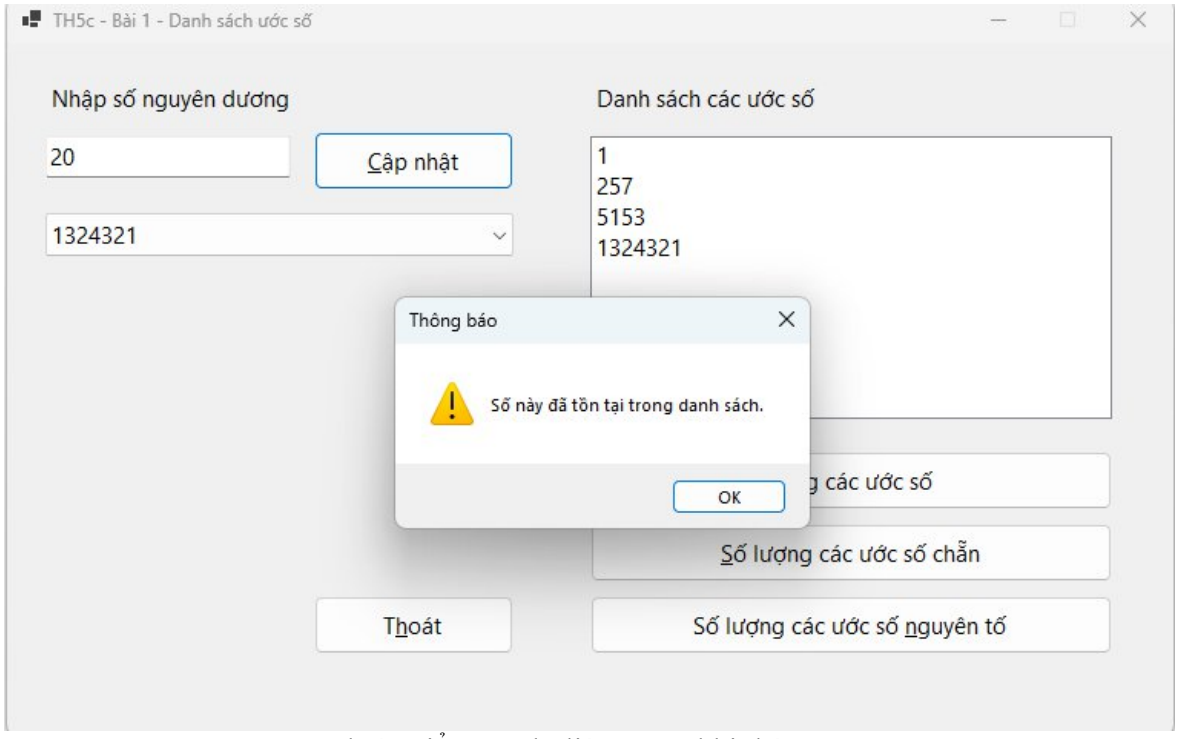
Hình 1. Giao diện và danh sách các ước số của 20.



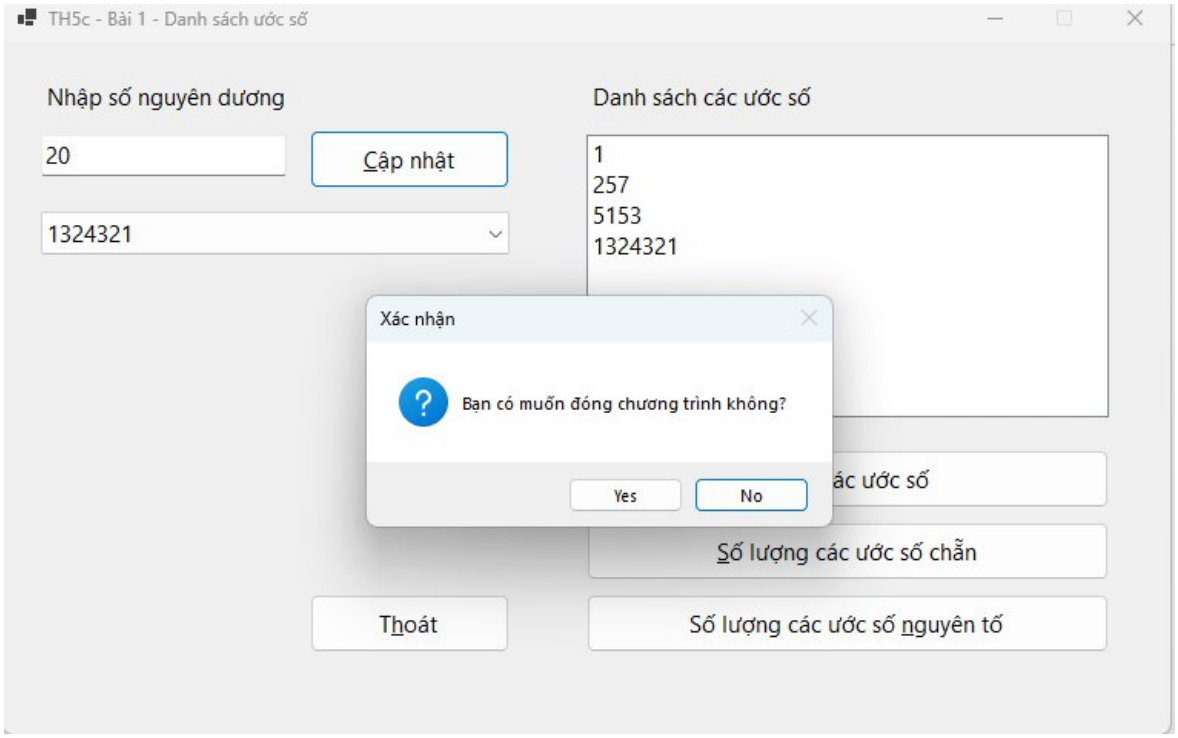
Hình 2. Kết quả tính tổng các ước số.



Hình 3. Kết quả đếm các loại ước số.



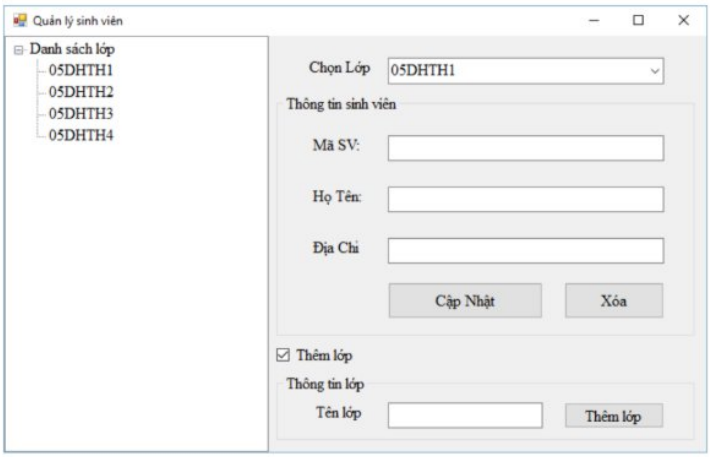
Hình 4. Kiểm tra dữ liệu trước khi thêm.



Hình 5. Xác nhận trước khi đóng Form.

Bài Tập Cơ Bản - Bài 2:

Bài 2. Thiết kế giao diện nhoị sau:

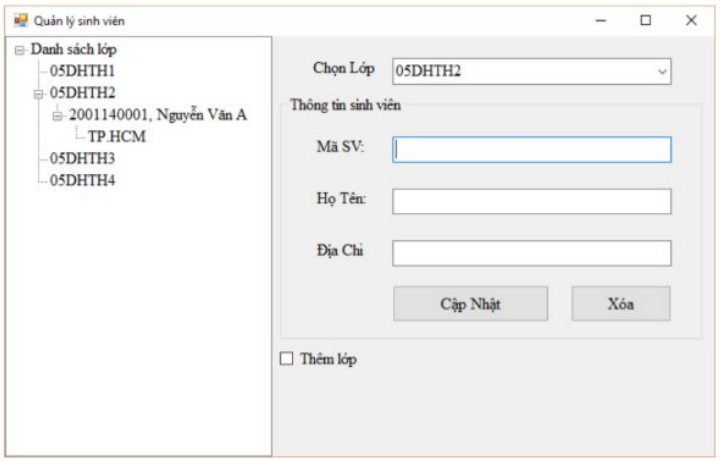


Yêu cầu:

– Khi Form hiện lên, danh sách lớp ở treeView đã tồn tại một số lớp; checkbox Thêm lớp là false (groupbox Thông tin lớp và các Control bên trong ẩn đi). Khi checkbox Thêm lớp là true groupBox Thông tin lớp và các control bên trong hiện lên.

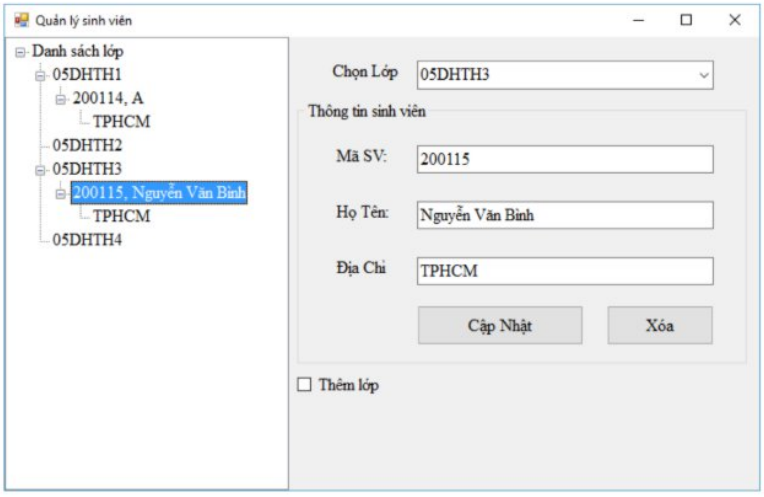
– Button Thêm lớp: Thực hiện thêm một lớp mới vào Danh sách lớp của TreeView (Kiểm tra tên lớp không được trùng trước khi thêm), đồng thời thêm lớp đó vào ComboBox chọn lớp.

– Button Cập Nhật: Thêm một sinh viên vào danh sách lớp đang chọn trên ComboBox với nội dung các node nhoị hình. Trước khi thêm kiểm tra thông tin nhập bao gồm: các textbox nhập không được để trống, không trùng mã sinh viên.



– Button Xóa: Chỉ cho phép xóa một node là sinh viên đang chọn trên TreeView (xác nhận trước khi xóa).

– Khi click chọn một node là mã sinh viên trong TreeView, hiển thị thông tin của sinh viên đó lên textbox tương ứng.



1. Yêu cầu bài tập

Thiết kế chương trình quản lý danh sách lớp và sinh viên bằng TreeView. Khi mở Form, chương trình có sẵn một số lớp trong TreeView và ComboBox chọn lớp.

Checkbox Thêm lớp ban đầu chưa được chọn và nhóm Thông tin lớp được ẩn. Khi chọn checkbox, nhóm thông tin lớp xuất hiện để người dùng nhập tên lớp mới.

Chương trình cho phép thêm lớp, thêm sinh viên vào lớp đang chọn, xóa sinh viên có xác nhận và hiển thị thông tin khi chọn node sinh viên. Tên lớp không được trùng, thông tin sinh viên không được để trống và mã sinh viên không được trùng.

2. Tổ chức chương trình

Bài tập được đặt trong project Bai02_QuanLySinhVien, thuộc thư mục BaiTapWinform\TH5c.

File Program.cs chứa điểm bắt đầu của chương trình và gọi Form frmQuanLySinhVien. File frmQuanLySinhVien.cs chứa lớp SinhVien, phần tạo giao diện và các phương thức xử lý sự kiện.

Lớp SinhVien gồm ba thuộc tính: MaSV, HoTen và DiaChi. Những thuộc tính này lưu thông tin sinh viên để sử dụng khi chọn hoặc xóa node trên TreeView.

Dữ liệu được lưu trong bộ nhớ trong thời gian chương trình hoạt động. Khi đóng và mở lại chương trình, danh sách sinh viên và những lớp được thêm mới không được giữ lại.

3. Thiết kế giao diện

TreeView được đặt bên trái để hiển thị danh sách lớp và sinh viên. Bên phải gồm ComboBox chọn lớp, nhóm thông tin sinh viên, checkbox Thêm lớp và nhóm thông tin lớp.

Nhóm thông tin sinh viên có các TextBox nhập mã sinh viên, họ tên, địa chỉ cùng hai nút Cập nhật và Xóa. Nhóm thông tin lớp có TextBox nhập tên lớp và nút Thêm lớp.

ComboBox sử dụng chế độ DropDownList để người dùng chỉ chọn những lớp đã tồn tại trong danh sách.

Cấu trúc TreeView gồm node gốc Danh sách lớp, các node lớp, các node sinh viên và node địa chỉ nằm bên dưới từng sinh viên. Node sinh viên hiển thị mã và họ tên, ví dụ “200115, Nguyễn Văn Bình”.

4. Khởi tạo danh sách lớp và điều khiển hiển thị

Khi Form được khởi tạo, chương trình thêm bốn lớp 05DHTH1, 05DHTH2, 05DHTH3 và 05DHTH4 vào TreeView và ComboBox. Lớp đầu tiên được chọn sẵn trong ComboBox.

Checkbox Thêm lớp có thuộc tính Checked bằng false. GroupBox Thông tin lớp có thuộc tính Visible bằng false nên được ẩn cùng các control bên trong.

Sự kiện CheckedChanged cập nhật thuộc tính Visible của GroupBox theo trạng thái checkbox. Khi checkbox được chọn, nhóm thông tin lớp xuất hiện và con trỏ được đặt vào ô nhập tên lớp.

5. Xử lý thêm lớp

Khi nhấn Thêm lớp, chương trình lấy tên lớp và loại bỏ khoảng trắng ở đầu và cuối bằng phương thức Trim. Nếu tên lớp trống, chương trình thông báo và yêu cầu nhập lại.

Chương trình duyệt các node lớp để kiểm tra tên đã tồn tại hay chưa. Việc so sánh không phân biệt chữ hoa, chữ thường. Vì vậy, “05DHTH1” và “05dhth1” được xem là cùng một tên lớp.

Nếu hợp lệ, tên lớp được thêm đồng thời vào TreeView và ComboBox bằng phương thức ThemLopVaoDanhSach. Lớp mới được chọn trong ComboBox để người dùng có thể thêm sinh viên ngay. Sau đó, ô nhập tên lớp được xóa và nhận lại con trỏ.

6. Xử lý thêm sinh viên

Nút Cập nhật thực hiện thêm sinh viên mới vào lớp đang chọn theo yêu cầu của đề bài.

Chương trình kiểm tra lần lượt mã sinh viên, họ tên và địa chỉ. Nếu một thông tin bị bỏ trống, chương trình hiển thị thông báo và đặt con trỏ vào ô tương ứng. Chương trình cũng kiểm tra người dùng đã chọn lớp hay chưa.

Mã sinh viên được kiểm tra bằng Dictionary với chế độ so sánh không phân biệt chữ hoa, chữ thường. Mã phải duy nhất trên toàn bộ các lớp, nên một mã đã có ở lớp này không được thêm lại vào lớp khác.

Sau khi kiểm tra hợp lệ, chương trình tạo đối tượng SinhVien và node hiển thị mã cùng họ tên. Đối tượng SinhVien được lưu trong thuộc tính Tag của node. Địa chỉ được thêm thành node con để có cấu trúc giống hình minh họa trong đề.

Node sinh viên được thêm vào đúng lớp đang chọn. Các node được mở rộng để nhìn thấy sinh viên vừa thêm. Cuối cùng, chương trình xóa các ô nhập và đặt con trỏ về ô mã sinh viên.

7. Hiển thị thông tin và xóa sinh viên

Sự kiện AfterSelect được thực hiện khi người dùng chọn node trong TreeView. Nếu thuộc tính Tag của node chứa đối tượng SinhVien, chương trình đưa mã, họ tên và địa chỉ lên các TextBox. ComboBox cũng được chuyển về đúng lớp của sinh viên.

Việc lưu đối tượng trong Tag giúp chương trình lấy dữ liệu trực tiếp, không cần tách chuỗi đang hiển thị trên node. Vì vậy, thông tin vẫn được lấy đúng ngay cả khi họ tên có dấu phẩy.

Khi nhấn Xóa, chương trình kiểm tra node đang chọn có phải node sinh viên hay không. Node gốc, node lớp và node địa chỉ không được phép xóa bằng chức năng này.

Nếu chọn đúng sinh viên, chương trình hiển thị hộp thoại xác nhận. Chọn No thì dữ liệu được giữ nguyên. Chọn Yes thì node sinh viên được xóa cùng node địa chỉ bên dưới, đồng thời mã sinh viên được loại khỏi Dictionary.

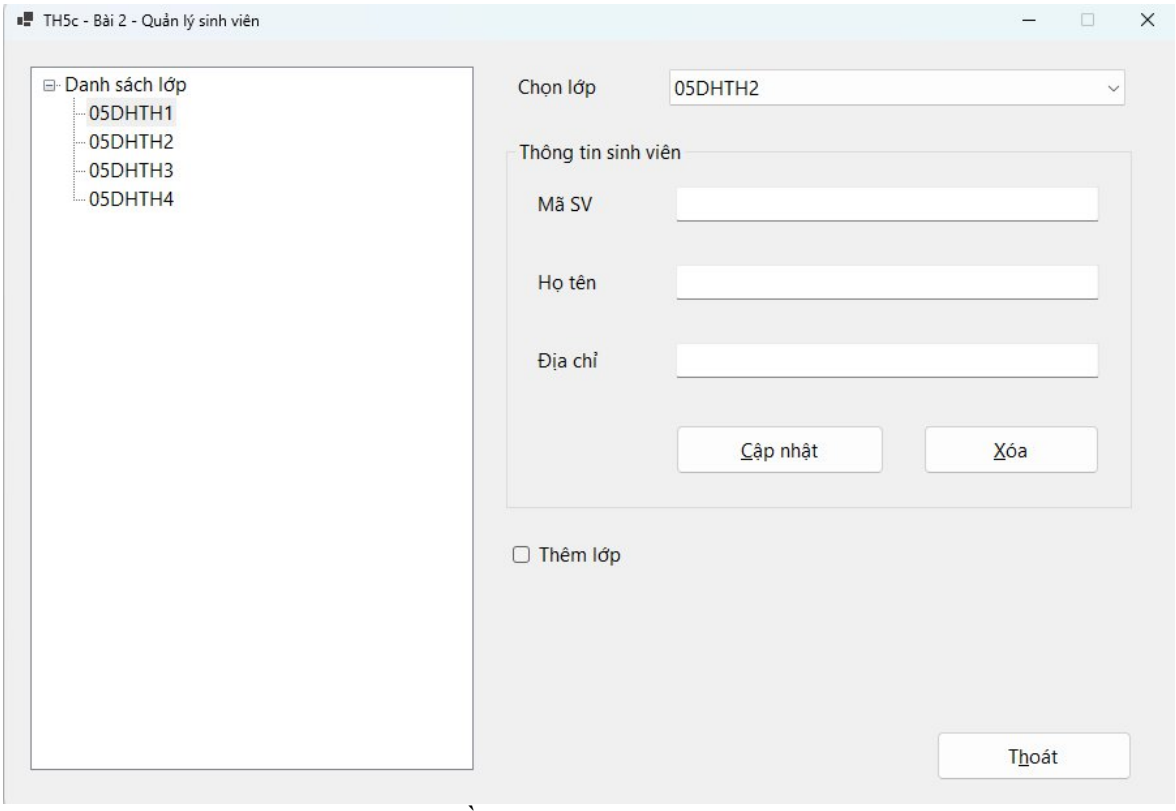
Sau khi xóa, các TextBox được làm trống. Mã sinh viên vừa xóa có thể được sử dụng để thêm lại một sinh viên.

8. Xác nhận đóng chương trình

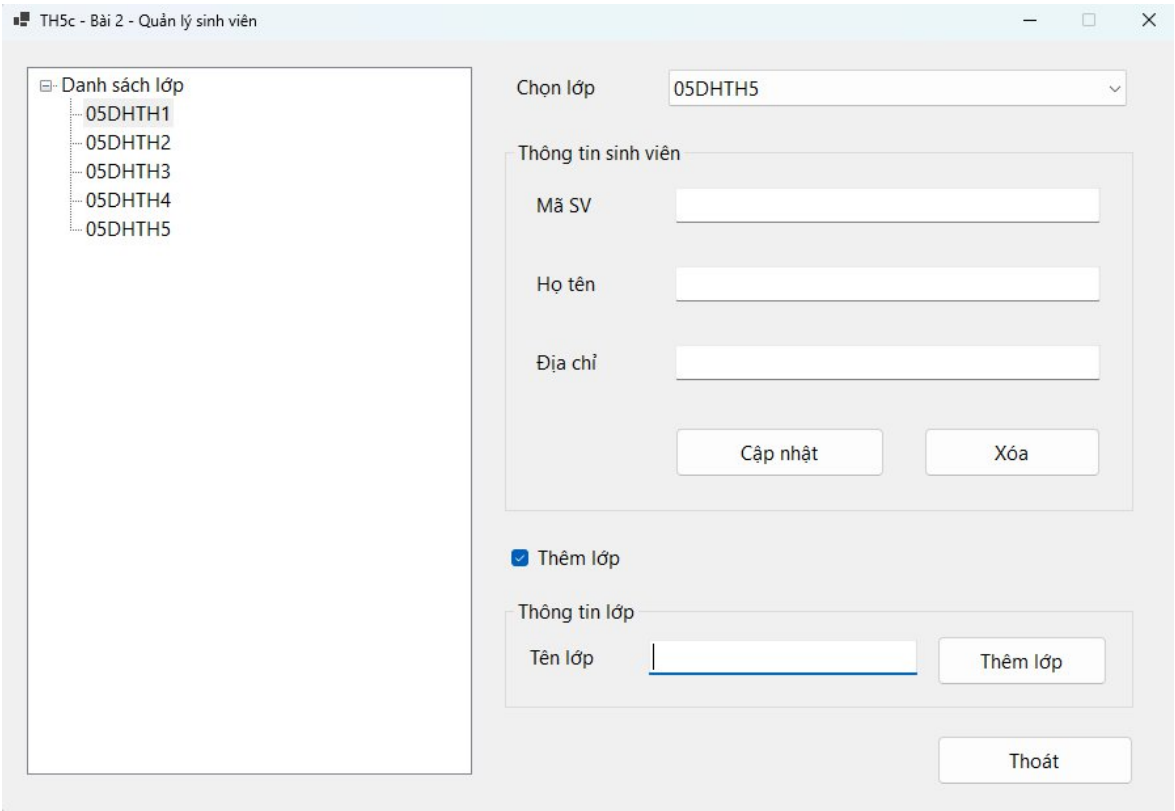
Chương trình xử lý sự kiện FormClosing để hỏi xác nhận trước khi đóng Form. Nếu người dùng chọn Yes, chương trình kết thúc. Nếu chọn No, thuộc tính e.Cancel được đặt thành true để hủy thao tác đóng.

Chức năng này áp dụng khi nhấn nút Thoát hoặc nút X trên cửa sổ.

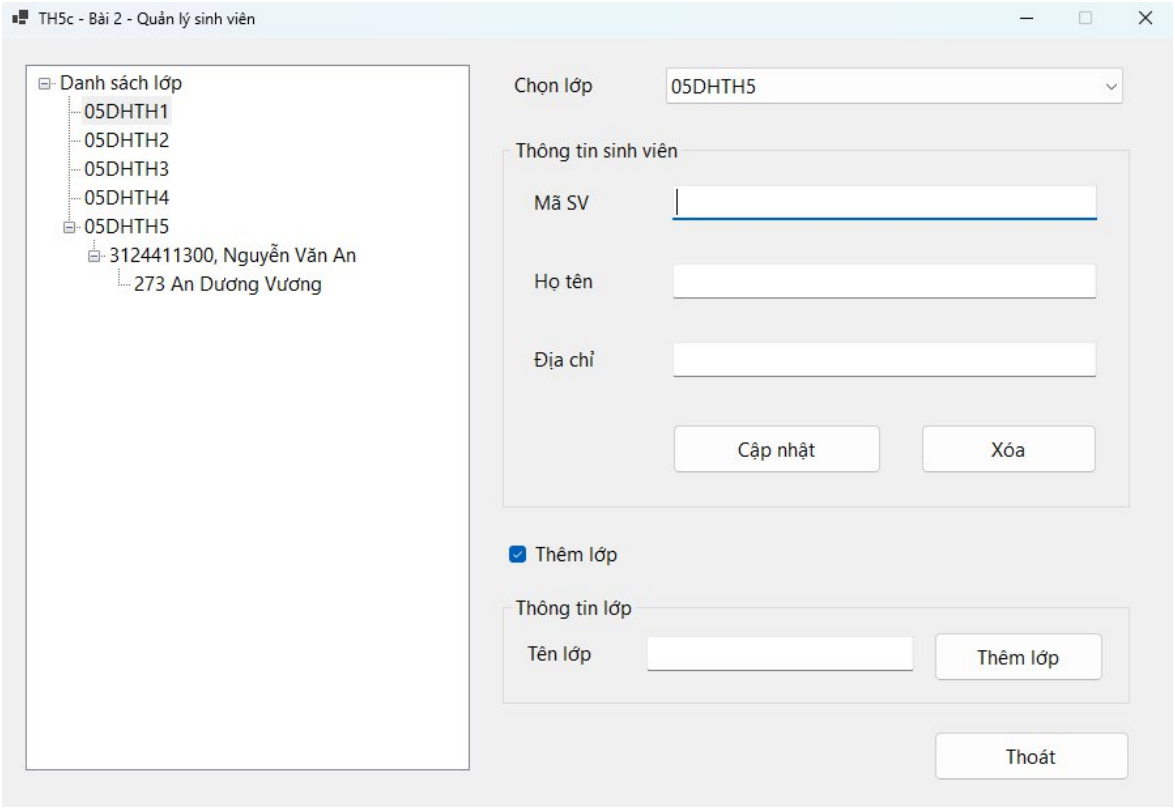
9. Hình ảnh kết quả thực hiện



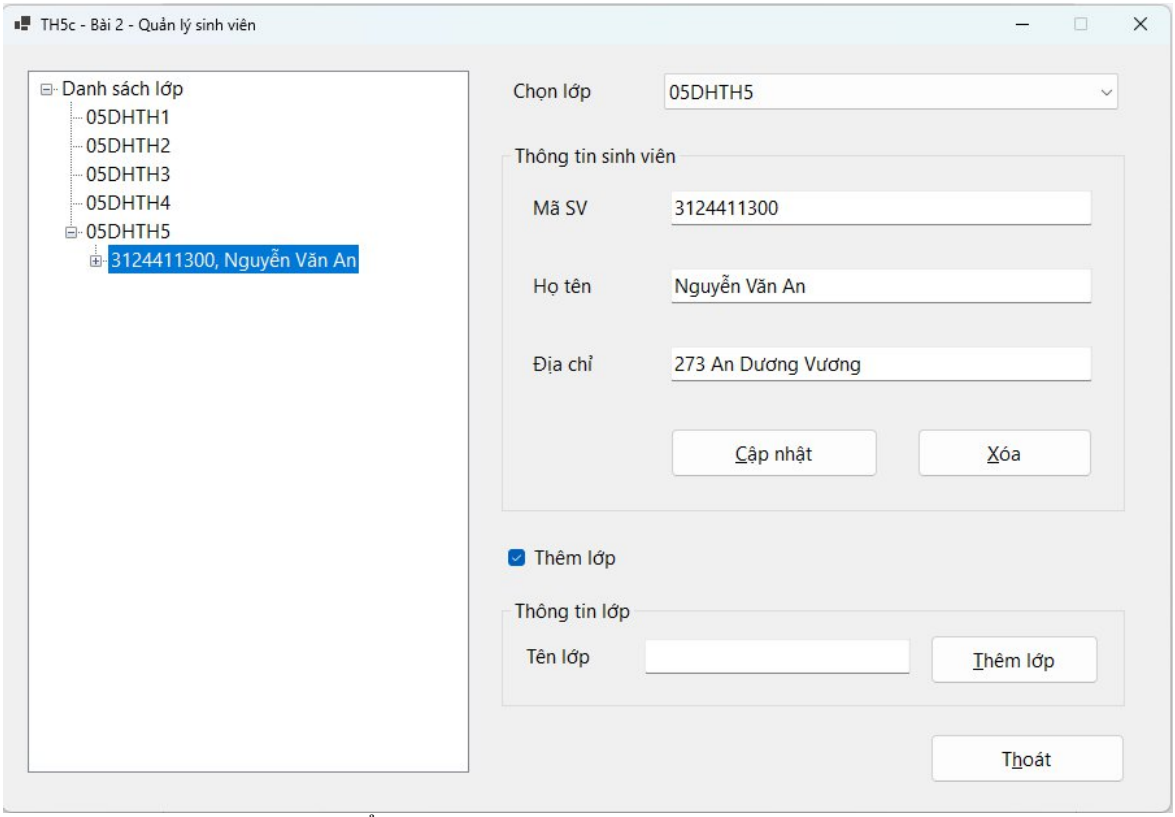
Hình 1. Giao diện ban đầu của chương trình quản lý sinh viên.



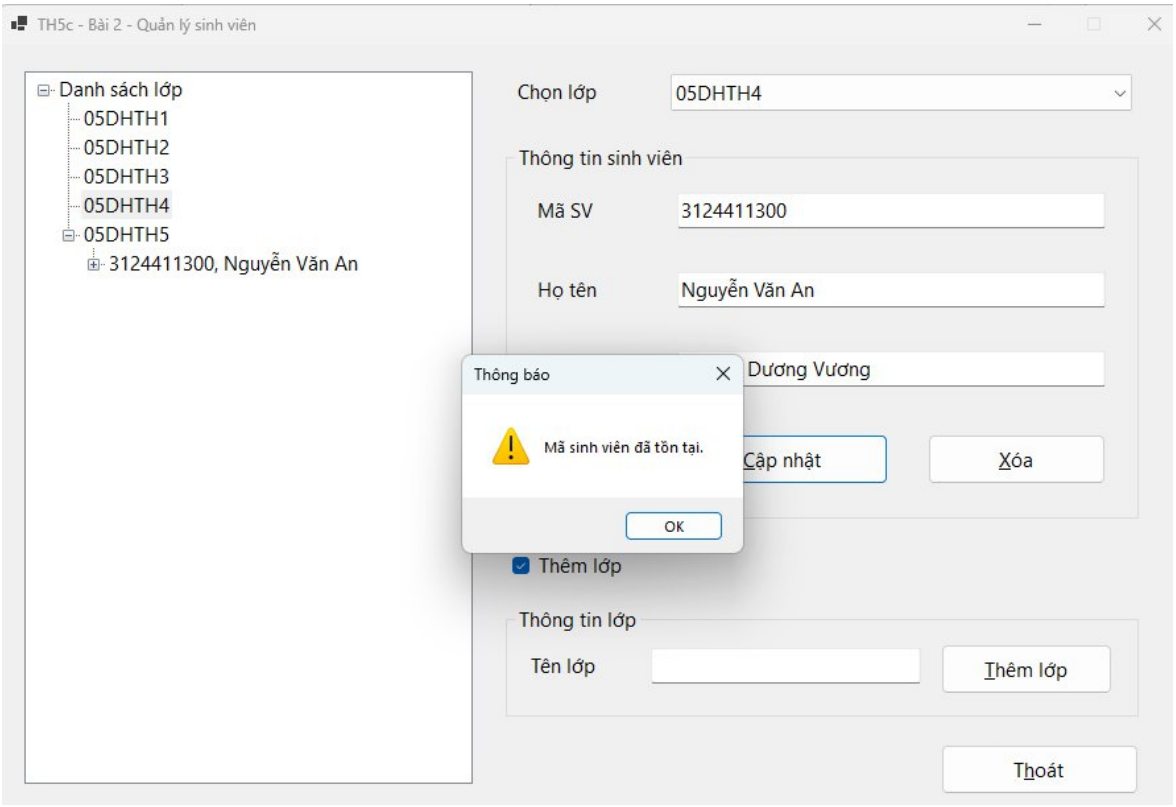
Hình 2. Thêm lớp mới vào TreeView và ComboBox.



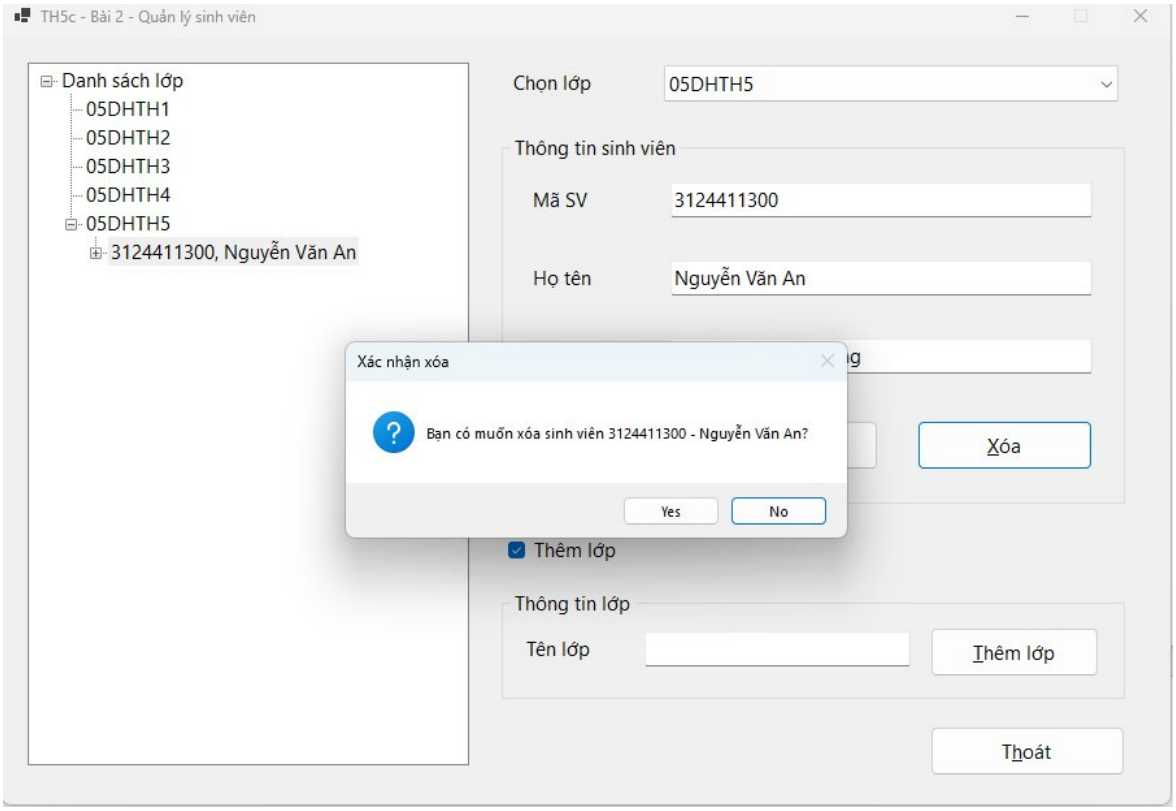
Hình 3. Sinh viên được thêm vào lớp đang chọn.



Hình 4. Hiển thị thông tin của sinh viên được chọn.



Hình 5. Thông báo khi dữ liệu bị trùng.

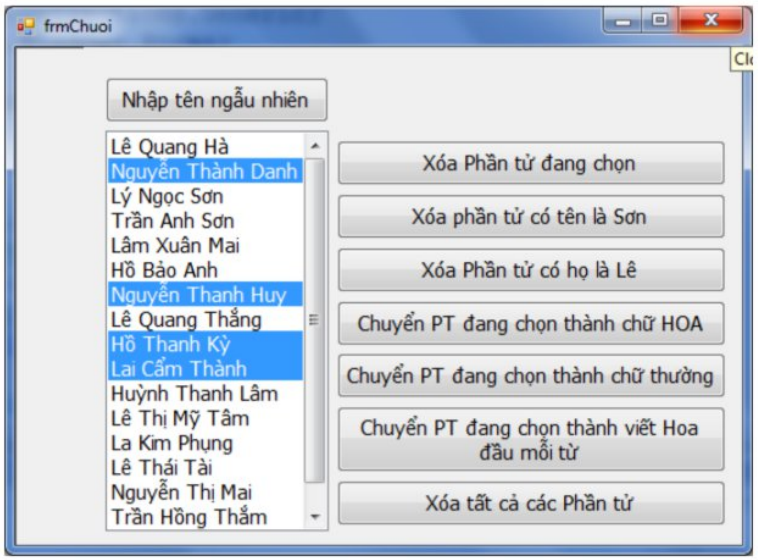


Hình 6. Xác nhận trước khi xóa sinh viên.

Bài Tập Nâng Cao - Bài 1:

3. Bài tập nâng cao

Bài 1. Thực hiện form xử lý chuỗi có giao diện nhoị sau:



10

Yêu cầu:

– btnNgauNhien: thực hiện thêm vào listbox 50 tên dựa vào mảng chuỗi HO, TENLOT, TEN được kết hợp ngẫu nhiên

HO = {“Lê”, “Nguyễn”, “Lý”, “Trần”, “Lâm”, “Hồ”, “Lai”, “Huỳnh”, “La”}

TENLOT = {“Quang”, “Thành”, “Ngọc”, “Anh”, Xuân”, “Bảo”, “Cẩm”, “Thị”, “Kim”, “Thái”, “Hồng”}

TEN = {“Hà”, “Danh”, “Sơn”, “Mai”, “Thắng”, “Kỳ”, “Thành”, “Lâm”, “Tâm”, “Phụng”, “Thắm”}

– Thực hiện các button tương ứng trên giao diện.

– Khi ngoịời dùng double click vào tên đang chọn thì sẽ mở Inputbox, cho phép ngoịời dùng thay đổi tên mới vào vị trí tên này.

1. Yêu cầu bài tập

Thiết kế chương trình tạo và xử lý danh sách họ tên bằng ListBox. Khi nhấn nút Nhập tên ngẫu nhiên, chương trình kết hợp dữ liệu từ ba mảng HO, TENLOT và TEN để thêm 50 tên vào danh sách.

Chương trình cho phép xóa các phần tử đang chọn, xóa những người có tên là Sơn, xóa những người có họ là Lê, chuyển các phần tử đang chọn thành chữ hoa, chữ thường hoặc viết hoa đầu mỗi từ. Người dùng cũng có thể xóa toàn bộ danh sách.

Khi nhấn đúp vào một tên, chương trình mở hộp nhập để sửa tên và thay nội dung ngay tại vị trí cũ.

2. Tổ chức chương trình

Bài tập được đặt trong project NangCao01_XuLyChuoi, thuộc thư mục BaiTapWinform\TH5c.

File Program.cs khởi động chương trình và mở Form frmXuLyChuoi. File frmXuLyChuoi.cs chứa giao diện chính, các mảng dữ liệu và phương thức xử lý danh sách. File frmNhapTen.cs tạo hộp nhập để người dùng sửa họ tên.

Danh sách họ tên được lưu trong thuộc tính Items của ListBox. Dữ liệu được giữ trong bộ nhớ khi chương trình đang hoạt động.

3. Thiết kế giao diện

Giao diện chính gồm ListBox bên trái và các nút xử lý bên phải. Phía trên ListBox là nút Nhập tên ngẫu nhiên.

ListBox được đặt SelectionMode là MultiExtended, cho phép chọn nhiều phần tử bằng phím Ctrl hoặc Shift. Thuộc tính HorizontalScrollbar được bật để người dùng có thể xem tên dài vượt chiều rộng của ListBox.

Hộp nhập tên gồm một TextBox, nút OK và nút Hủy. TextBox hiển thị tên hiện tại để người dùng chỉnh sửa. Nhấn Enter tương ứng với OK, còn Esc tương ứng với Hủy.

4. Tạo họ tên ngẫu nhiên

Chương trình sử dụng ba mảng HO, TENLOT và TEN theo dữ liệu trong đề bài. Mỗi họ tên được tạo bằng cách lấy ngẫu nhiên một phần tử từ từng mảng rồi ghép lại, giữa các phần có một dấu cách.

Phương thức Random.Shared.Next sinh vị trí ngẫu nhiên trong phạm vi của từng mảng. Vòng lặp được thực hiện 50 lần, mỗi lần tạo một họ tên và thêm vào ListBox.

Mỗi lần nhấn nút, chương trình thêm 50 tên vào danh sách hiện tại. Tên có thể bị trùng vì đề bài không yêu cầu các tên phải duy nhất.

BeginUpdate và EndUpdate được dùng để hạn chế việc vẽ lại ListBox trong quá trình thêm nhiều phần tử.

5. Xóa phần tử trong danh sách

Đối với chức năng xóa các phần tử đang chọn, chương trình lấy danh sách vị trí từ SelectedIndices trước khi thực hiện xóa. Những vị trí này được sắp xếp giảm dần để xóa từ cuối về đầu. Cách xử lý này tránh việc vị trí bị dịch chuyển làm xóa sai phần tử.

Đối với chức năng xóa tên Sơn và họ Lê, chương trình tách họ tên thành các từ bằng khoảng trắng. Từ cuối được xem là tên gọi, còn từ đầu được xem là họ theo cách biểu diễn họ tên của bài tập.

Việc so sánh không phân biệt chữ hoa và chữ thường. Vì vậy, các tên đã được chuyển thành chữ hoa hoặc chữ thường vẫn được kiểm tra đúng.

Chương trình duyệt danh sách từ cuối về đầu khi xóa theo điều kiện để không bỏ sót các phần tử liên tiếp. Nút Xóa tất cả gọi phương thức Items.Clear để làm trống ListBox.

6. Chuyển đổi chữ hoa và chữ thường

Các chức năng chuyển đổi chỉ tác động lên những phần tử đang được chọn. Nếu chưa chọn phần tử nào, chương trình hiển thị thông báo.

Chương trình sử dụng CultureInfo với mã vi-VN để thực hiện chuyển đổi theo quy tắc tiếng Việt. Phương thức ToUpper chuyển chuỗi thành chữ hoa, còn ToLower chuyển chuỗi thành chữ thường.

Đối với chức năng viết hoa đầu mỗi từ, chương trình chuyển chuỗi về chữ thường trước, sau đó gọi TextInfo.ToTitleCase. Bước chuyển về chữ thường giúp xử lý cả những tên đang được viết hoàn toàn bằng chữ hoa.

Nội dung mới được gán vào đúng vị trí cũ trong ListBox. Sau khi chuyển đổi, chương trình khôi phục các vị trí đang chọn để người dùng có thể tiếp tục xử lý.

7. Sửa tên bằng hộp nhập

Sự kiện MouseDoubleClick xác định vị trí phần tử mà người dùng nhấn đúp bằng phương thức IndexFromPoint. Nếu nhấn vào vùng trống, chương trình không mở hộp nhập.

Khi nhấn đúp vào một tên, Form frmNhapTen được mở bằng ShowDialog. Hộp nhập hiển thị sẵn tên cũ và cho phép người dùng thay đổi nội dung.

Nếu chọn OK, chương trình kiểm tra tên không được để trống. Các khoảng trắng thừa được loại bỏ trước khi trả tên mới về Form chính. Form chính thay nội dung tại đúng vị trí đã nhấn đúp, không thêm phần tử mới vào cuối danh sách.

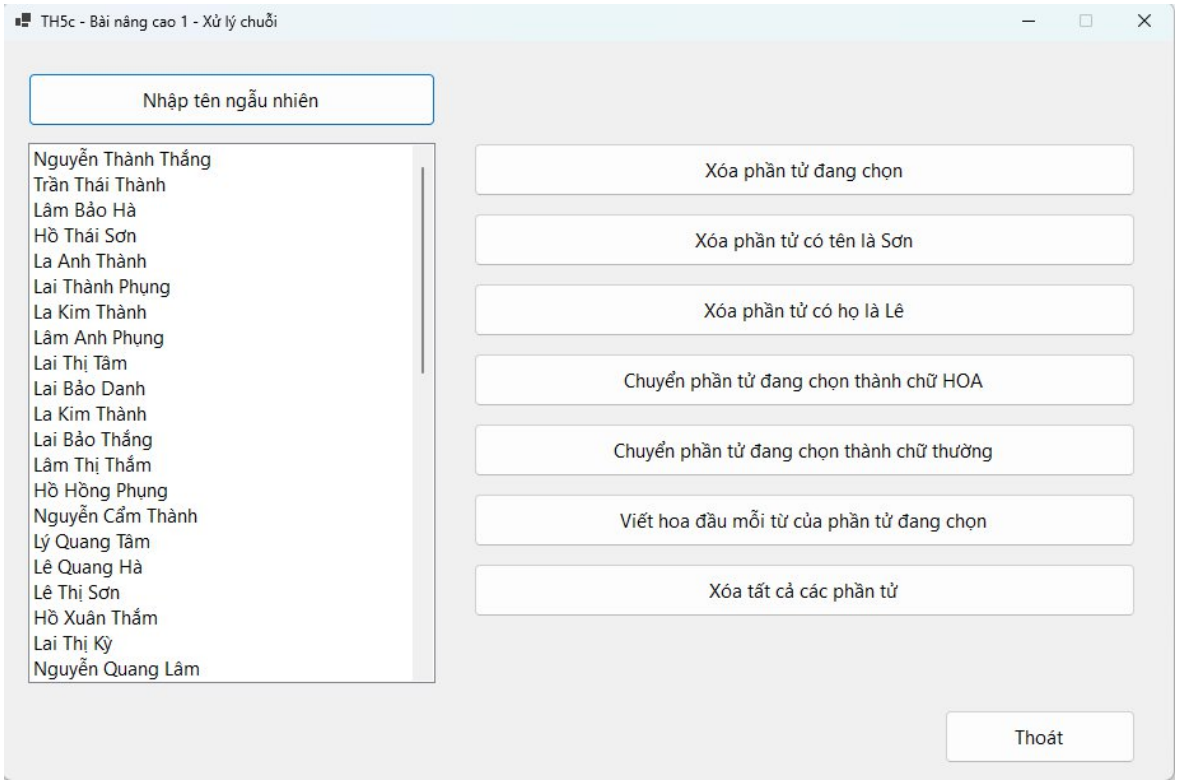
Nếu chọn Hủy hoặc đóng hộp nhập, tên cũ được giữ nguyên.

8. Xác nhận đóng chương trình

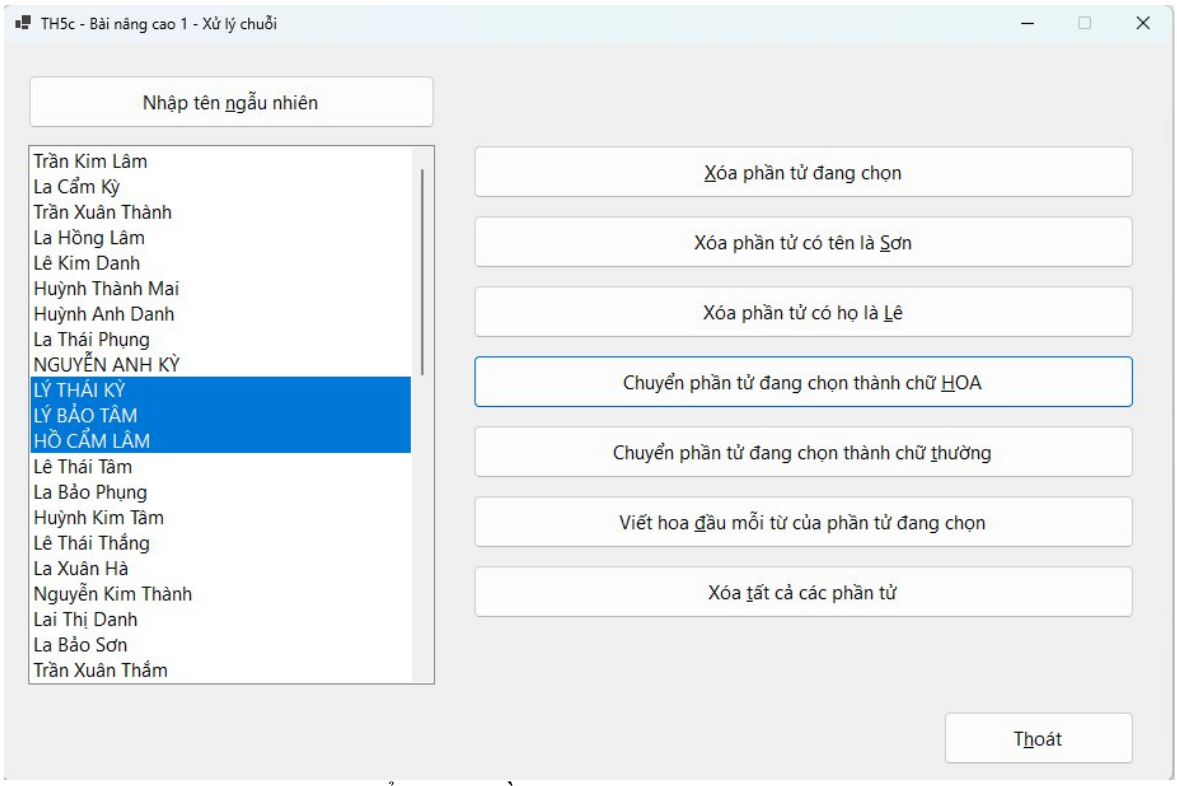
Sự kiện FormClosing hiển thị hộp thoại xác nhận trước khi đóng Form chính. Chọn Yes thì chương trình kết thúc, chọn No thì thao tác đóng được hủy bằng e.Cancel.

Hộp nhập tên không sử dụng xác nhận này. Người dùng có thể đóng hoặc chọn Hủy để trở lại danh sách mà không thay đổi tên.

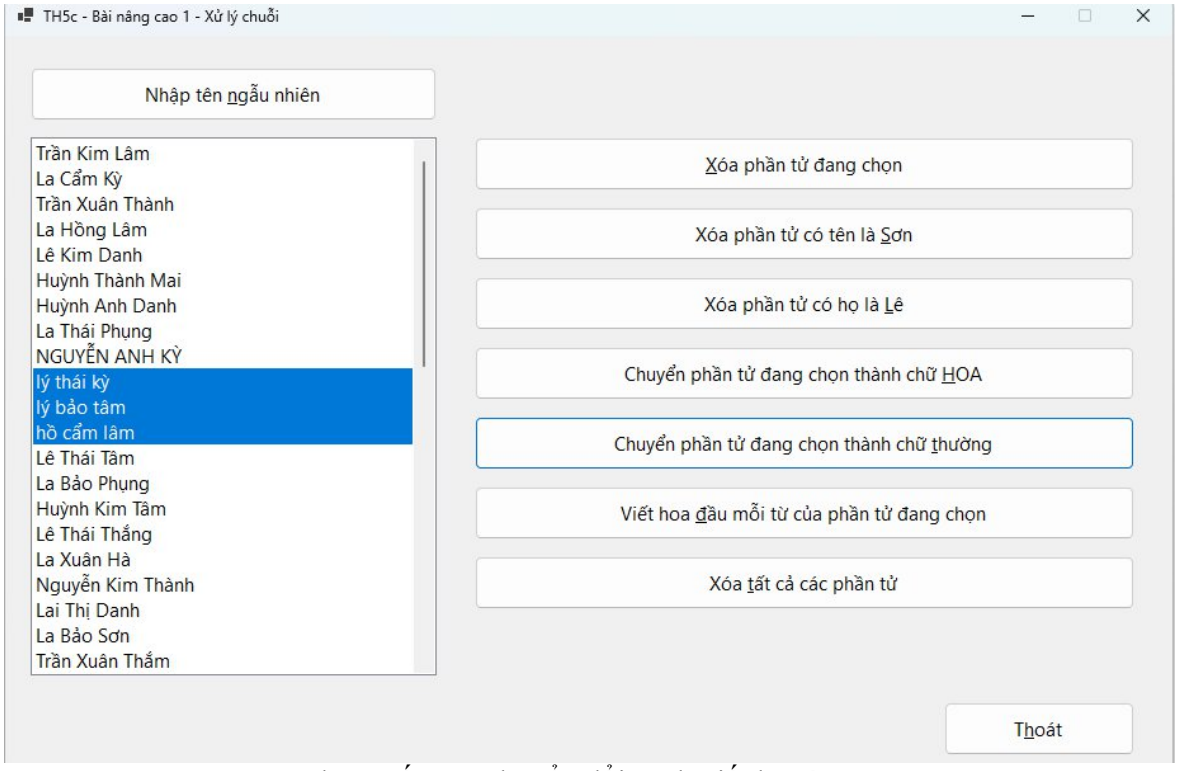
9. Hình ảnh kết quả thực hiện



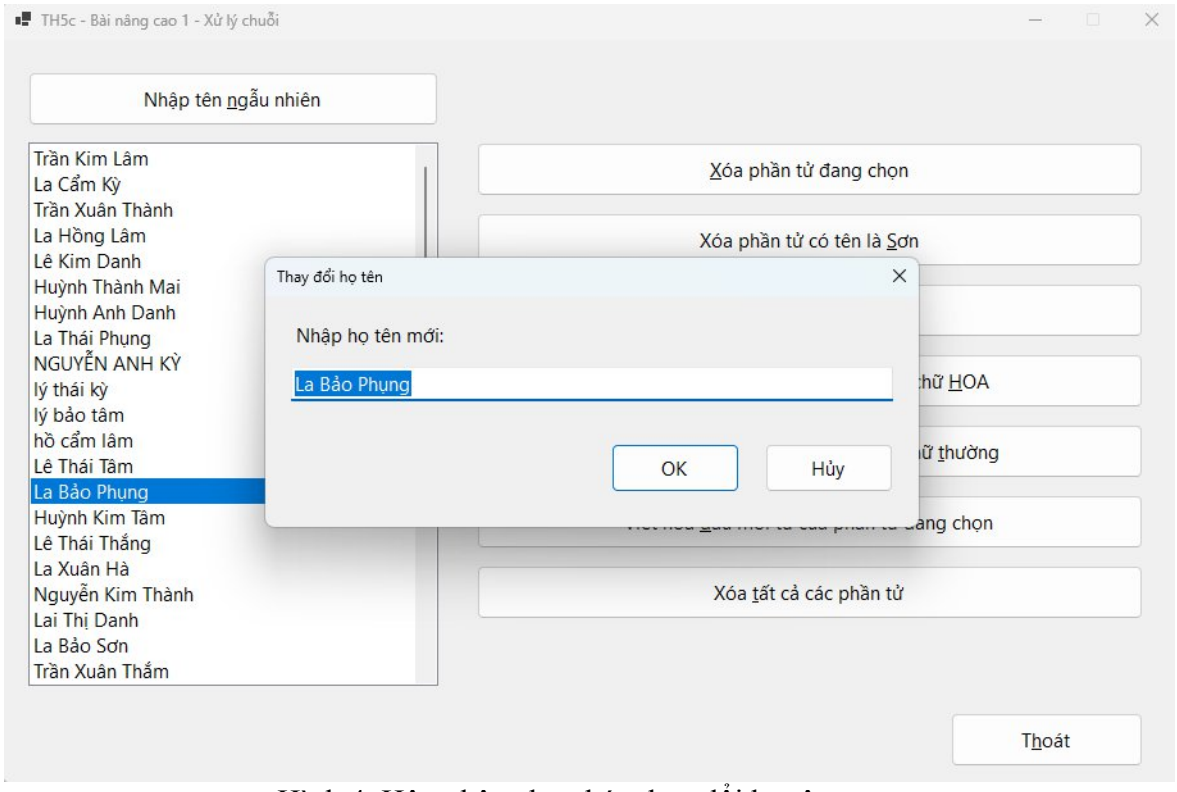
Hình 1. Danh sách họ tên được tạo ngẫu nhiên.



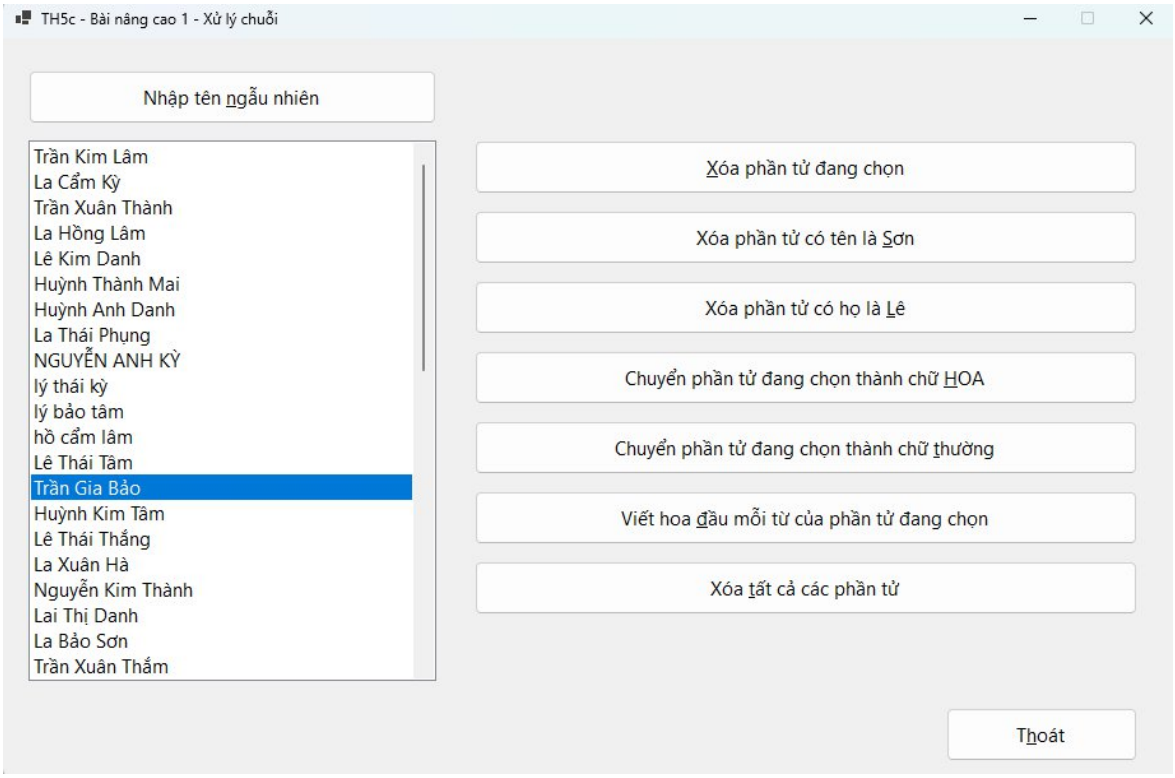
Hình 2. Chuyển các phần tử đang chọn thành chữ hoa.



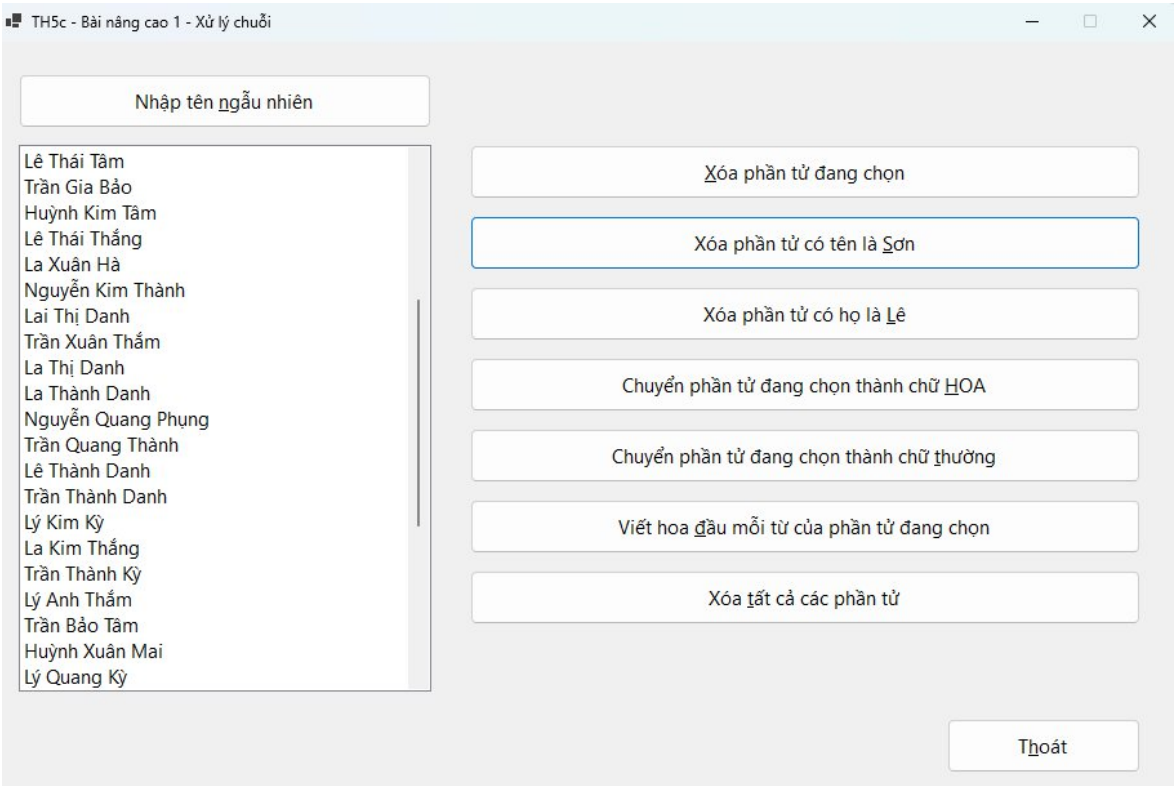
Hình 3. Kết quả chuyển đổi cách viết họ tên.



Hình 4. Hộp nhập cho phép thay đổi họ tên.



Hình 5. Tên mới được thay tại vị trí cũ.

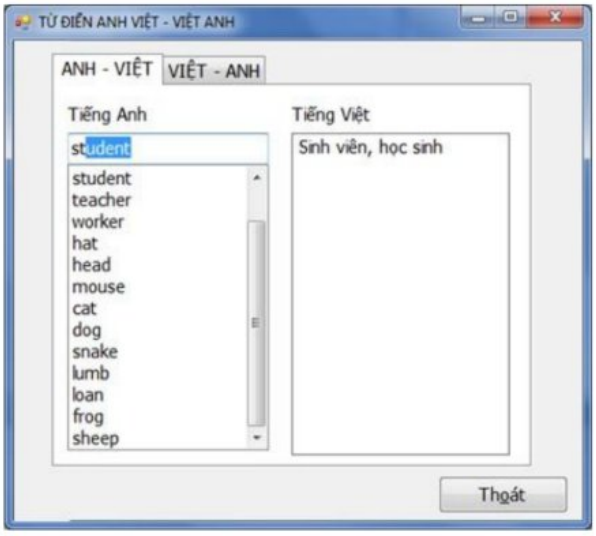


Hình 6. Xóa các phần tử thỏa mãn điều kiện.

Bài Tập Về Nhà - Bài 1:

4. Bài tập về nhà

Bài 1. Viết chương trình từ điển Anh – Việt và Việt – Anh



Yêu cầu:

– Khi nhập vào combobox từ cần tra thì chương trình sẽ dò tìm đến chữ nào khớp với ký tự gần nhất.

– Khi nhấn button Enter hoặc double click vào từ cần tra thì nghĩa tương ứng của từ sẽ hiển thị vào textbox bên phải tương ứng.

– Danh sách các từ lưu sẵn vào List

1. Yêu cầu bài tập

Thiết kế chương trình từ điển có hai chiều tra cứu Anh – Việt và Việt – Anh. Khi người dùng nhập vào ComboBox, chương trình gợi ý từ phù hợp với phần nội dung đang nhập.

Khi nhấn nút Enter, phím Enter hoặc nhấn đúp vào từ trong danh sách, nghĩa tương ứng được hiển thị trong TextBox. Danh sách từ được lưu sẵn bằng List.

2. Tổ chức chương trình

Bài tập được đặt trong project VeNha01_TuDien, thuộc thư mục BaiTapWinform\TH5c.

File Program.cs khởi động chương trình và mở Form frmTuDien. File frmTuDien.cs chứa lớp MucTu, danh sách dữ liệu, giao diện và các sự kiện tra cứu.

Lớp MucTu có hai thuộc tính là Tu và Nghia. Thuộc tính Tu lưu nội dung cần tra, còn Nghia lưu kết quả tương ứng.

Chương trình sử dụng hai List riêng cho hai chiều tra cứu. Việc tách riêng giúp biểu diễn rõ các từ đồng nghĩa và trường hợp một từ tiếng Anh có nhiều nghĩa tiếng Việt.

3. Thiết kế giao diện

Giao diện sử dụng TabControl với hai tab ANH – VIỆT và VIỆT – ANH. Mỗi tab có ComboBox nhập từ, ListBox hiển thị danh sách từ, TextBox hiển thị nghĩa và nút Enter.

ComboBox được đặt chế độ DropDown để người dùng có thể nhập hoặc chọn từ. TextBox kết quả được đặt ReadOnly để người dùng không sửa trực tiếp nghĩa hiển thị. Thuộc tính Multiline và thanh cuộn dọc hỗ trợ hiển thị nội dung dài.

Phương thức TaoTab được dùng chung cho cả hai tab. Mỗi lần gọi, chương trình truyền vào tên tab, ngôn ngữ nguồn, ngôn ngữ đích và danh sách dữ liệu tương ứng.

4. Lưu và hiển thị danh sách từ

Danh sách Anh – Việt chứa các từ mẫu như student, teacher, worker, computer, book và school. Danh sách Việt – Anh chứa các mục tương ứng như sinh viên, giáo viên, công nhân, máy tính, sách và trường học.

Trước khi hiển thị, danh sách được sắp xếp theo ngôn ngữ của từng tab. Những từ này sau đó được đưa vào cả ComboBox và ListBox theo cùng thứ tự để thuận tiện đồng bộ lựa chọn.

Dữ liệu được khai báo sẵn trong code, nên chương trình có thể hoạt động mà không cần kết nối mạng. Bộ từ mẫu có phạm vi giới hạn và có thể được mở rộng bằng cách thêm các đối tượng MucTu vào List.

5. Gợi ý khi nhập từ

ComboBox sử dụng AutoCompleteMode là SuggestAppend và AutoCompleteSource là ListItems. Khi người dùng nhập, chương trình gợi ý những từ có trong danh sách và có thể điền tiếp phần còn lại của từ.

Sự kiện TextUpdate tìm mục từ đầu tiên có phần đầu khớp với nội dung đang nhập. Chương trình chọn mục đó trên ListBox và cuộn đến vị trí tương ứng để người dùng nhìn thấy.

Việc tìm kiếm không phân biệt chữ hoa, chữ thường. Với từ tiếng Việt, chương trình sử dụng quy tắc so sánh của ngôn ngữ vi-VN.

Nếu người dùng xóa hết nội dung hoặc không có từ khớp, ListBox bỏ lựa chọn. Nghĩa cũ được xóa khi người dùng bắt đầu nhập từ mới để tránh nhầm lẫn.

6. Đồng bộ lựa chọn giữa các control

Khi người dùng chọn một từ trong ComboBox, chương trình chọn cùng từ trong ListBox. Khi chọn từ trong ListBox, chương trình đưa từ đó lên ComboBox.

Hai thao tác này có thể làm các sự kiện lựa chọn gọi lẫn nhau. Vì vậy, chương trình sử dụng biến dangDongBo để đánh dấu quá trình cập nhật giữa các control.

Khi đang gợi ý theo phần nhập, biến này cũng ngăn sự kiện của ListBox tự thay toàn bộ nội dung mà người dùng đang gõ. Nhờ đó, người dùng có thể tiếp tục nhập từng ký tự.

7. Tra cứu và hiển thị nghĩa

Các thao tác nhấn nút Enter, nhấn phím Enter và nhấn đúp vào từ đều gọi chung hàm TraTu.

Hàm này lấy từ trong ComboBox và loại bỏ khoảng trắng ở đầu, cuối. Nếu nội dung trống, chương trình yêu cầu nhập hoặc chọn từ.

Nếu có nội dung, chương trình tìm mục từ khớp đầy đủ và không phân biệt chữ hoa, chữ thường. Khi tìm thấy, nghĩa được đưa vào TextBox. Nếu không tìm thấy, TextBox hiển thị thông báo “Không tìm thấy từ trong danh sách”.

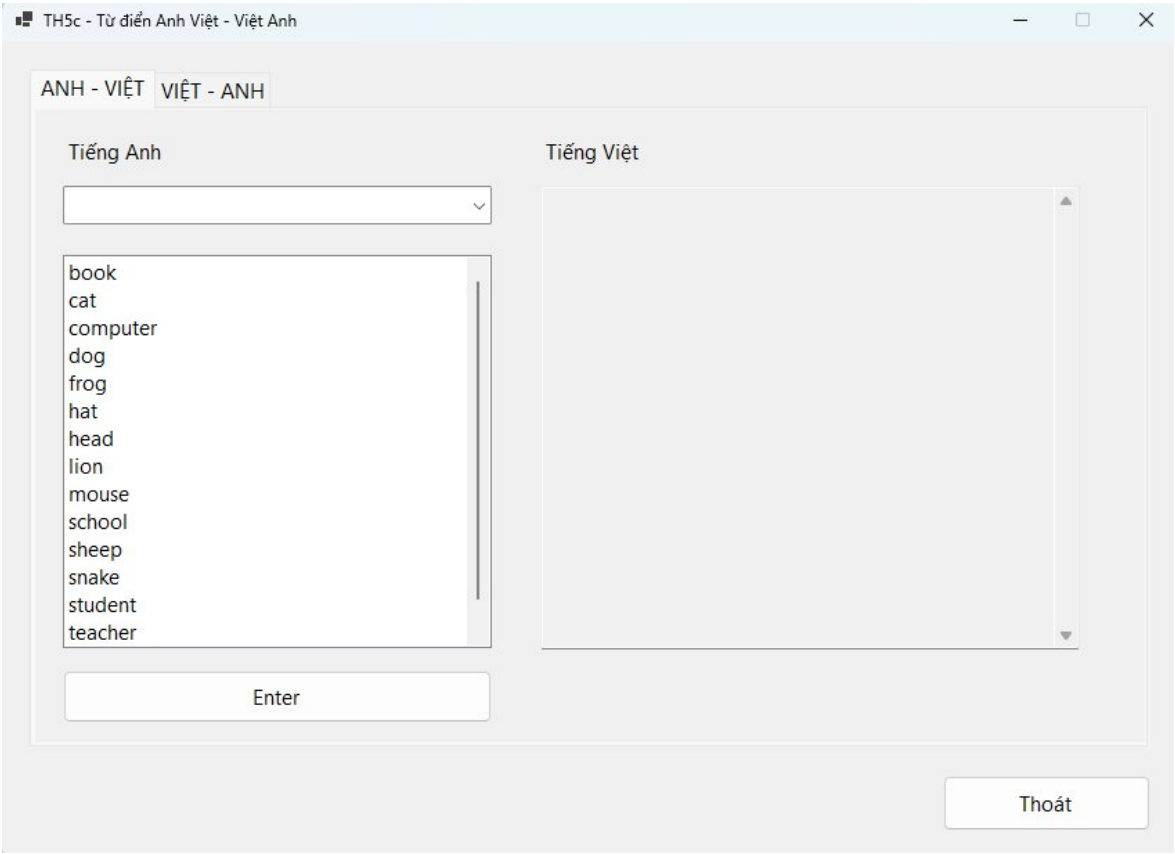
Khi nhấn đúp trong ListBox, chương trình xác định phần tử tại vị trí con trỏ bằng IndexFromPoint. Nhấn vào vùng trống không thực hiện tra cứu. Nhấn vào một từ sẽ chọn đúng từ đó trước khi hiển thị nghĩa.

8. Xác nhận đóng chương trình

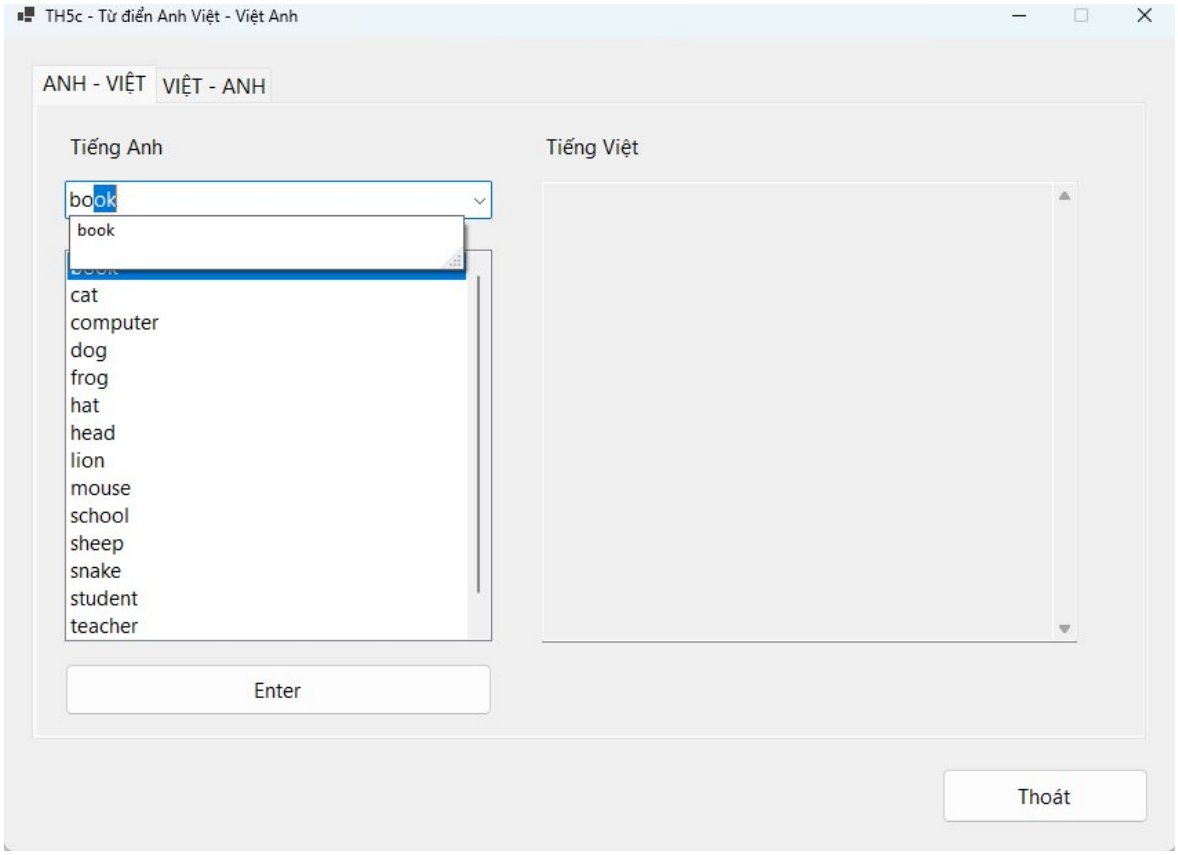
Chương trình sử dụng sự kiện FormClosing để hỏi xác nhận trước khi đóng cửa sổ.

Nếu chọn Yes, chương trình kết thúc. Nếu chọn No, chương trình đặt e.Cancel bằng true để hủy thao tác đóng. Chức năng này áp dụng cho cả nút Thoát và nút X trên cửa sổ.

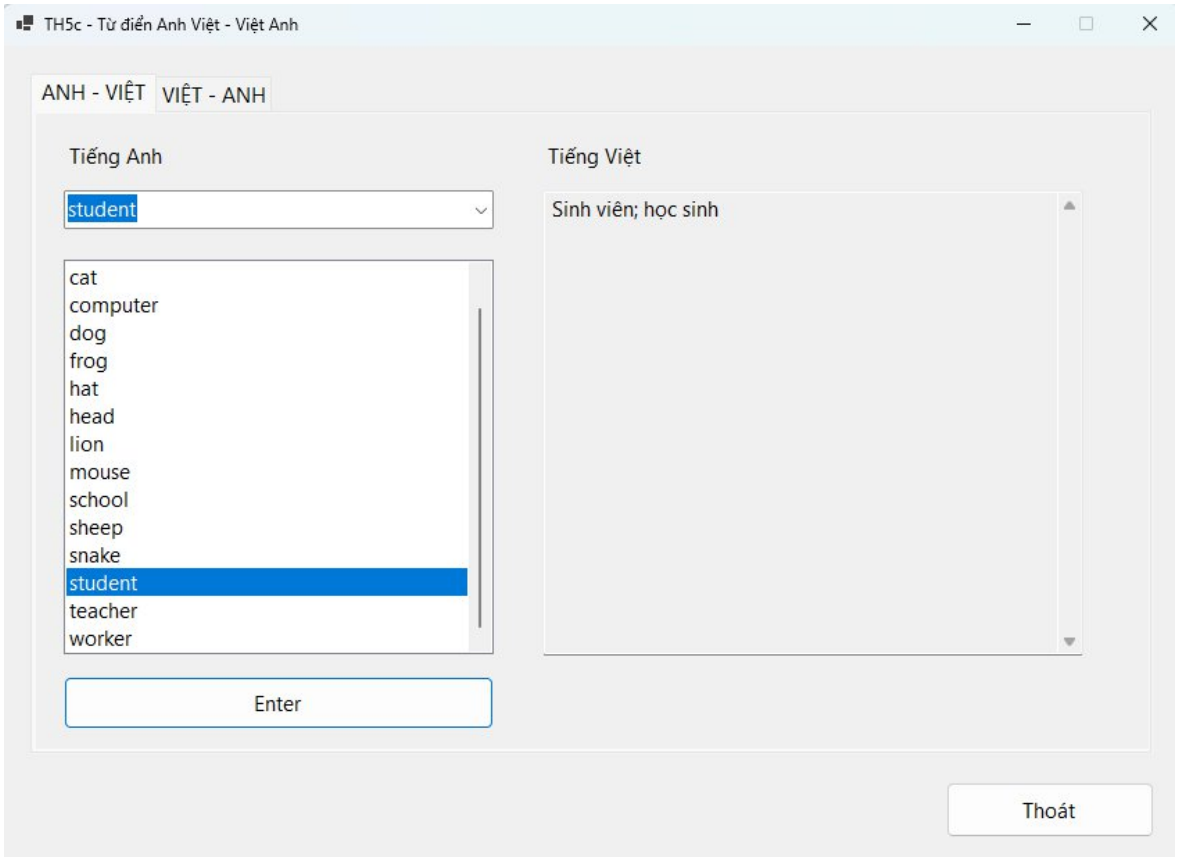
9. Hình ảnh kết quả thực hiện



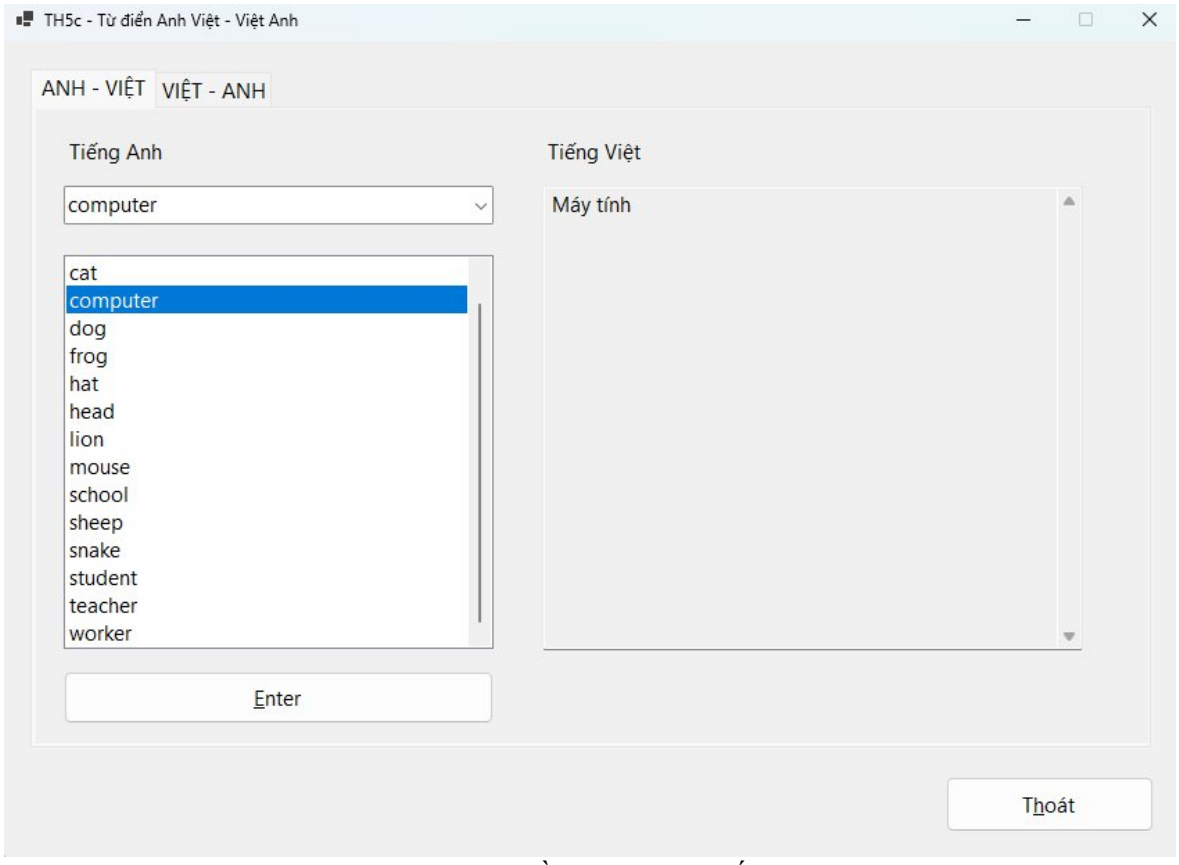
Hình 1. Giao diện tra cứu Anh – Việt.



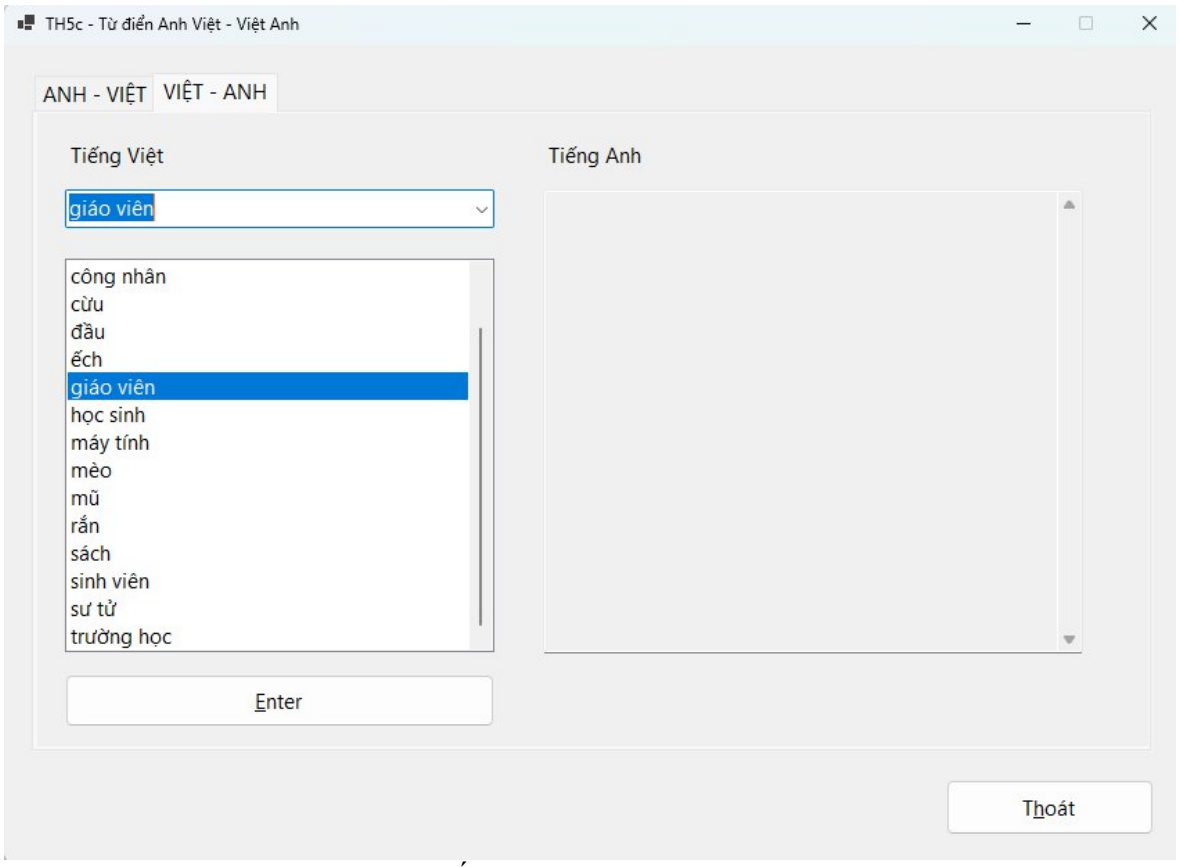
Hình 2. Gợi ý từ theo nội dung đang nhập.



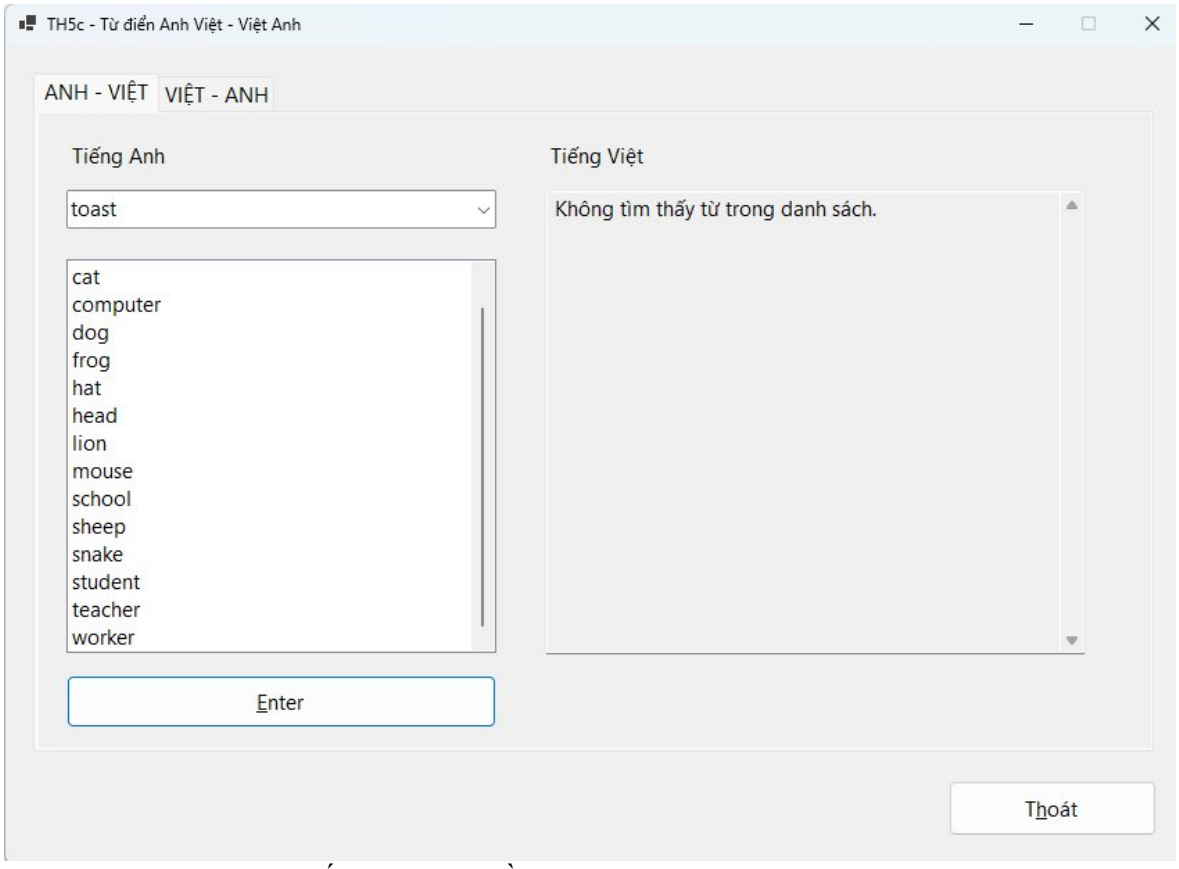
Hình 3. Kết quả tra nghĩa của từ student.



Hình 4. Tra nghĩa bằng thao tác nhấn đúp.



Hình 5. Kết quả tra cứu Việt – Anh.



Hình 6. Kết quả khi từ cần tra không có trong danh sách.

Bài Tập Về Nhà - Bài 2:

– Danh sách các từ lọu săn vào List

Bài 2. Viết chương trình thêm các phần tử vào listbox (listbox được chọn nhiều phần tử) các số tự nhiên N được nhập từ textbox.

11



Yêu cầu: các button lệnh thực hiện các công việc sau:

- Tính tổng các phần tử trong Listbox, hiển thị lên Messagebox.
- Xóa phần tử đầu và cuối của listbox.
- Xóa các phần tử đang chọn trong listbox.
- Tăng giá trị mỗi phần tử lên 2.
- Thay mỗi giá trị của mỗi phần tử bằng bình pương của chính nó
- Thực hiện chọn các phần tử trong listbox là số chẵn.
- Thực hiện chọn các phần tử trong listbox là số lẻ.

1. Yêu cầu bài tập

Thiết kế chương trình cho phép nhập các số tự nhiên từ TextBox và thêm vào ListBox. ListBox phải cho phép chọn nhiều phần tử.

Các nút xử lý thực hiện tính tổng các phần tử, xóa phần tử đầu và cuối, xóa những phần tử đang chọn, tăng mỗi phần tử lên 2, thay mỗi phần tử bằng bình phương của nó và chọn các phần tử chẵn hoặc lẻ.

Kết quả tính tổng được hiển thị bằng MessageBox.

2. Tổ chức chương trình

Bài tập được đặt trong project VeNha02_ListBoxSo, thuộc thư mục BaiTapWinform\TH5c.

File Program.cs khởi động chương trình và mở Form frmListBoxSo. File frmListBoxSo.cs chứa giao diện, kiểm tra dữ liệu và các phương thức xử lý danh sách.

Các số được lưu trực tiếp trong Items của ListBox dưới dạng BigInteger. Kiểu dữ liệu này hỗ trợ số nguyên lớn, giúp tránh tràn số do giới hạn của int hoặc long khi thực hiện bình phương nhiều lần.

Dữ liệu chỉ được lưu trong bộ nhớ trong thời gian chương trình hoạt động.

3. Thiết kế giao diện

Giao diện gồm hai nhóm chính. Nhóm Danh sách số chứa TextBox nhập số, nút Nhập và ListBox hiển thị dữ liệu. Nhóm Xử lý ListBox chứa các nút thực hiện yêu cầu của bài tập.

ListBox được đặt SelectionMode là MultiExtended. Người dùng có thể giữ Ctrl để chọn nhiều phần tử riêng lẻ hoặc giữ Shift để chọn một dãy phần tử.

Thuộc tính HorizontalScrollbar được bật để hỗ trợ xem các giá trị có nhiều chữ số. Nút Kết thúc được đặt phía dưới giao diện.

4. Nhập số và kiểm tra dữ liệu

Trong bài giải, số tự nhiên gồm số 0 và các số nguyên dương.

Sự kiện KeyPress chỉ cho phép người dùng gõ các chữ số từ 0 đến 9 và các ký tự điều khiển như Backspace. Khi nhấn Nhập, chương trình tiếp tục kiểm tra nội dung bằng điều kiện chỉ chứa chữ số và phương thức BigInteger.TryParse.

Bước kiểm tra khi nhấn nút xử lý cả trường hợp người dùng dán dữ liệu vào TextBox. Nội dung trống, số âm, số thập phân hoặc chuỗi chứa chữ cái đều không được thêm vào danh sách.

Nếu dữ liệu hợp lệ, số được thêm vào ListBox. Chương trình xóa ô nhập và đặt con trỏ trở lại TextBox. Phím Enter cũng thực hiện chức năng nhập nhờ thuộc tính AcceptButton.

Các số trùng nhau vẫn được phép thêm vì đề bài không yêu cầu kiểm tra trùng.

5. Tính tổng các phần tử

Khi nhấn nút Tổng các phần tử trong List, chương trình kiểm tra danh sách có dữ liệu hay không. Nếu danh sách rỗng, chương trình thông báo và yêu cầu nhập số trước.

Nếu có dữ liệu, chương trình duyệt toàn bộ các phần tử, cộng lần lượt vào biến tổng kiểu BigInteger và hiển thị kết quả bằng MessageBox.

Phép tính áp dụng cho toàn bộ danh sách, không chỉ những phần tử đang được chọn.

6. Xóa phần tử đầu cuối và các phần tử đang chọn

Chức năng xóa đầu và cuối thực hiện xóa phần tử cuối trước, sau đó xóa phần tử đầu nếu danh sách vẫn còn dữ liệu.

Với danh sách chỉ có một phần tử, chương trình chỉ xóa một lần. Với danh sách có hai phần tử, cả hai được xóa và danh sách trở thành rỗng.

Chức năng xóa các phần tử đang chọn lấy những vị trí từ SelectedIndices và sắp xếp giảm dần. Sau đó, chương trình xóa từ cuối về đầu để tránh việc vị trí bị dịch chuyển trong quá trình xóa.

Nếu chưa chọn phần tử nào, chương trình hiển thị thông báo và giữ nguyên danh sách.

7. Tăng giá trị và thay bằng bình phương

Hai chức năng tăng mỗi phần tử lên 2 và thay bằng bình phương sử dụng chung phương thức BienDoiDanhSach.

Với chức năng tăng lên 2, mỗi giá trị n được thay bằng n + 2. Với chức năng bình phương, mỗi giá trị được thay bằng n nhân n. Cả hai chức năng đều tác động lên toàn bộ danh sách.

Chương trình tính trước các giá trị mới, sau đó nạp lại ListBox. Các vị trí đang chọn và vị trí cuộn được lưu lại rồi khôi phục sau khi cập nhật.

BeginUpdate và EndUpdate hạn chế việc vẽ lại trong quá trình nạp dữ liệu. Refresh yêu cầu giao diện hiển thị ngay các giá trị mới.

Mỗi thao tác sử dụng giá trị hiện tại. Vì vậy, nếu người dùng tăng các số lên 2 rồi nhấn bình phương, chương trình bình phương những số đã được tăng.

8. Chọn số chẵn số lẻ và kết thúc chương trình

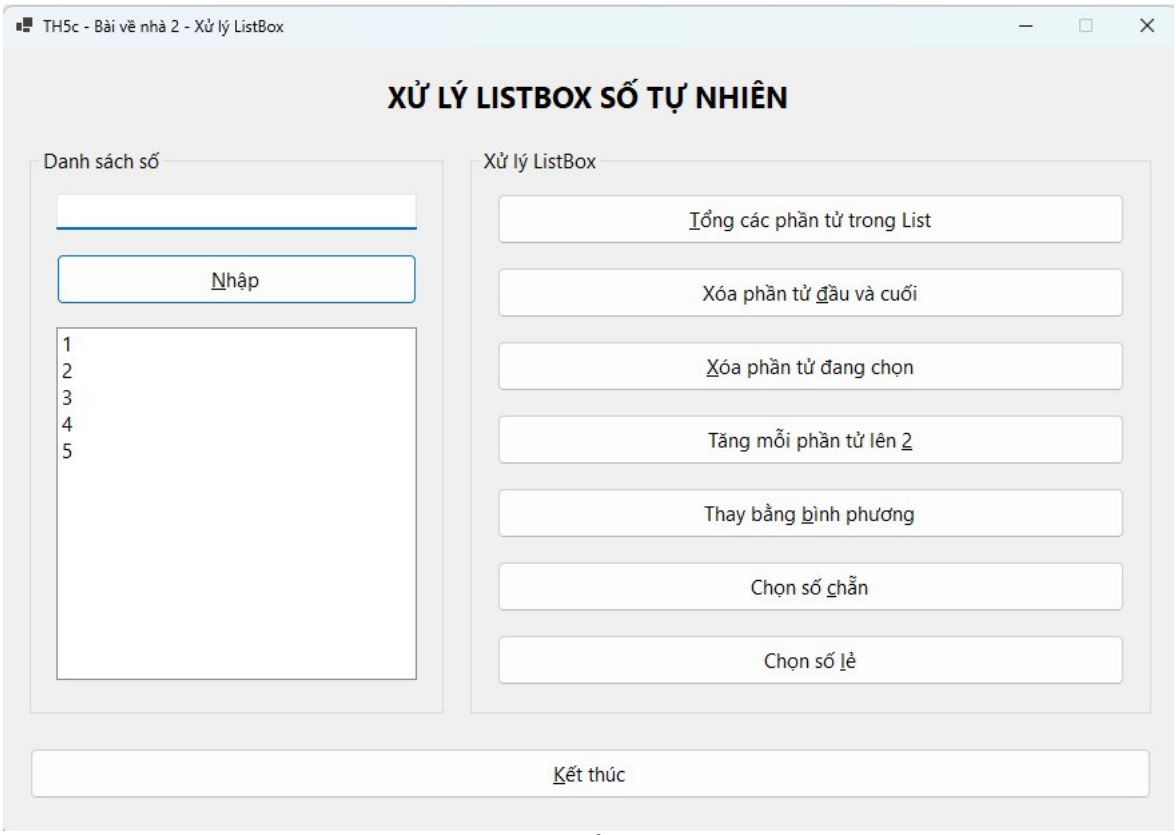
Khi nhấn Chọn số chẵn hoặc Chọn số lẻ, chương trình bỏ toàn bộ lựa chọn cũ rồi duyệt danh sách.

Thuộc tính IsEven của BigInteger xác định tính chẵn lẻ. Nếu giá trị phù hợp với yêu cầu, chương trình gọi SetSelected để chọn phần tử tại vị trí tương ứng. Số 0 được xem là số chẵn.

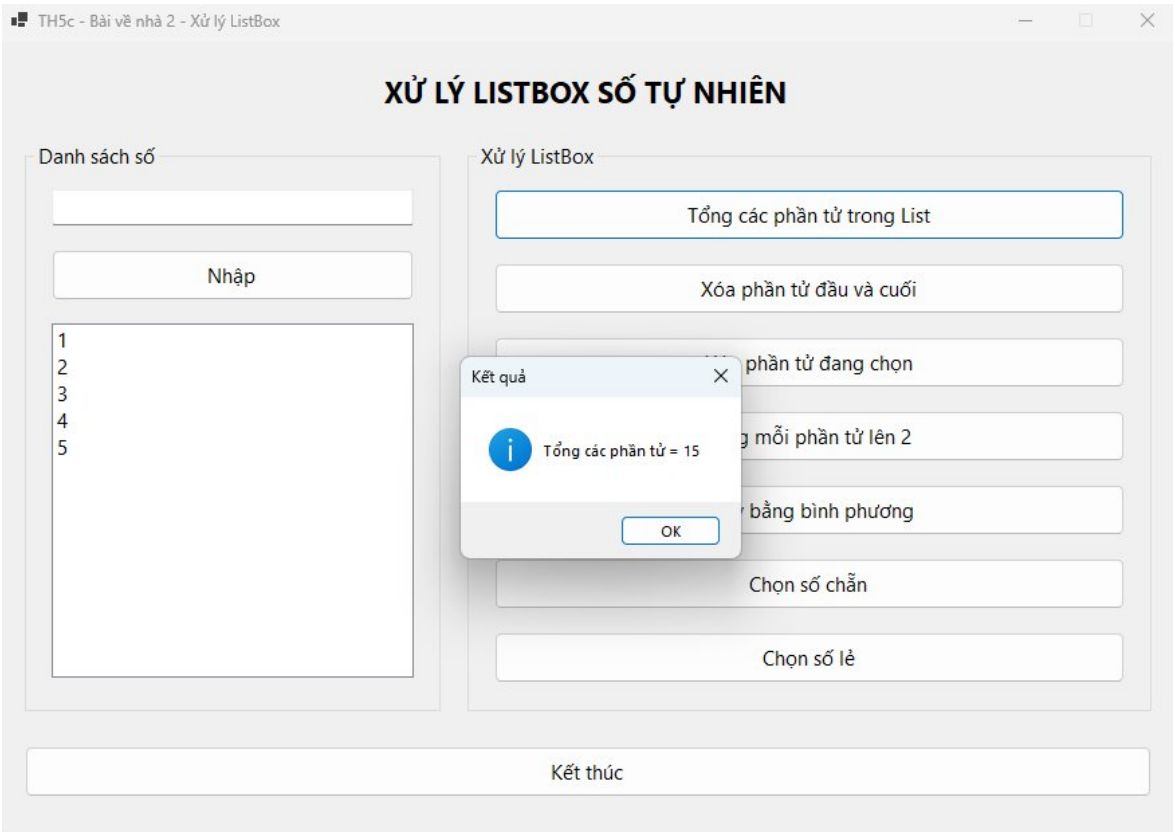
Các nút này chỉ thay đổi trạng thái lựa chọn, không xóa hoặc thay đổi giá trị của phần tử.

Khi kết thúc chương trình, sự kiện FormClosing hiển thị hộp thoại xác nhận. Chọn Yes thì chương trình đóng; chọn No thì thao tác đóng được hủy.

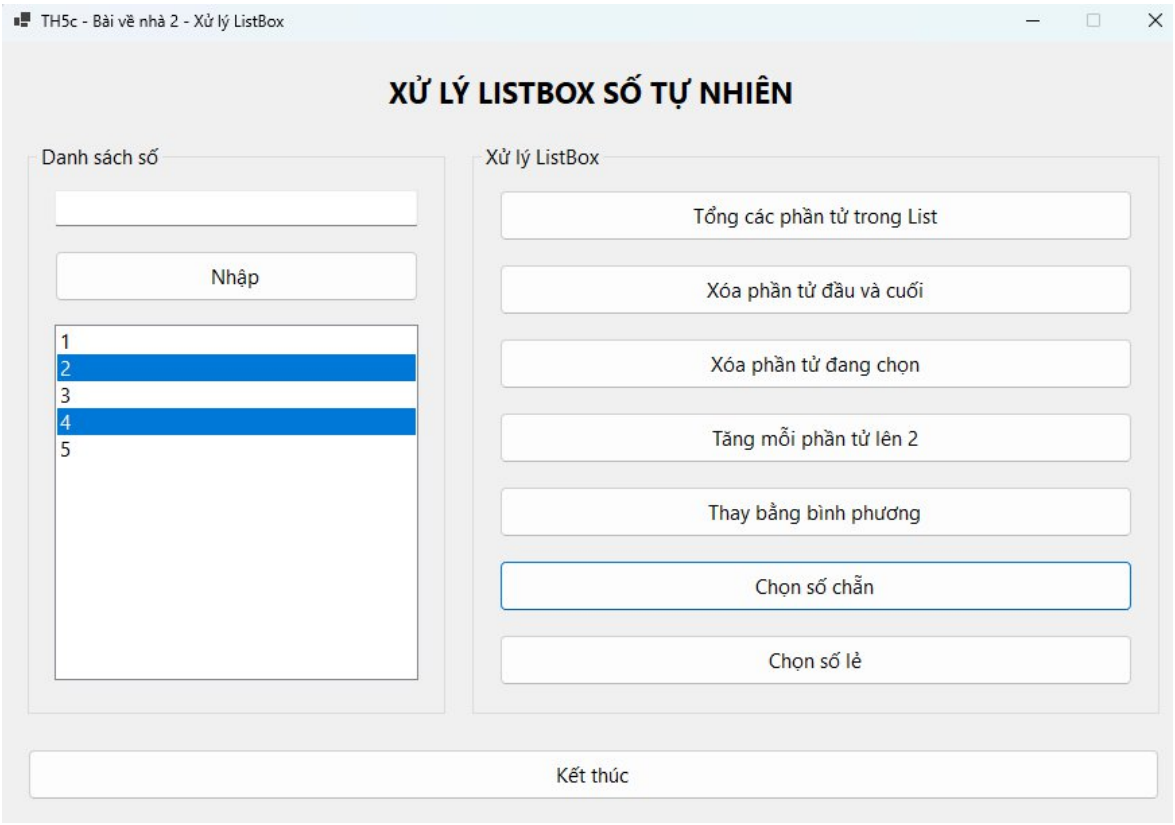
9. Hình ảnh kết quả thực hiện



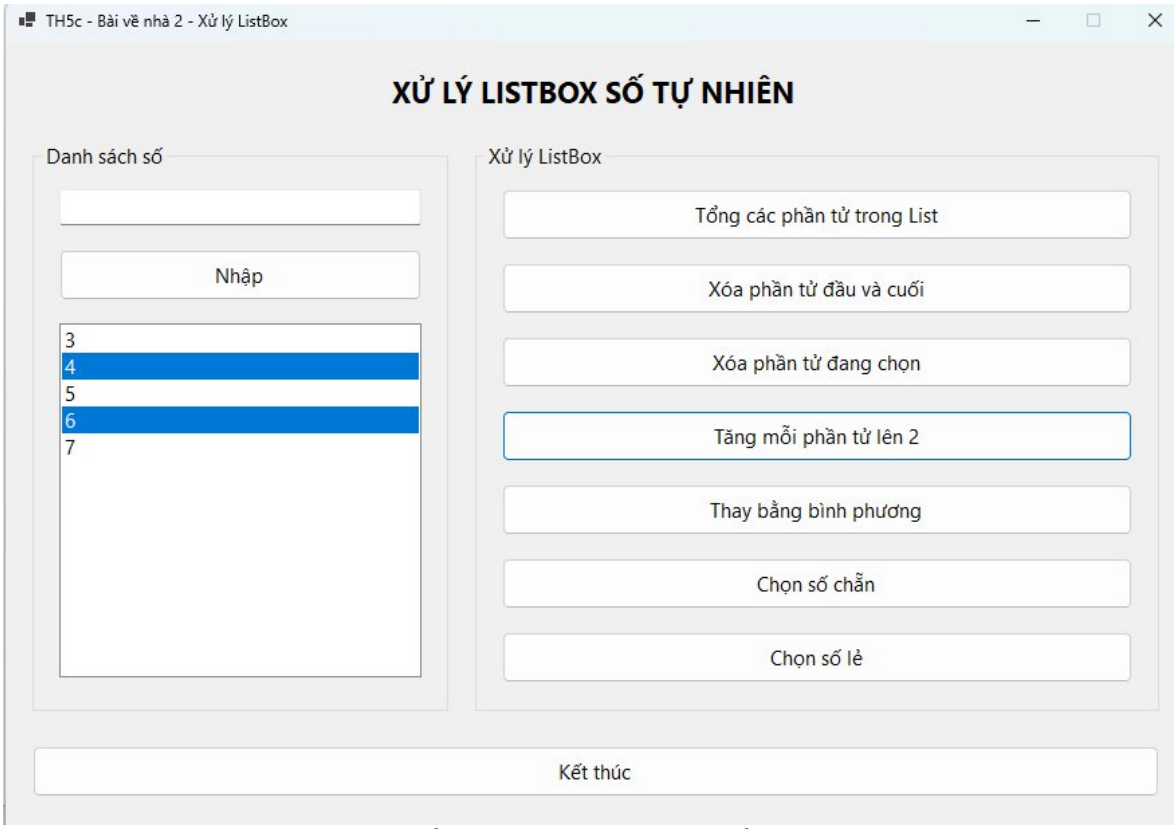
Hình 1. Danh sách các số tự nhiên đã nhập.



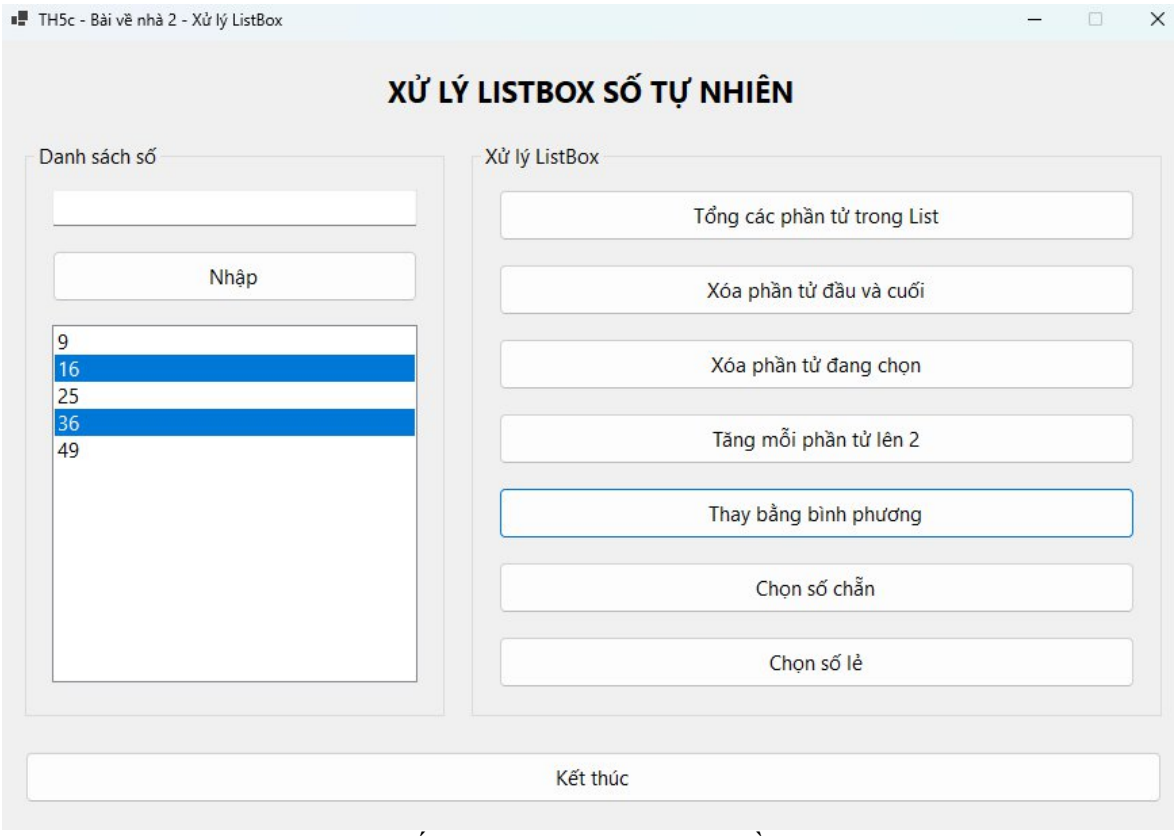
Hình 2. Kết quả tính tổng các phần tử.



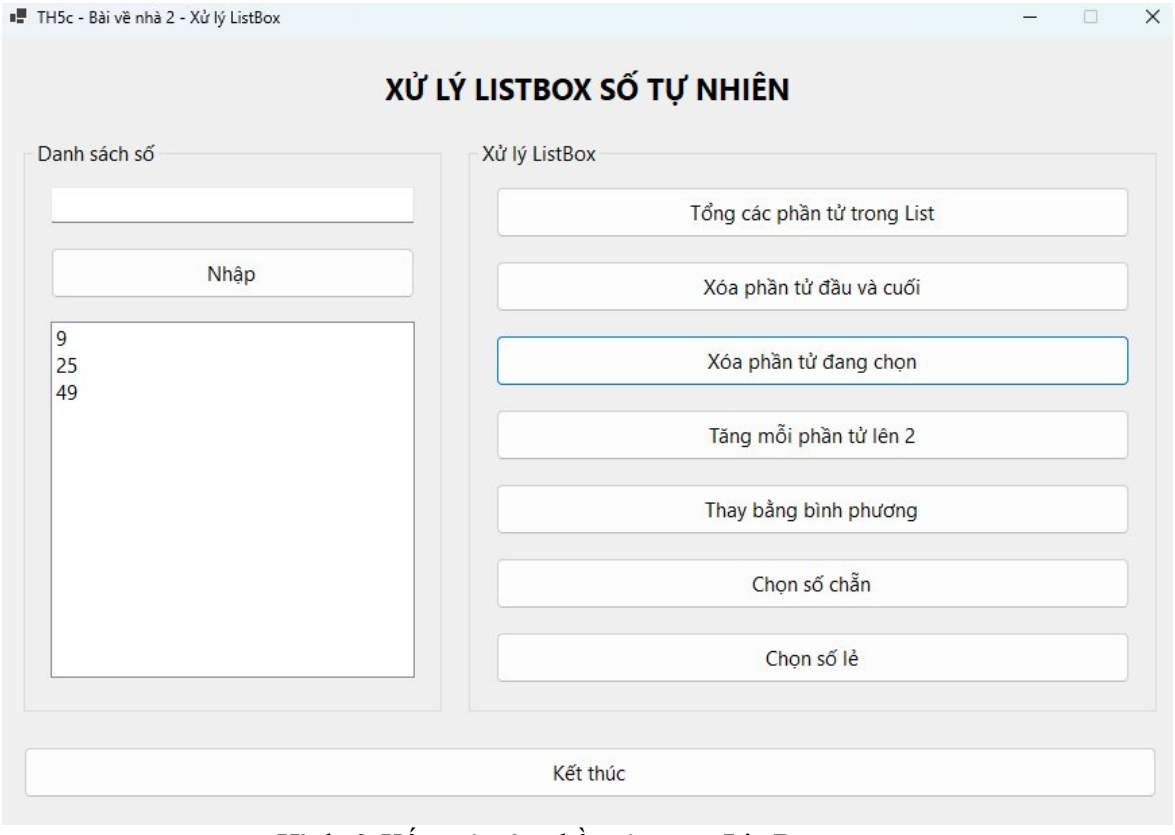
Hình 3. Lựa chọn các phần tử theo tính chẵn lẻ.



Hình 4. Kết quả tăng giá trị các phần tử.



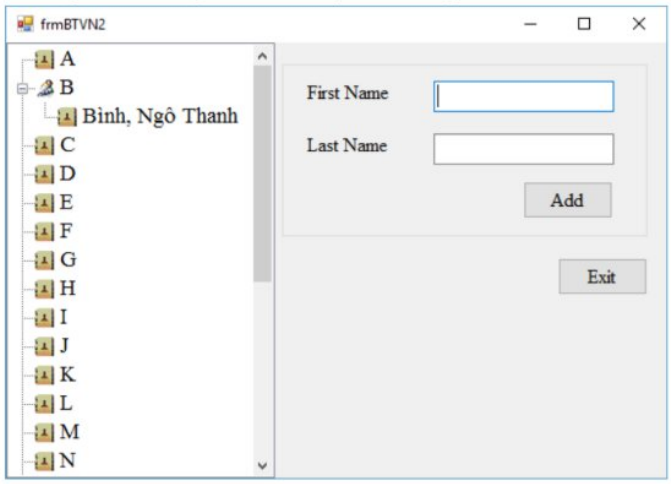
Hình 5. Kết quả bình phương các phần tử.



Hình 6. Kết quả xóa phần tử trong ListBox.

Bài Tập Về Nhà - Bài 3:

Bài 3. Thiết kế giao diện nhập danh bạ với yêu cầu nhoị sau:



12

Yêu cầu:

– Khi form được load lên treeView chứa tất cả các ký tự từ A→ Z.

– Nhằm mục đích tiện lợi cho ngoịời sử dụng, khi muốn thêm một ngoịời mới vào danh bạ, chương trình sẽ đoịa ngoịời này vào treeView ở vị trí node tương ứng với chữ cái đầu của tên (xem hình).

1. Yêu cầu bài tập

Thiết kế chương trình nhập danh bạ bằng TreeView. Khi Form được mở, TreeView phải có sẵn các node từ A đến Z.

Người dùng nhập First Name và Last Name rồi nhấn Add. Chương trình xác định chữ cái đầu của tên và thêm người đó vào nhóm chữ cái tương ứng.

Nội dung hiển thị theo hình minh họa trong đề. Ví dụ, First Name là Bình và Last Name là Ngô Thanh thì node mới có nội dung "Bình, Ngô Thanh" và được đặt bên dưới node B.

2. Tổ chức chương trình

Bài tập được đặt trong project VeNha03_DanhBa, thuộc thư mục BaiTapWinform\TH5c.

File Program.cs khởi động chương trình và mở Form frmDanhBa. File frmDanhBa.cs chứa phần thiết kế giao diện, khởi tạo các nhóm chữ cái, kiểm tra dữ liệu và thêm người vào danh bạ.

Danh bạ được lưu trong các node của TreeView trong thời gian chương trình hoạt động. Chương trình chưa thực hiện lưu dữ liệu ra file hoặc cơ sở dữ liệu.

3. Thiết kế giao diện

TreeView được đặt bên trái để hiển thị các nhóm chữ cái và những người trong từng nhóm. Bên phải là nhóm Thông tin danh bạ gồm TextBox nhập First Name, TextBox nhập Last Name và nút Add.

First Name được dùng để nhập tên gọi. Last Name được dùng để nhập họ và tên đệm theo ví dụ trong đề.

TreeView sử dụng HideSelection bằng false để người dùng vẫn nhìn thấy node đang chọn khi chuyển con trỏ sang ô nhập. Nút Exit dùng để kết thúc chương trình.

Thuộc tính AcceptButton được gán cho nút Add để người dùng có thể nhấn Enter sau khi nhập thông tin.

4. Khởi tạo các nhóm chữ cái

Sự kiện Load của Form tạo các node từ A đến Z bằng vòng lặp trên kiểu char. Mỗi chữ cái được thêm một lần, tạo thành 26 nhóm.

Mỗi node có khóa và nội dung hiển thị cùng là chữ cái tương ứng. Khóa được sử dụng để tìm nhanh nhóm cần thêm người, còn nội dung hiển thị giúp người dùng nhận biết nhóm trên giao diện.

Các nhóm chữ cái được tạo ngay khi mở Form, kể cả khi danh bạ chưa có người nào.

5. Kiểm tra và chuẩn hóa thông tin

Trước khi thêm, chương trình kiểm tra cả First Name và Last Name không được để trống. Nếu thiếu thông tin, chương trình hiển thị MessageBox và đặt con trỏ vào ô cần nhập.

Phương thức ChuanHoaKhoangTrang loại bỏ khoảng trắng ở đầu, cuối và những khoảng trắng thừa giữa các từ. Những từ còn lại được nối bằng một dấu cách.

Chuỗi cũng được chuẩn hóa Unicode về FormC để thông tin tiếng Việt có dấu được lưu nhất quán.

Tên phải bắt đầu bằng chữ cái có thể đưa vào một nhóm từ A đến Z. Nếu tên bắt đầu bằng chữ số hoặc ký tự đặc biệt, chương trình thông báo và không thêm vào danh bạ.

6. Xác định chữ cái đầu của tên

Chương trình gọi phương thức LayChuCaiDau để xác định nhóm của người được thêm.

Chuỗi tên được chuẩn hóa về FormD để tách chữ gốc và dấu tổ hợp. Các dấu được bỏ qua, còn chữ cái đầu được chuyển thành chữ hoa. Nhờ đó, tên bắt đầu bằng chữ có dấu vẫn được đưa vào đúng nhóm, chẳng hạn Ánh thuộc nhóm A và Ứng thuộc nhóm U.

Chữ Đ được xử lý riêng và chuyển về D vì ký tự này không được tách thành D bằng FormD.

Việc bỏ dấu chỉ phục vụ xác định nhóm. Nội dung họ tên hiển thị trong danh bạ vẫn giữ nguyên dấu tiếng Việt.

7. Thêm người vào TreeView

Sau khi xác định chữ cái đầu, chương trình tìm node tương ứng bằng khóa trong TreeView.

Nội dung node mới được ghép theo dạng “First Name, Last Name”. Node này được thêm vào tập hợp Nodes của nhóm chữ cái tìm được.

Sau khi thêm, nhóm được mở rộng để hiển thị người mới. Chương trình chọn node vừa tạo và gọi EnsureVisible để cuộn đến vị trí đó nếu cần.

Cuối cùng, hai ô nhập được làm trống và con trỏ trở lại First Name để người dùng tiếp tục nhập người khác.

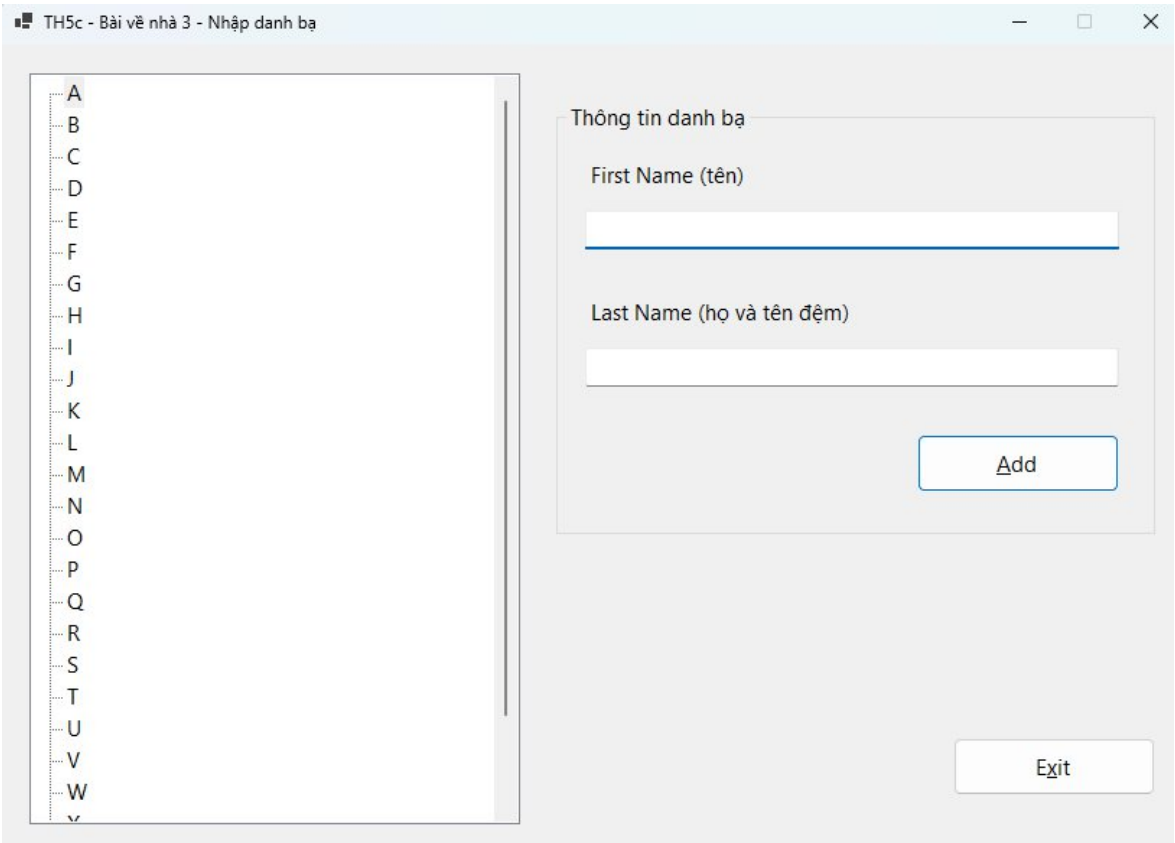
Chương trình cho phép tên trùng nhau vì đề bài không yêu cầu kiểm tra trùng và nhiều người có thể có cùng họ tên.

8. Xác nhận đóng chương trình

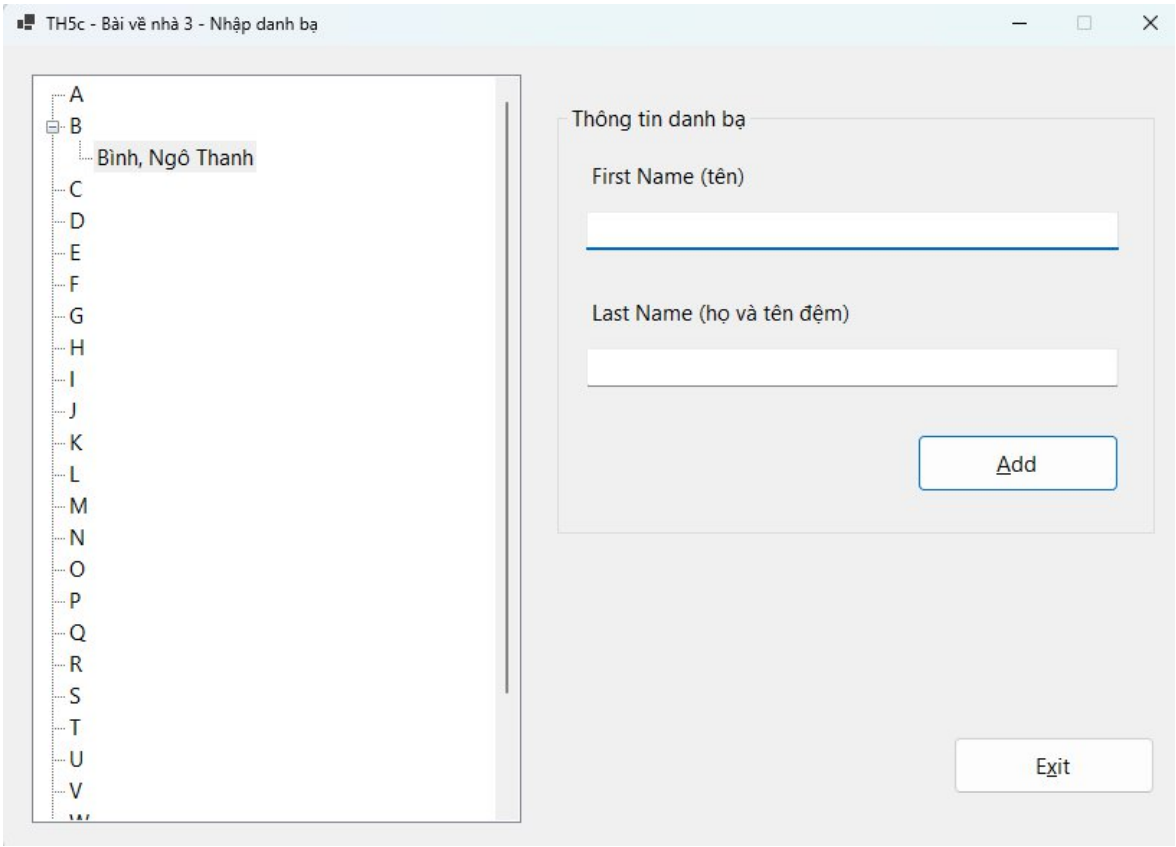
Sự kiện FormClosing hiển thị hộp thoại hỏi người dùng có muốn đóng chương trình hay không.

Nếu chọn Yes, chương trình kết thúc. Nếu chọn No, chương trình đặt e.Cancel bằng true để hủy thao tác đóng. Cách xử lý này áp dụng khi nhấn Exit hoặc nút X trên cửa sổ.

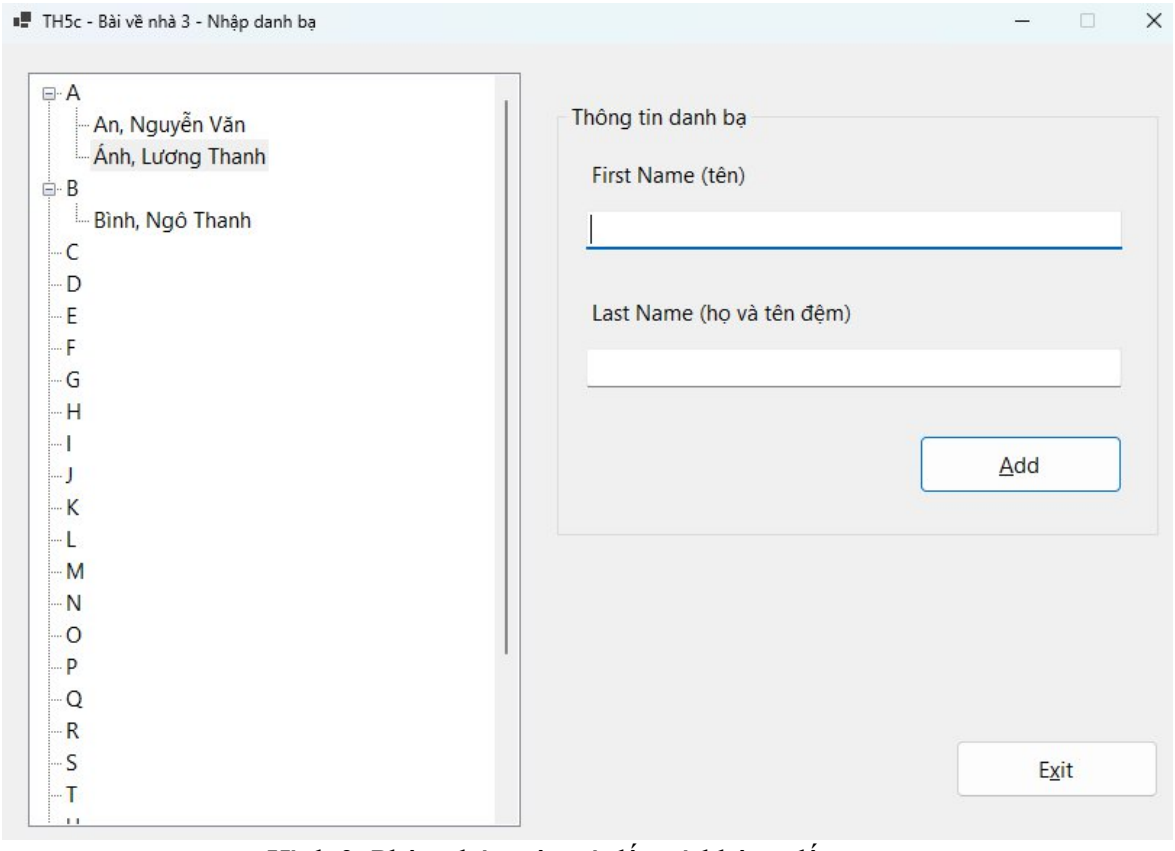
9. Hình ảnh kết quả thực hiện



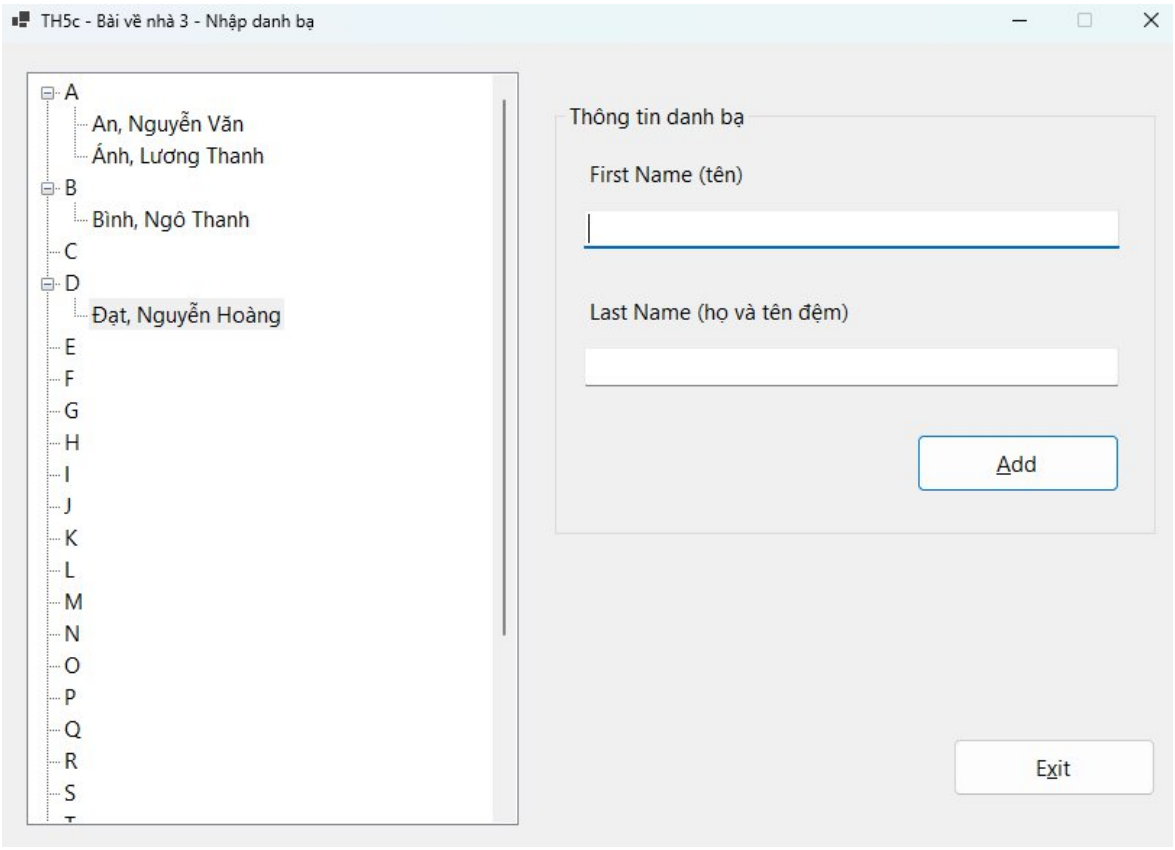
Hình 1. Giao diện danh bạ khi mở chương trình.



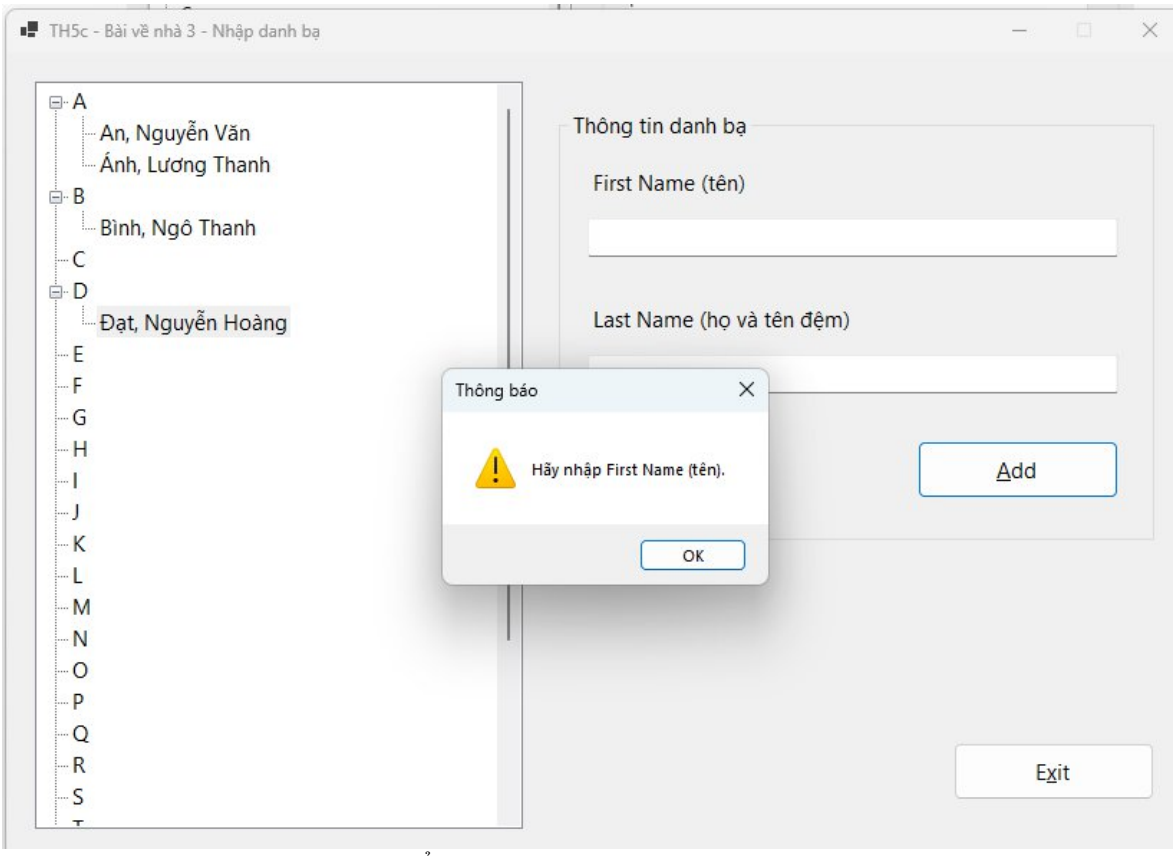
Hình 2. Thêm người vào nhóm theo chữ cái đầu của tên.



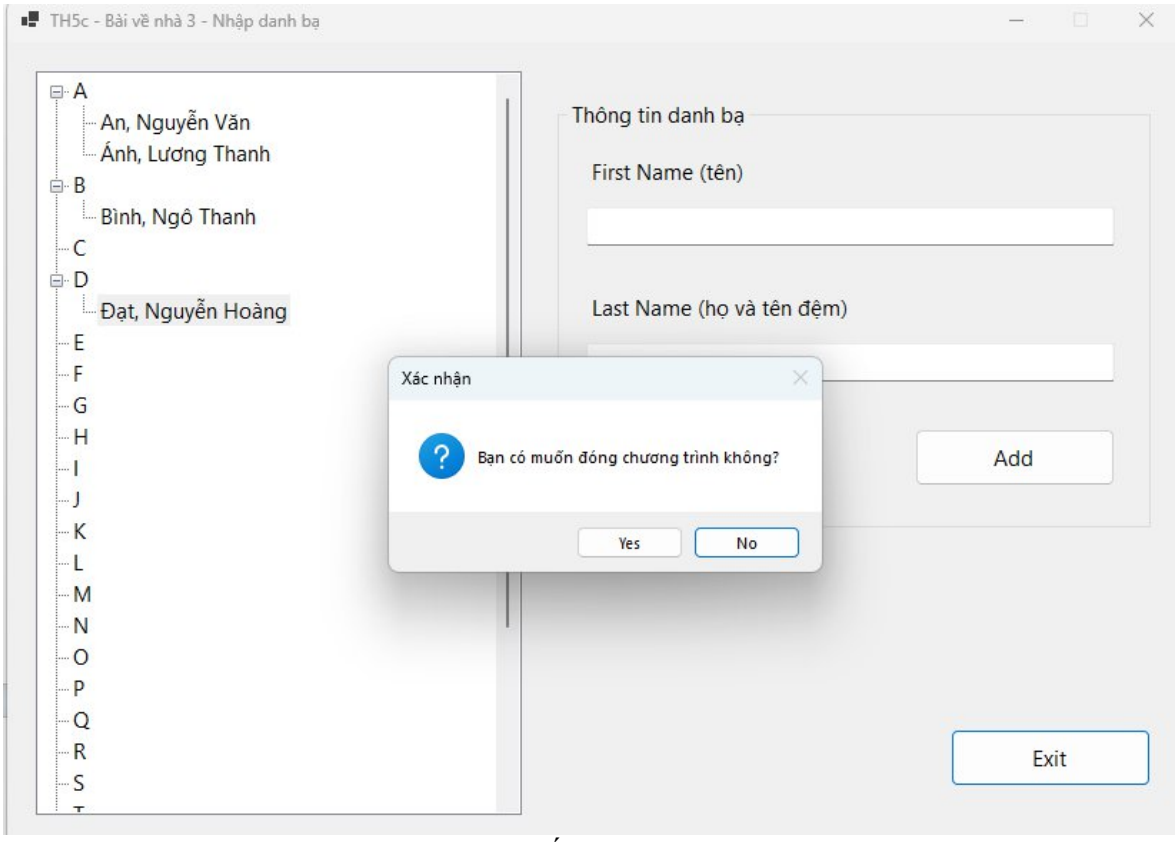
Hình 3. Phân nhóm tên có dấu và không dấu.



Hình 4. Xử lý chữ Đ khi xác định nhóm danh bạ.



Hình 5. Kiểm tra thông tin trước khi thêm.

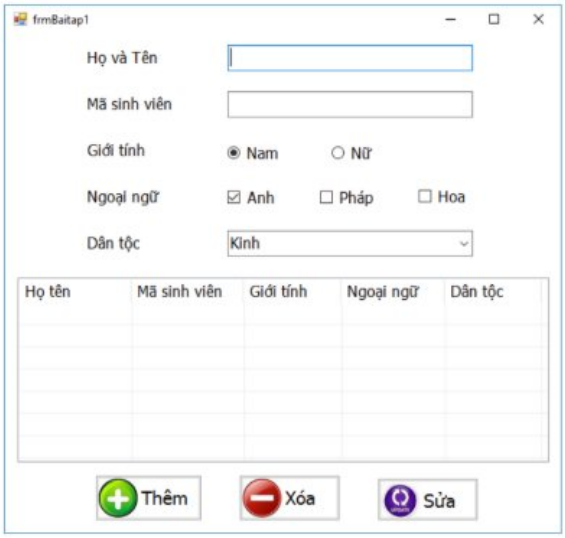


Hình 6. Xác nhận kết thúc chương trình.

THỰC HÀNH 5D: LẬP TRÌNH WINDOW FORM NÂNG CAO (2)

Bài Tập Cơ Bản - Bài 1:

Bài 1. Thiết kế giao diện sau:



Yêu cầu:

– Khi Load form lên thực hiện thêm dữ liệu vào Combobox dân tộc.

– Thực hiện Thêm, Xóa, Sửa từ các control lên Listview. Lưu ý "Thêm không cho trùng mã sinh viên", không cho sửa Mã sinh viên.

– Sử dụng ContextMenustrip cho thực hiện Xóa khi kích chọn phải chuột vào Listview.

– Hỏi xác nhận trước khi đóng Form.

1. Yêu cầu bài tập

Thiết kế chương trình quản lý thông tin sinh viên gồm họ tên, mã sinh viên, giới tính, ngoại ngữ và dân tộc. Khi Form được mở, chương trình nạp dữ liệu vào ComboBox dân tộc.

Chương trình thực hiện thêm, xóa và sửa thông tin trên ListView. Khi thêm, mã sinh viên không được trùng với dữ liệu đã tồn tại. Khi sửa, mã sinh viên phải được giữ nguyên.

ListView có ContextMenuStrip để thực hiện xóa bằng thao tác nhấn chuột phải. Chương trình hỏi xác nhận trước khi đóng Form.

2. Tổ chức chương trình

Bài tập được đặt trong project Bai01_ListViewSinhVien, thuộc thư mục BaiTapWinform\TH5d.

File Program.cs khởi động chương trình và mở Form frmListViewSinhVien. File frmListViewSinhVien.cs chứa giao diện, các sự kiện và phương thức thêm, xóa, sửa dữ liệu.

Thông tin sinh viên được lưu trực tiếp trong các ListViewItem. Mỗi dòng có năm cột tương ứng với họ tên, mã sinh viên, giới tính, ngoại ngữ và dân tộc.

Dữ liệu chỉ được lưu trong bộ nhớ khi chương trình đang hoạt động.

3. Thiết kế giao diện

TextBox được dùng để nhập họ tên và mã sinh viên. Hai RadioButton cho phép chọn giới tính Nam hoặc Nữ. Các CheckBox cho phép chọn một hoặc nhiều ngoại ngữ gồm Anh, Pháp và Hoa.

ComboBox dân tộc được đặt chế độ DropDownList để người dùng chọn từ những giá trị đã nạp.

ListView sử dụng View.Details để hiển thị theo cột. FullRowSelect cho phép chọn cả dòng, GridLines hiển thị đường kẻ và HideSelection giữ trạng thái hiển thị lựa chọn khi người dùng chuyển sang các ô nhập.

ListView chỉ cho chọn một dòng tại một thời điểm để xác định rõ sinh viên cần sửa hoặc xóa. Ngoài các nút theo đề, chương trình có nút Nhập mới để trở lại trạng thái thêm sinh viên.

4. Nạp dữ liệu và kiểm tra thông tin

Trong sự kiện Load, chương trình thêm các giá trị Kinh, Hoa, Khmer, H’Mông và Khác vào ComboBox dân tộc. Sau đó, giao diện được đưa về trạng thái nhập mới.

Trước khi thêm hoặc sửa, chương trình kiểm tra họ tên và mã sinh viên không được để trống, đồng thời yêu cầu chọn dân tộc.

Ngoại ngữ không bắt buộc. Nếu không chọn ngoại ngữ nào, cột Ngoại ngữ được để trống. Nếu chọn nhiều ngoại ngữ, các giá trị được ghép bằng dấu phẩy.

5. Thêm sinh viên

Khi nhấn Thêm, chương trình lấy mã sinh viên sau khi loại bỏ khoảng trắng ở đầu và cuối. Các dòng trong ListView được duyệt để kiểm tra mã đã tồn tại hay chưa.

Việc so sánh mã không phân biệt chữ hoa và chữ thường. Vì vậy, SV001 và sv001 được xem là cùng một mã. Nếu trùng, chương trình thông báo và không thêm dữ liệu.

Nếu hợp lệ, chương trình tạo ListViewItem chứa năm thông tin từ giao diện và thêm vào ListView. Sau khi thêm, các ô nhập được làm trống để người dùng tiếp tục nhập sinh viên khác.

6. Chọn và sửa sinh viên

Sự kiện SelectedIndexChanged đưa dữ liệu của dòng đang chọn lên các control. Giới tính được đưa vào RadioButton, ngoại ngữ được tách thành các giá trị để chọn CheckBox và dân tộc được chọn lại trong ComboBox.

Khi có dòng được chọn, nút Sửa và Xóa được bật, nút Thêm được tắt. TextBox mã sinh viên được đặt ReadOnly để ngăn người dùng thay đổi mã.

Phương thức SuaSinhVien chỉ cập nhật họ tên, giới tính, ngoại ngữ và dân tộc. Cột mã sinh viên không được gán lại, giúp giữ nguyên mã cả ở giao diện nhập và trong dữ liệu.

Nút Nhập mới bỏ lựa chọn hiện tại, xóa thông tin nhập và mở khóa ô mã sinh viên để thêm người khác.

7. Xóa bằng Button và ContextMenuStrip

Nút Xóa và mục Xóa sinh viên trong ContextMenuStrip cùng gọi phương thức XoaSinhVien.

Khi nhấn chuột phải vào một dòng, chương trình chọn đúng dòng tại vị trí con trỏ. Nếu nhấn chuột phải vào vùng trống, lựa chọn được bỏ và mục xóa bị vô hiệu hóa.

Trước khi xóa, chương trình hiển thị mã cùng họ tên để người dùng xác nhận. Chọn Yes thì dòng được loại khỏi ListView. Chọn No thì dữ liệu được giữ nguyên.

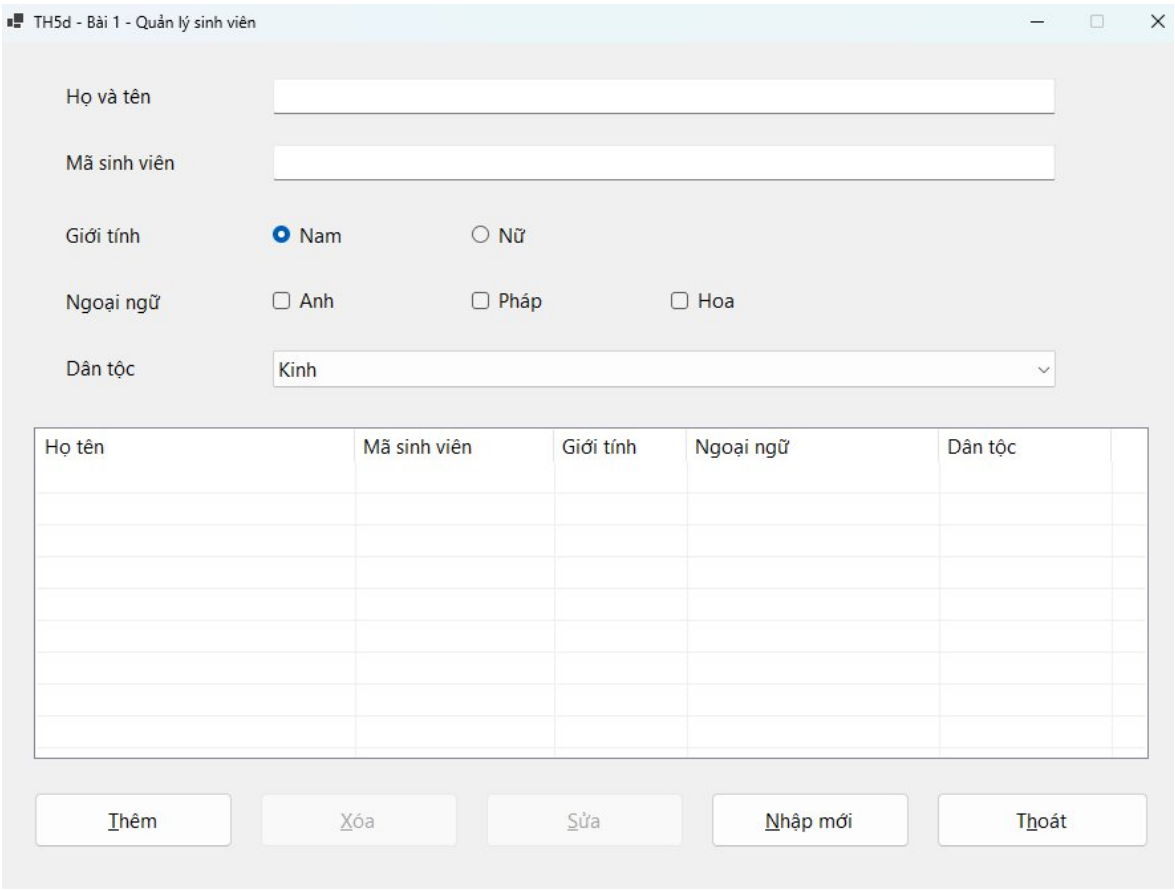
Sau khi xóa, chương trình trở về trạng thái nhập mới.

8. Xác nhận đóng chương trình

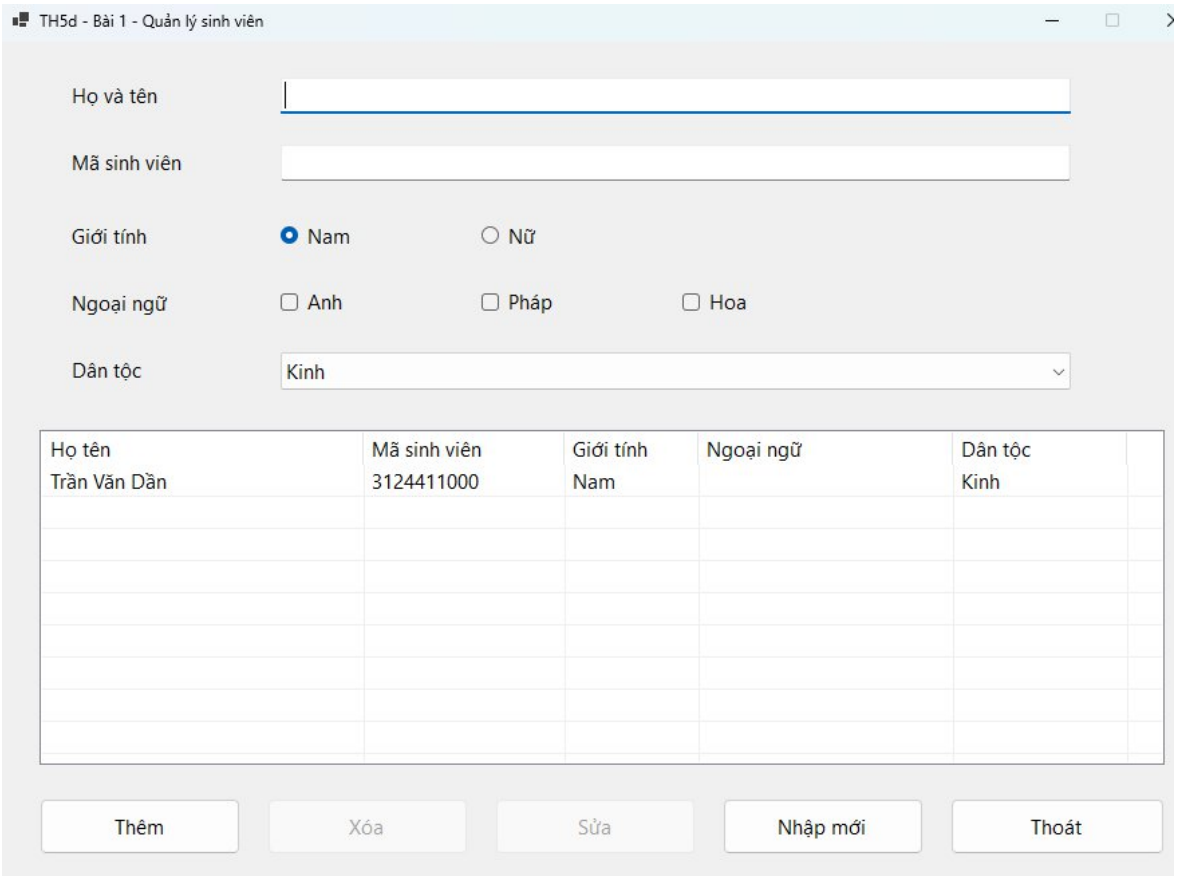
Chương trình xử lý sự kiện FormClosing để hỏi xác nhận khi người dùng nhấn Thoát hoặc nút X trên cửa sổ.

Nếu chọn Yes, Form được đóng. Nếu chọn No, thuộc tính e.Cancel được đặt thành true để hủy thao tác đóng.

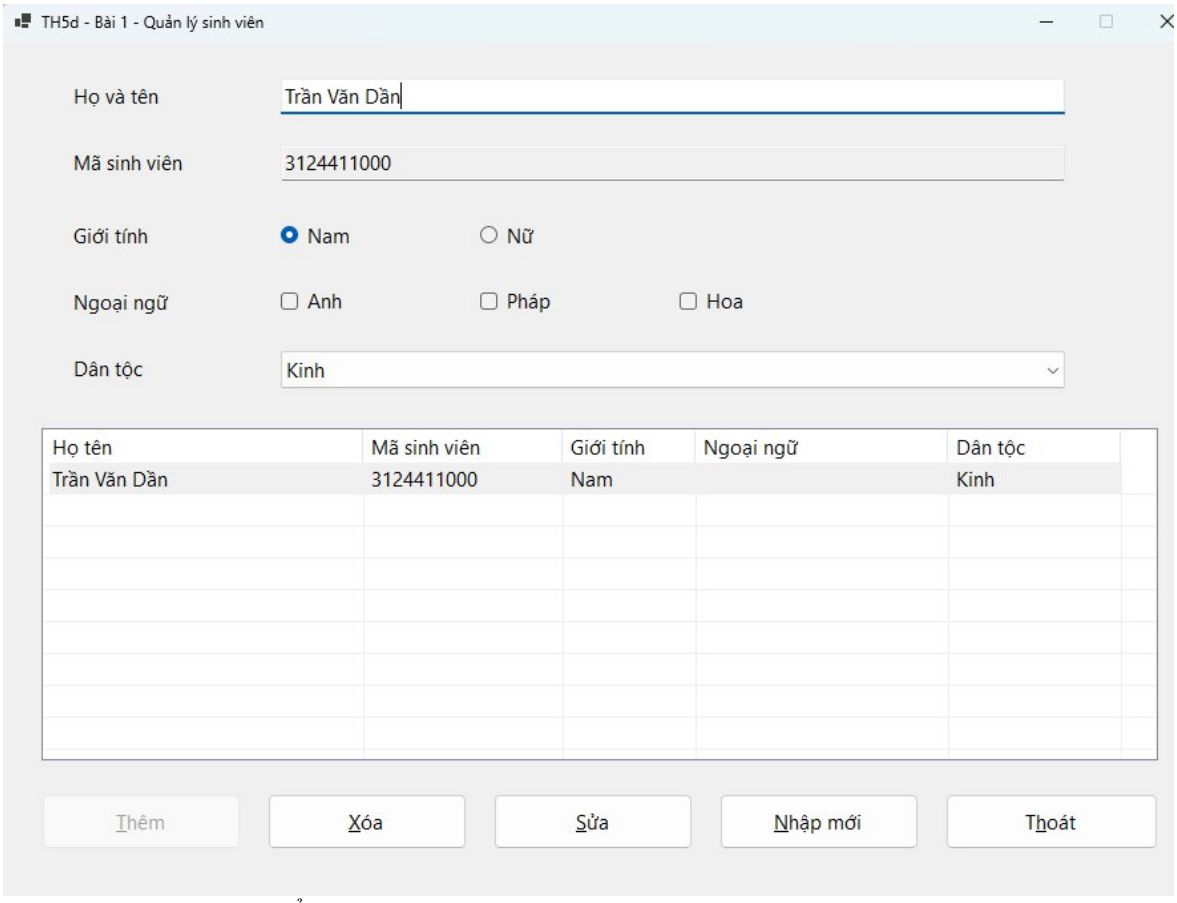
9. Hình ảnh kết quả thực hiện



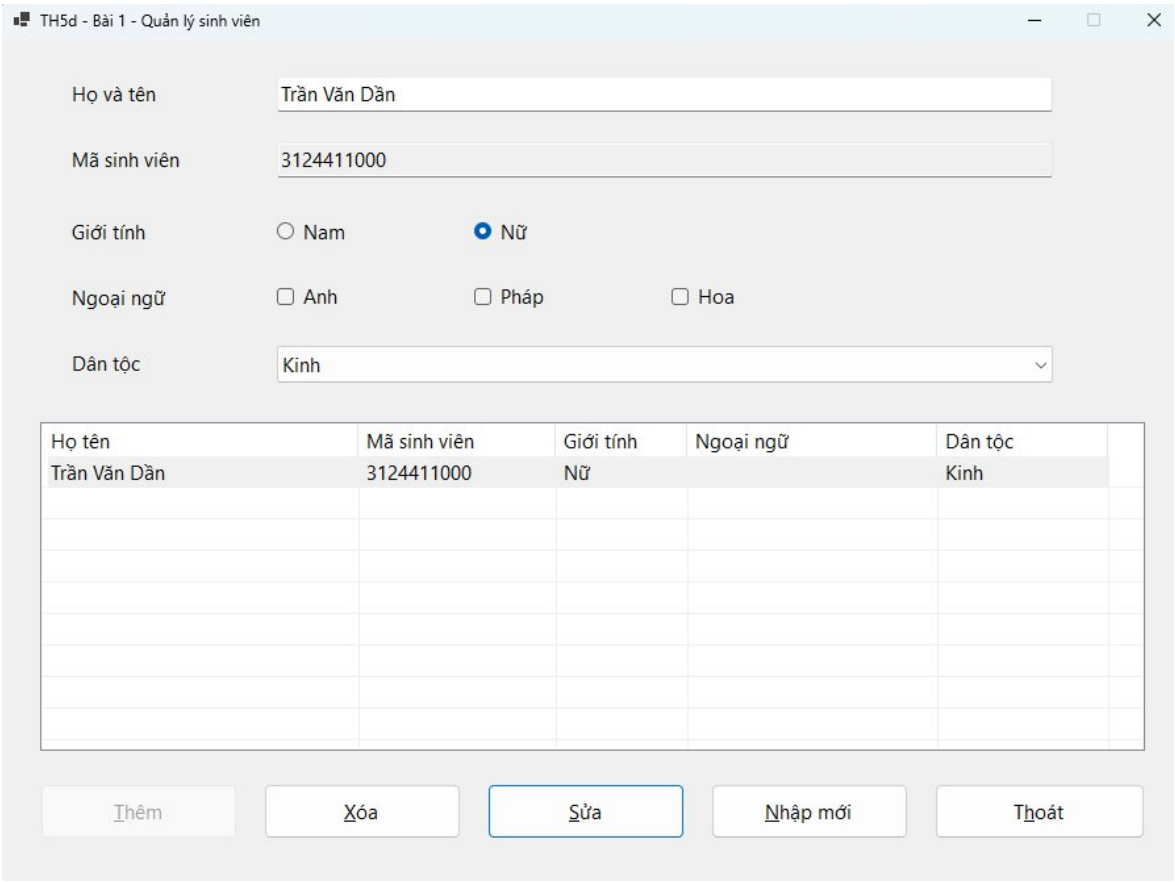
Hình 1. Giao diện nhập thông tin sinh viên.



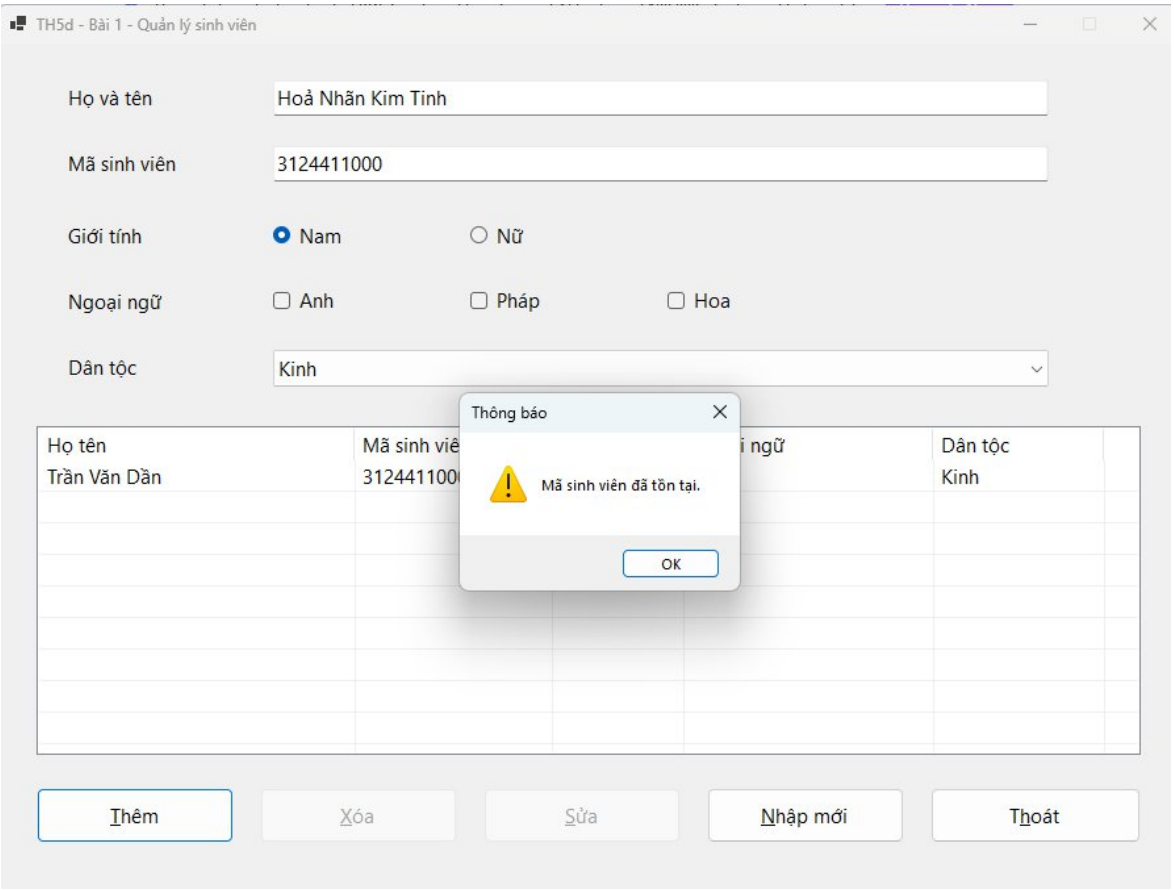
Hình 2. Thông tin sinh viên được thêm vào danh sách.



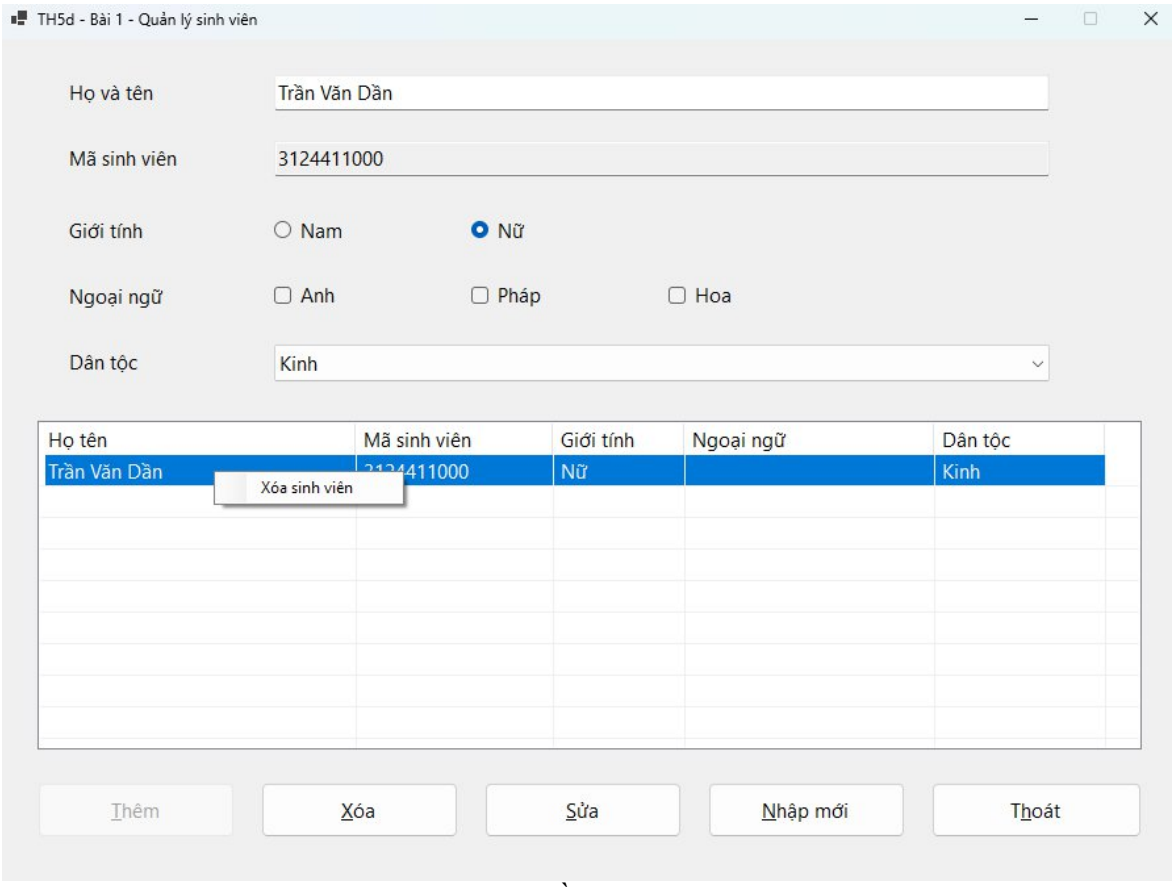
Hình 3. Hiển thị thông tin và giữ nguyên mã sinh viên khi sửa.



Hình 4. Cập nhật thông tin sinh viên trên ListView.



Hình 5. Kiểm tra mã sinh viên trước khi thêm.



Hình 6. Xóa sinh viên bằng ContextMenuStrip.

Bài Tập Cơ Bản - Bài 2:

Hỏi xác nhận trước khi đóng Form.

Bài 2. Thiết kế chương trình quản lý thông tin tài khoản cho một ngân hàng với giao diện sau:



Yêu cầu:

– Form load lên có hiện trạng nhoị hình trên.

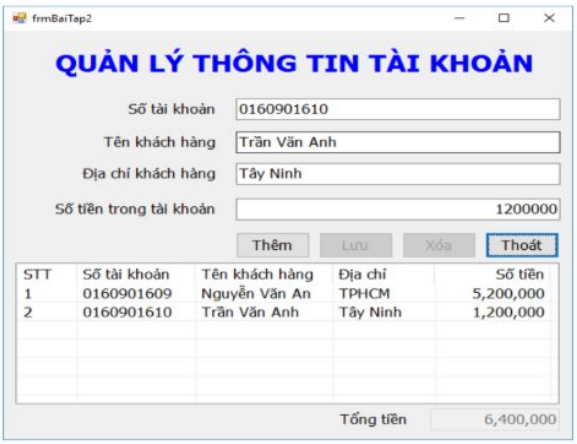
– Khi nhấn vào button "Thêm", text của button chuyển thành "Hủy" và Enabled button "Loịu", các control textbox là rỗng, đặt con trỏ vào textbox Số tài khoản.

– Button "Loịu" có tác dụng loịu dữ liệu từ các textbox vào trong listview.

– Button "Xóa" có tác dụng xóa mẫu tin hiện hành trong listview (Enabled = true khi một mẫu tin được chọn trong listview).

– Khi click một khách hàng trong listview, hiển thị thông tin lên các textbox tướng ứng.

– Thực hiện tính Tổng tiền, STT được đánh tự động khi thực hiện Thêm, Xóa.



1. Yêu cầu bài tập

Thiết kế chương trình quản lý tài khoản gồm số tài khoản, tên khách hàng, địa chỉ và số tiền trong tài khoản.

Khi mở Form, nút Lưu và Xóa bị vô hiệu hóa. Khi nhấn Thêm, nút chuyển thành Hủy, nút Lưu được bật, các ô nhập được làm trống và con trỏ được đặt vào ô Số tài khoản.

Nút Lưu thêm thông tin từ các TextBox vào ListView. Khi chọn một dòng, chương trình hiển thị thông tin lên các ô tương ứng và cho phép xóa dòng đang chọn.

Chương trình tự động đánh số thứ tự và tính tổng tiền sau khi thêm hoặc xóa.

2. Tổ chức chương trình

Bài tập được đặt trong project Bai02_TaiKhoanNganHang, thuộc thư mục BaiTapWinform\TH5d.

File Program.cs khởi động chương trình và mở Form frmTaiKhoanNganHang. File frmTaiKhoanNganHang.cs chứa lớp TaiKhoan, giao diện và các phương thức xử lý.

Lớp TaiKhoan lưu số tài khoản, tên khách hàng, địa chỉ và số tiền. Mỗi đối tượng được gắn vào thuộc tính Tag của ListViewItem để giữ dữ liệu gốc, độc lập với nội dung đã định dạng trên giao diện.

Dữ liệu được lưu trong bộ nhớ khi chương trình đang chạy, chưa được lưu vào cơ sở dữ liệu hoặc tập tin.

3. Thiết kế giao diện

Giao diện có bốn TextBox nhập thông tin, các nút Thêm, Lưu, Xóa và Thoát. ListView hiển thị năm cột gồm STT, Số tài khoản, Tên khách hàng, Địa chỉ và Số tiền.

ListView sử dụng View.Details, GridLines và FullRowSelect để hiển thị dữ liệu theo bảng và chọn cả dòng. MultiSelect bằng false để mỗi lần chỉ xử lý một tài khoản.

Ô Tổng tiền được đặt ReadOnly để người dùng không chỉnh sửa kết quả tính toán. Các cột và ô tiền được căn phải để dễ đọc.

4. Điều khiển trạng thái Thêm và Hủy

Biến dangThemMoi xác định chương trình có đang nhập tài khoản mới hay không.

Khi nhấn Thêm, chương trình chuyển sang trạng thái thêm mới, đổi nội dung nút thành Hủy và bật nút Lưu. Các ô nhập được làm trống và con trỏ chuyển đến ô Số tài khoản.

ListView tạm thời bị vô hiệu hóa trong lúc nhập mới để tránh việc chọn một dòng khác làm thay đổi thông tin đang nhập.

Khi nhấn Hủy, chương trình bỏ dữ liệu chưa lưu và trở về trạng thái ban đầu. Các tài khoản đã có trong danh sách không bị thay đổi.

5. Kiểm tra và lưu tài khoản

Số tài khoản được lưu bằng kiểu string để giữ nguyên những chữ số 0 ở đầu. Chương trình kiểm tra số tài khoản chỉ gồm các chữ số và chưa tồn tại trong danh sách.

Tên khách hàng và địa chỉ không được để trống. Số tiền phải là số nguyên không âm, nhập bằng chữ số và không có dấu phân cách.

Số tiền được lưu bằng kiểu decimal. Trước khi thêm, chương trình kiểm tra tổng mới có vượt giới hạn của kiểu dữ liệu hay không. Nếu dữ liệu không hợp lệ, chương trình thông báo và giữ trạng thái nhập để người dùng sửa.

Khi hợp lệ, chương trình tạo đối tượng TaiKhoan và một dòng ListViewItem. Dữ liệu được lưu trong Tag, còn số tiền trên giao diện được định dạng để dễ đọc.

Sau khi lưu, chương trình cập nhật STT, tổng tiền và trở về trạng thái chờ thêm tài khoản khác.

6. Hiển thị thông tin tài khoản được chọn

Sự kiện SelectedIndexChanged kiểm tra có một dòng đang được chọn hay không. Nếu có, nút Xóa được bật và dữ liệu được lấy từ đối tượng TaiKhoan trong Tag để đưa lên các TextBox.

Số tiền trong ô nhập được hiển thị không có dấu phân cách. Số tiền trên ListView và ô Tổng tiền được định dạng theo quy tắc tiếng Việt.

Nếu không còn dòng được chọn, nút Xóa bị vô hiệu hóa và các ô thông tin được làm trống.

Theo phạm vi bài tập, nút Lưu dùng để thêm mới. Việc thay đổi nội dung ô nhập khi xem một tài khoản không tự cập nhật dòng đã lưu.

7. Xóa tài khoản

Chức năng Xóa chỉ thực hiện khi có một dòng được chọn và chương trình không ở trạng thái thêm mới.

Trước khi xóa, chương trình hiển thị số tài khoản và tên khách hàng để người dùng xác nhận. Chọn No thì giữ nguyên dữ liệu; chọn Yes thì dòng được loại khỏi ListView.

Sau khi xóa, chương trình đánh lại STT, tính lại tổng tiền, làm trống các ô thông tin và vô hiệu hóa nút Xóa.

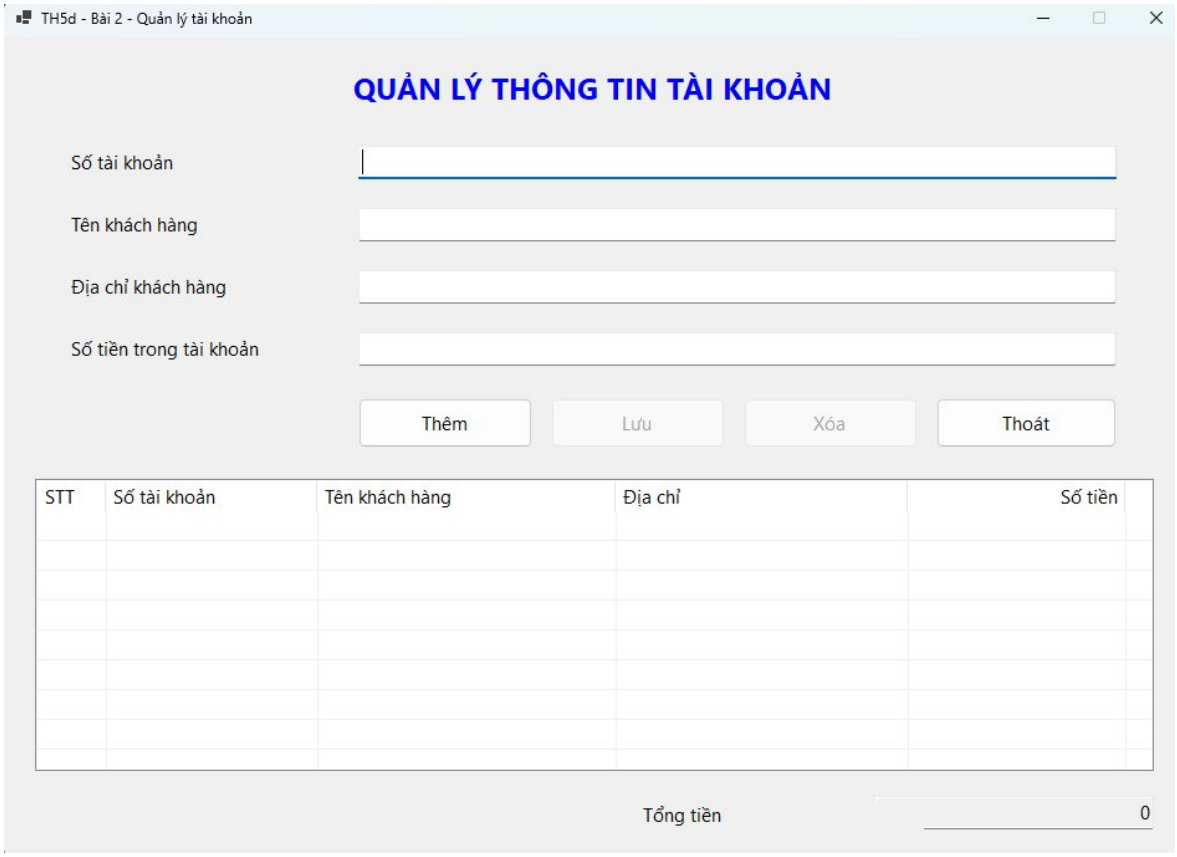
8. Đánh số thứ tự tổng tiền và đóng chương trình

Phương thức CapNhatSTTVaTong duyệt ListView theo thứ tự hiện tại và gán STT từ 1. Vì vậy, khi xóa một dòng, các dòng còn lại được đánh số liên tục.

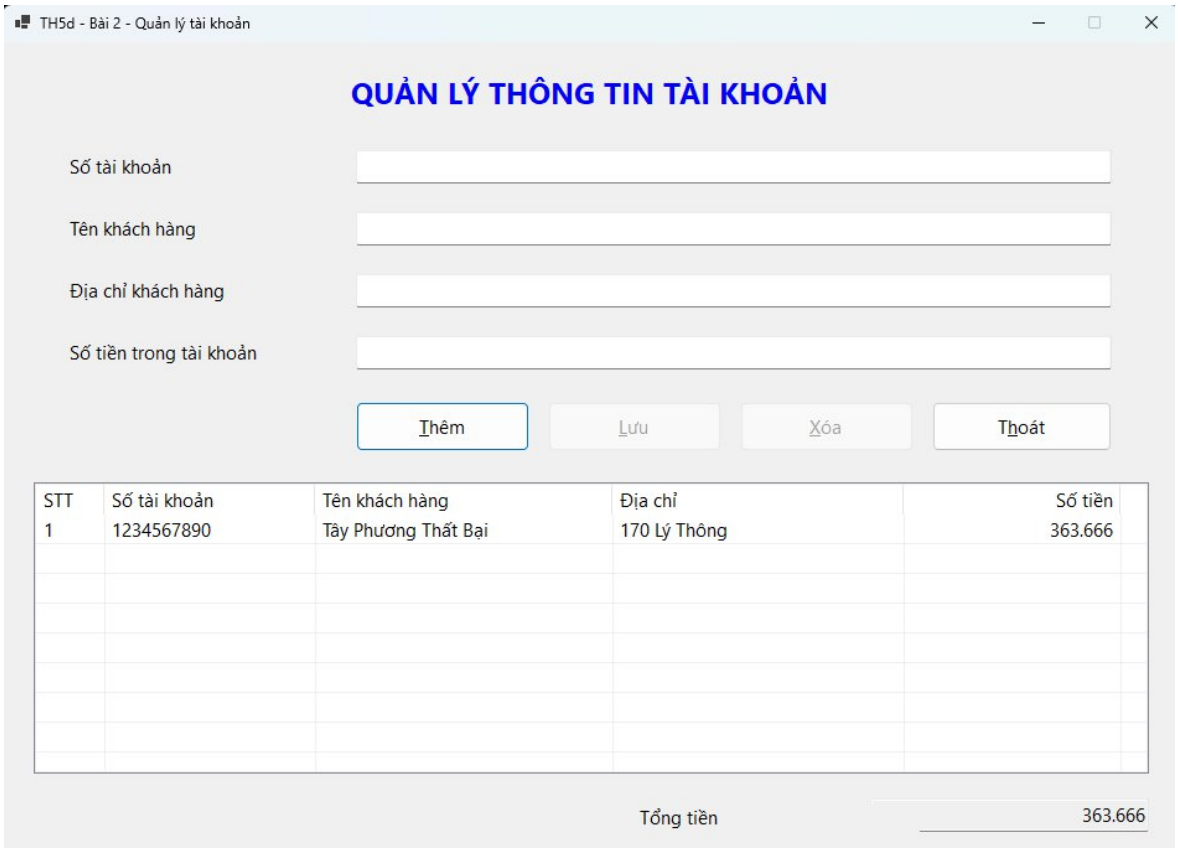
Tổng tiền được tính bằng cách cộng thuộc tính SoTien của tất cả đối tượng TaiKhoan trong ListView. Chương trình sử dụng dữ liệu gốc, không phân tích lại chuỗi tiền đã định dạng.

Khi danh sách rỗng, tổng tiền bằng 0. Khi đóng Form, chương trình hỏi xác nhận và chỉ kết thúc nếu người dùng chọn Yes.

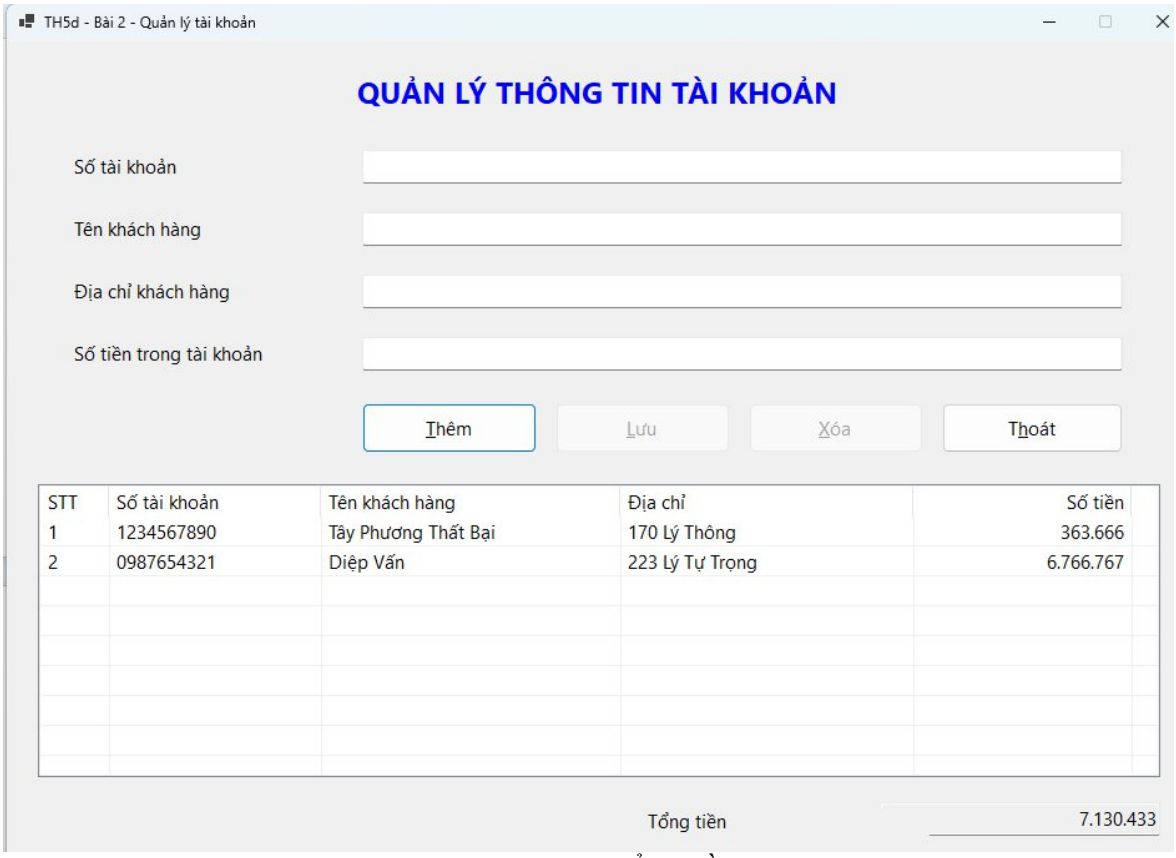
9. Hình ảnh kết quả thực hiện



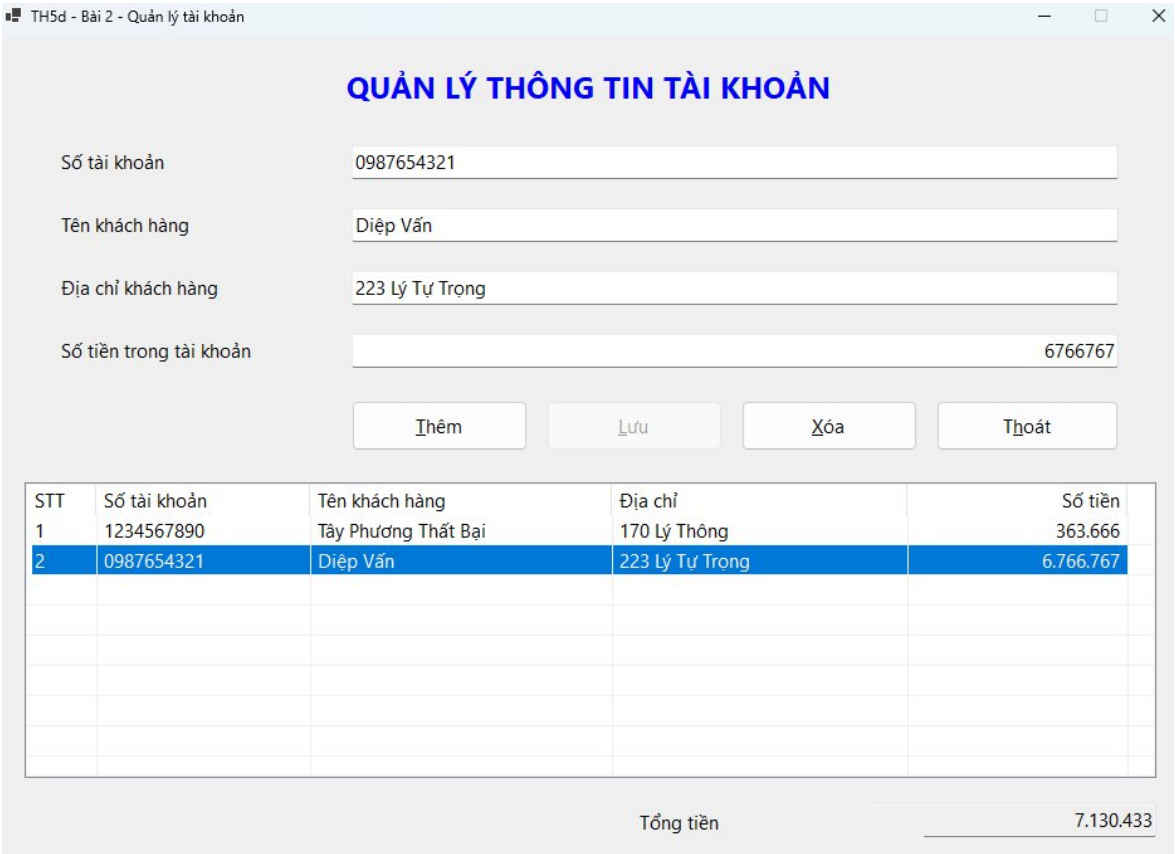
Hình 1. Trạng thái ban đầu của chương trình.



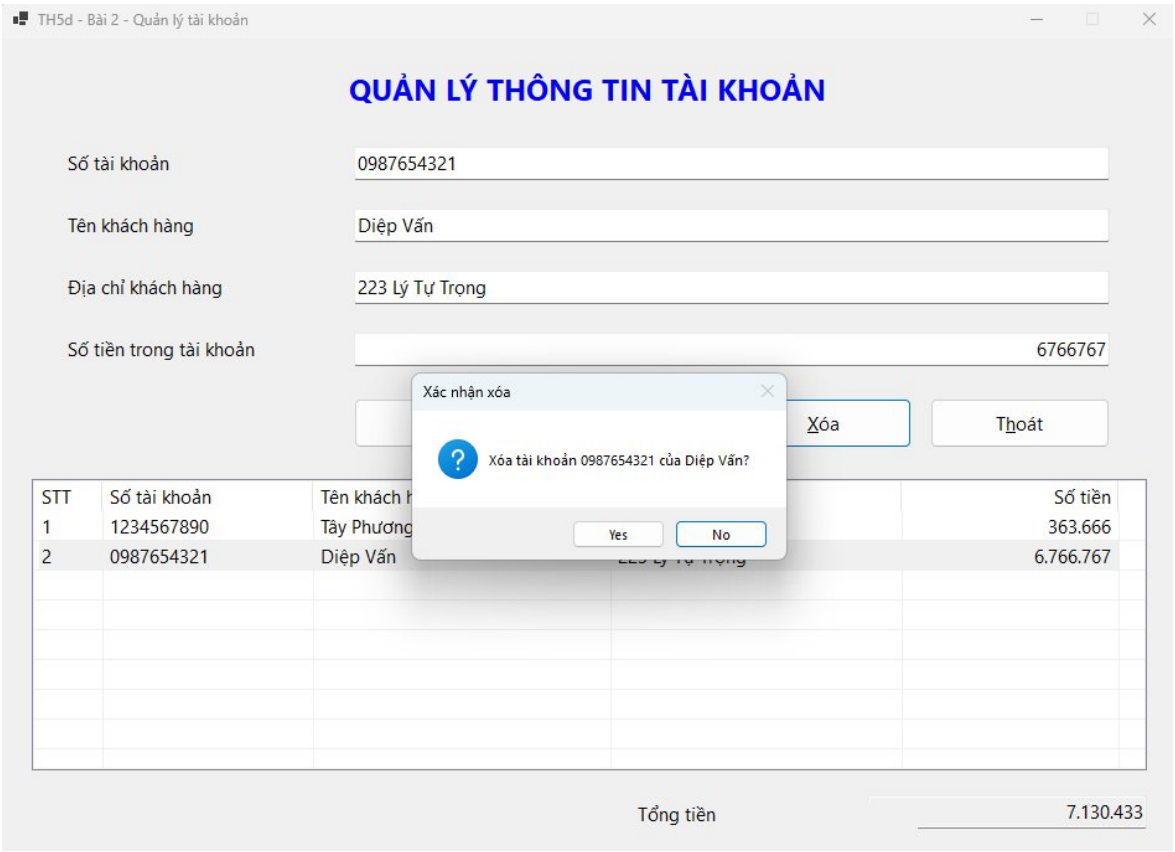
Hình 2. Trạng thái nhập tài khoản mới.



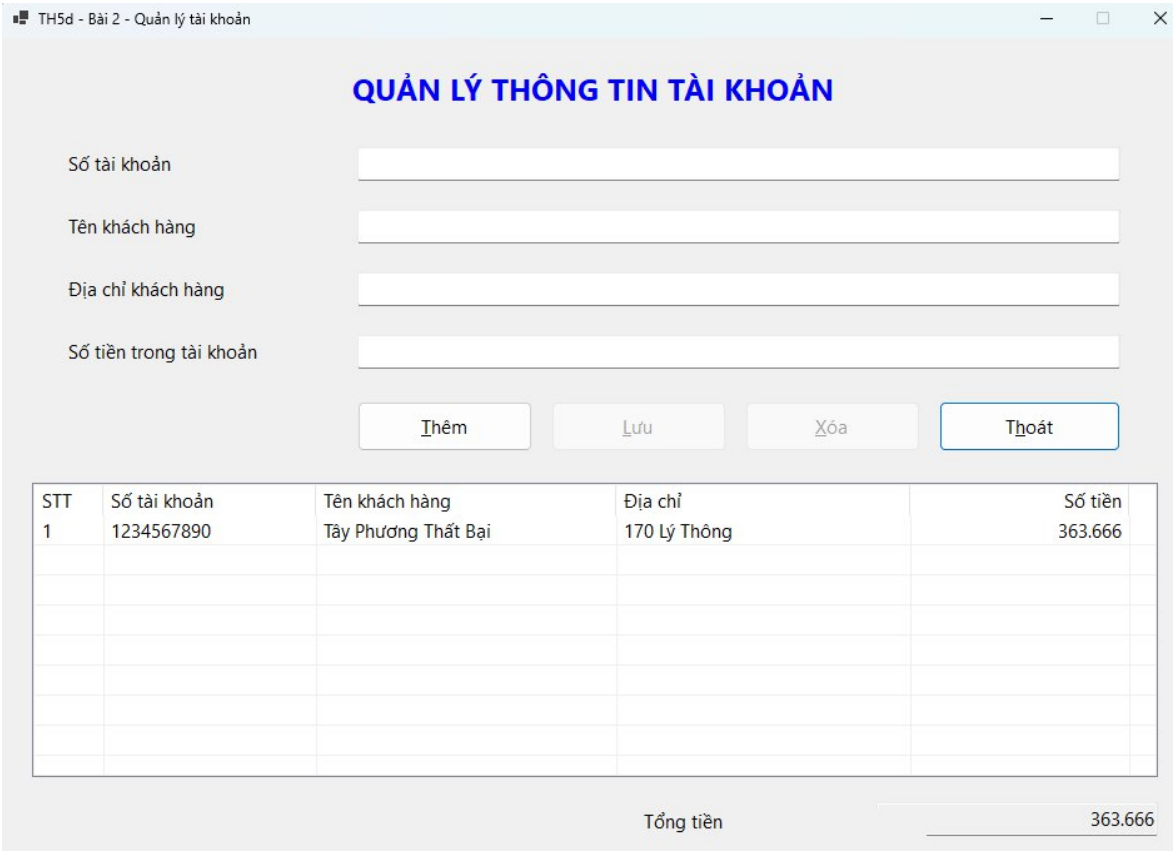
Hình 3. Thông tin tài khoản và tổng tiền sau khi lưu.



Hình 4. Hiển thị thông tin tài khoản được chọn.



Hình 5. Xác nhận trước khi xóa tài khoản.

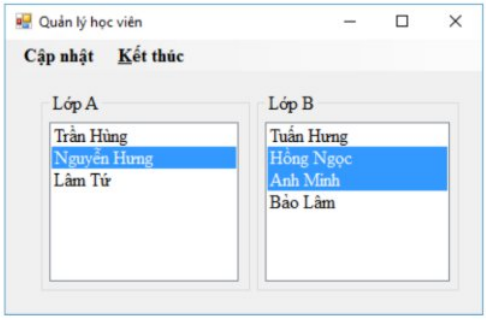


Hình 6. Kết quả cập nhật danh sách sau khi xóa.

Bài Tập Nâng Cao - Bài 1:

3. Bài tập nâng cao

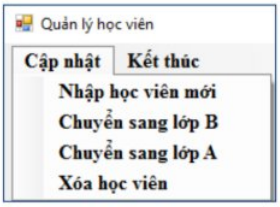
Bài 1. Thiết kế form có giao diện sau:



Yêu cầu:

– Khi người sử dụng click vào mục kết thúc thì xuất hiện một Messagebox hỏi người sử dụng sau đó mới quyết định có dừng chương trình hay không.

– Trong menu Cập Nhật có các mục:



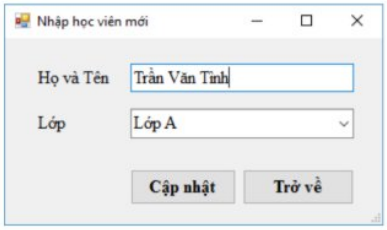
+ Xóa học viên: Xóa tất cả các học viên đang chọn trong Lớp A và Lớp B. (các listbox Lớp A và Lớp B cho phép chọn cùng lúc nhiều mục).

+ Chuyển sang lớp A: Chuyển các mục đang chọn trong Lớp B sang Lớp A

+ Chuyển sang lớp B: Chuyển các mục đang chọn trong Lớp A sang Lớp B

Lưu ý: Khi hai mục trên được chọn nếu không có học viên nào được chọn trong listbox thì xuất hiện một MessageBox yêu cầu người sử dụng phải chọn học viên trước.

+ Nhập học viên mới: Khi mục này được chọn thì xuất hiện form cho phép nhập thông tin nhoị sau (combobox Lớp: chứa Lớp A và Lớp B).



- Khi ngoịời sử dụng click vào button Cập Nhật:
  - Nếu Họ và Tên khác rỗng thì:
    - Phụ thuộc vào lớp đã chọn trong combobox cập nhật họ tên vào listbox tướng ứng trên form chính.
  - Nếu Họ và Tên rỗng:
    - Xuất hiện một MessageBox yêu cầu phải nhập họ tên
    - Đặt con trỏ vào textbox Họ và Tên.
- Khi ngoịời sử dụng click vào button Trở về thì đóng form nhập thông tin và trở về form chính.

1. Yêu cầu bài tập

Thiết kế chương trình quản lý học viên gồm hai ListBox tương ứng với lớp A và lớp B. Hai danh sách phải cho phép chọn nhiều học viên.

Menu Cập nhật có các chức năng xóa học viên đang chọn, chuyển học viên từ lớp B sang lớp A, chuyển học viên từ lớp A sang lớp B và nhập học viên mới.

Form nhập cho phép nhập họ tên và chọn lớp. Họ tên không được để trống. Nút Cập nhật thêm học viên vào lớp tương ứng trên Form chính, còn nút Trở về đóng Form nhập.

Khi chọn Kết thúc, chương trình phải hỏi xác nhận trước khi đóng.

2. Tổ chức chương trình

Bài tập được đặt trong project NangCao01_QuanLyHocVien, thuộc thư mục BaiTapWinform\TH5d.

File Program.cs khởi động chương trình và mở Form frmQuanLyHocVien. File frmQuanLyHocVien.cs chứa MenuStrip, hai ListBox và các phương thức xóa, chuyển lớp, mở Form nhập.

File frmNhapHocVien.cs chứa giao diện nhập học viên và sự kiện gửi thông tin về Form chính.

Họ tên học viên được lưu trực tiếp trong Items của hai ListBox. Dữ liệu được giữ trong bộ nhớ trong thời gian chương trình hoạt động.

3. Thiết kế giao diện chính

Form chính có MenuStrip gồm mục Cập nhật và Kết thúc. Trong Cập nhật có các mục Xóa học viên, Chuyển sang lớp A, Chuyển sang lớp B và Nhập học viên mới.

Hai ListBox được đặt cạnh nhau để hiển thị học viên của từng lớp. SelectionMode được đặt là MultiExtended, cho phép dùng Ctrl để chọn nhiều mục riêng lẻ hoặc Shift để chọn một dãy.

Thanh cuộn ngang được bật để hỗ trợ hiển thị họ tên dài. Hai danh sách ban đầu rỗng và được bổ sung dữ liệu thông qua Form nhập.

4. Nhập học viên mới

Khi chọn Nhập học viên mới, chương trình tạo Form frmNhapHocVien và mở bằng ShowDialog. Form nhập chứa TextBox họ tên, ComboBox lớp và hai nút Cập nhật, Trở về.

ComboBox có sẵn Lớp A và Lớp B, sử dụng DropDownList để người dùng chỉ chọn một trong hai lớp. Lớp A được chọn mặc định.

Khi nhấn Cập nhật, chương trình loại bỏ khoảng trắng thừa và kiểm tra họ tên. Nếu trống, chương trình thông báo và đưa con trỏ về TextBox. Nếu hợp lệ, thông tin được gửi về Form chính để thêm vào đúng ListBox.

Sau khi thêm, ô họ tên được làm trống và nhận lại con trỏ. Form nhập vẫn mở để người dùng tiếp tục nhập học viên khác. Nhấn Trở về hoặc phím Esc để đóng Form nhập.

5. Trao đổi dữ liệu giữa hai Form

Form nhập khai báo sự kiện HocVienDuocThem với hai thông tin là họ tên và lớp.

Trước khi mở Form nhập, Form chính đăng ký xử lý sự kiện này. Khi Form nhập phát sự kiện, Form chính kiểm tra tên lớp và thêm họ tên vào ListBox tương ứng.

Cách tổ chức này giúp Form nhập gửi dữ liệu mà không cần trực tiếp truy cập các control riêng của Form chính. Giao diện nhập và phần quản lý danh sách được tách thành hai Form rõ ràng.

6. Chuyển học viên giữa các lớp

Hai mục chuyển lớp sử dụng chung phương thức ChuyenHocVien. Khi chuyển sang lớp A, nguồn là ListBox lớp B và đích là ListBox lớp A. Khi chuyển sang lớp B, hai vai trò được đảo lại.

Chương trình kiểm tra lớp nguồn có học viên đang chọn hay không. Nếu chưa chọn, MessageBox yêu cầu chọn học viên trước khi chuyển.

Các vị trí và họ tên được lấy trước khi thay đổi danh sách. Học viên được thêm vào cuối lớp đích theo thứ tự trong lớp nguồn, sau đó được xóa khỏi lớp nguồn.

Việc xóa thực hiện từ vị trí lớn đến nhỏ để tránh sai lệch chỉ số. BeginUpdate và EndUpdate hạn chế việc vẽ lại giao diện khi chuyển nhiều học viên.

7. Xóa học viên đang chọn

Chức năng Xóa học viên lấy các vị trí đang chọn ở cả lớp A và lớp B. Nếu cả hai danh sách đều không có lựa chọn, chương trình thông báo và không xóa dữ liệu.

Nếu có học viên được chọn, chương trình xóa các mục tương ứng trong cả hai ListBox. Việc chọn ở lớp này không làm mất lựa chọn ở lớp còn lại, nên người dùng có thể xóa học viên của hai lớp trong cùng một thao tác.

Các vị trí được xóa theo thứ tự giảm dần để các phần tử chưa xử lý không bị dịch chuyển.

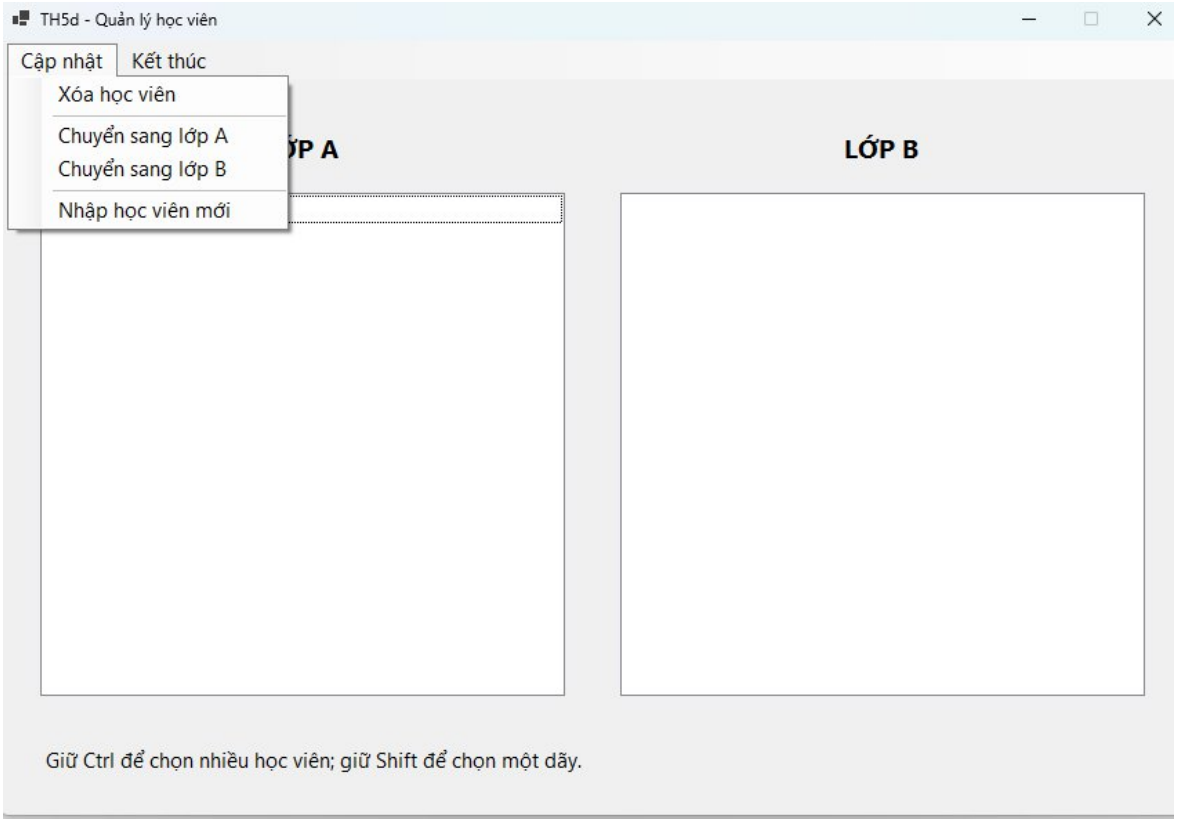
8. Kết thúc chương trình

Mục Kết thúc gọi phương thức Close của Form chính. Sự kiện FormClosing hiển thị hộp thoại hỏi người dùng có muốn kết thúc chương trình hay không.

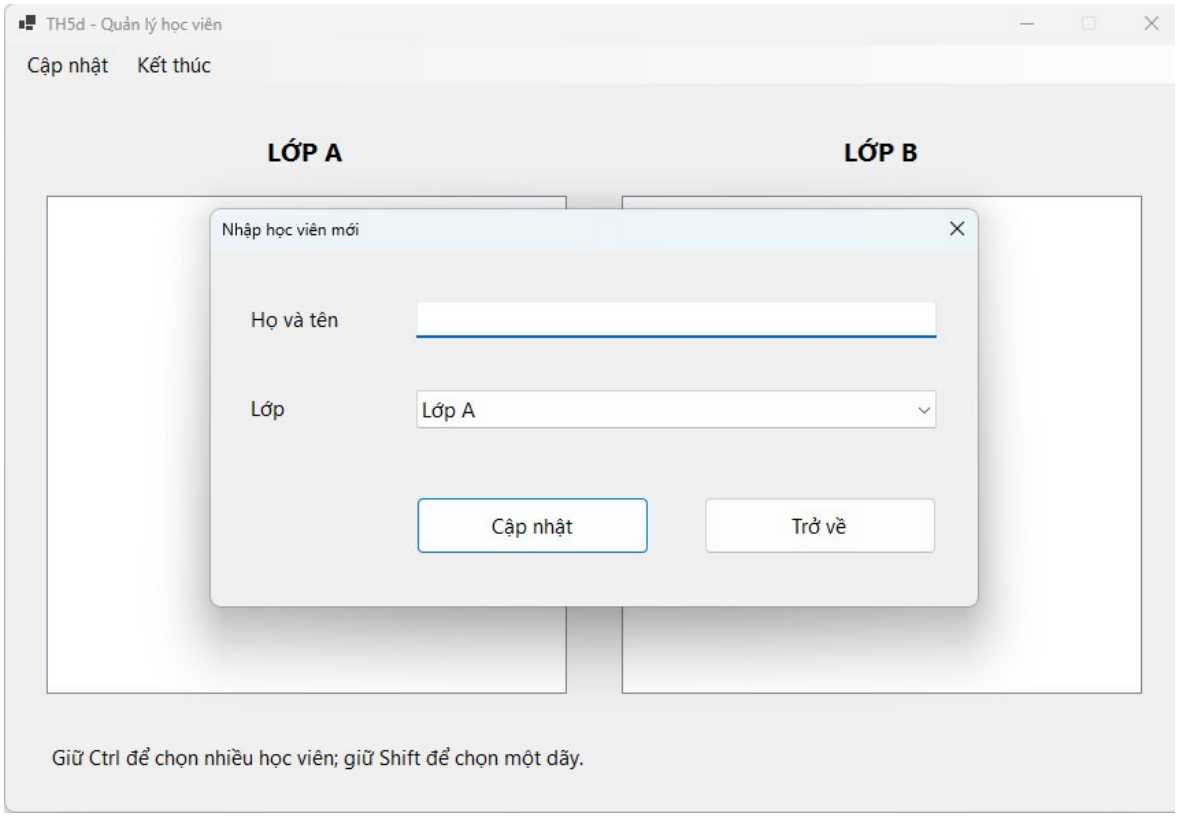
Chọn Yes thì chương trình đóng. Chọn No thì thao tác đóng được hủy bằng e.Cancel.

Form nhập sử dụng nút Trở về để đóng riêng cửa sổ nhập và quay lại Form chính, không kết thúc toàn bộ chương trình.

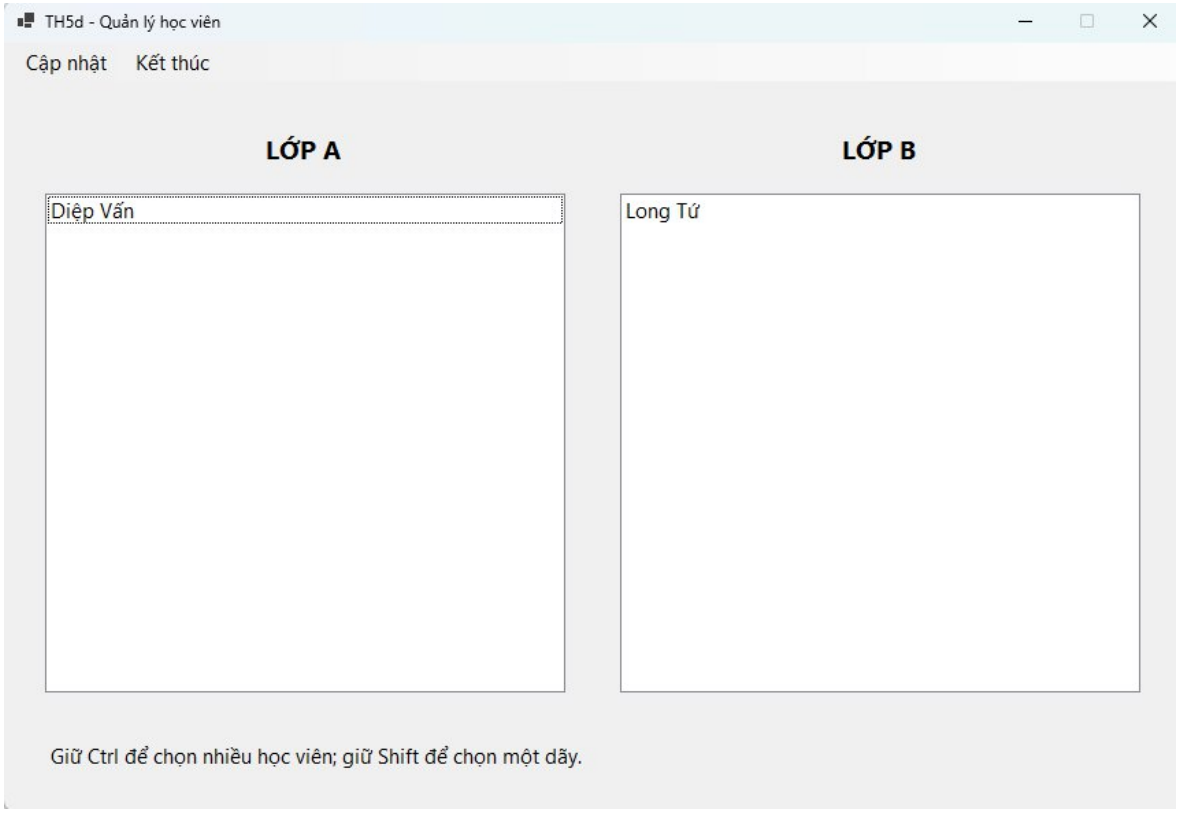
9. Hình ảnh kết quả thực hiện



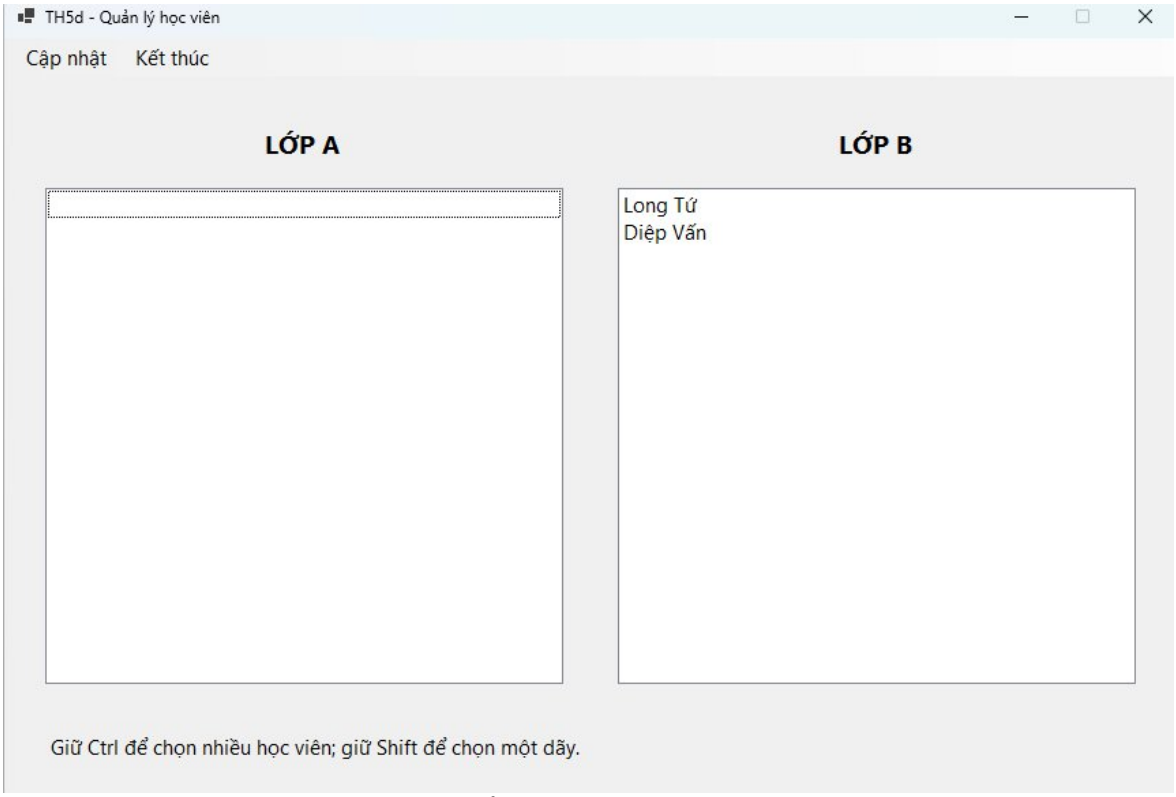
Hình 1. Giao diện quản lý học viên và menu chức năng.



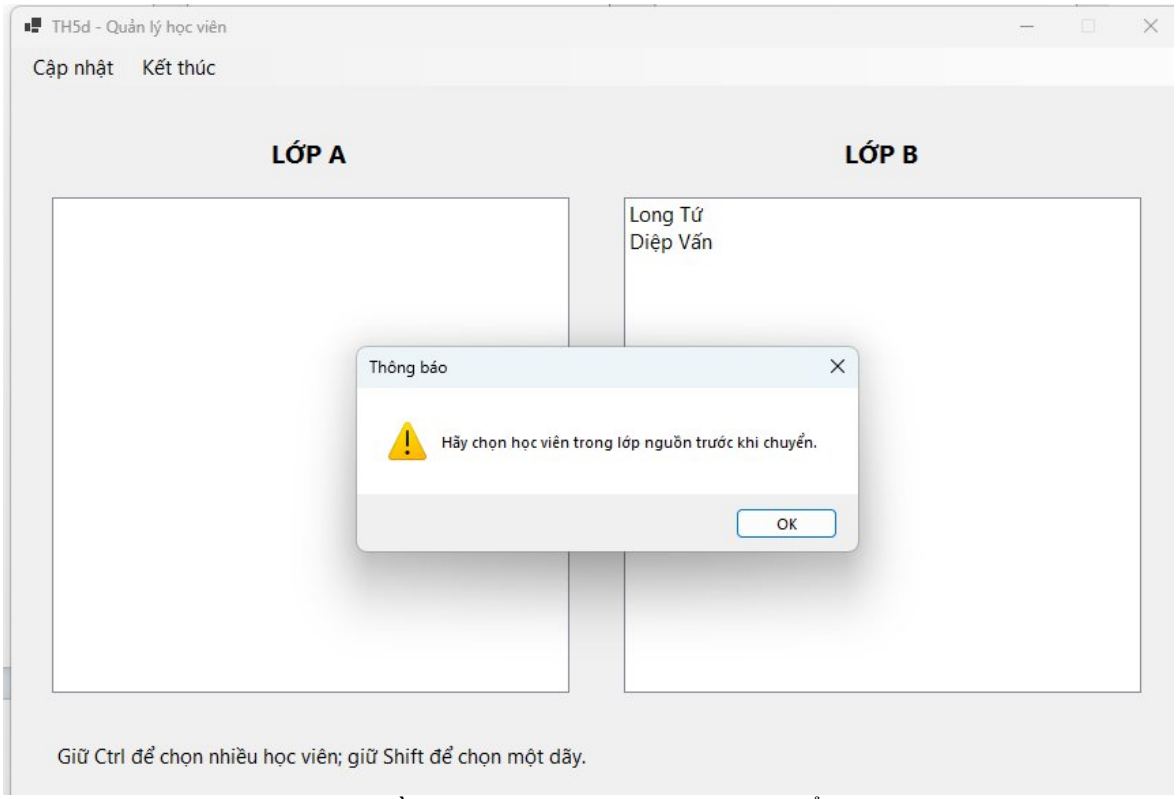
Hình 2. Nhập họ tên và chọn lớp cho học viên.



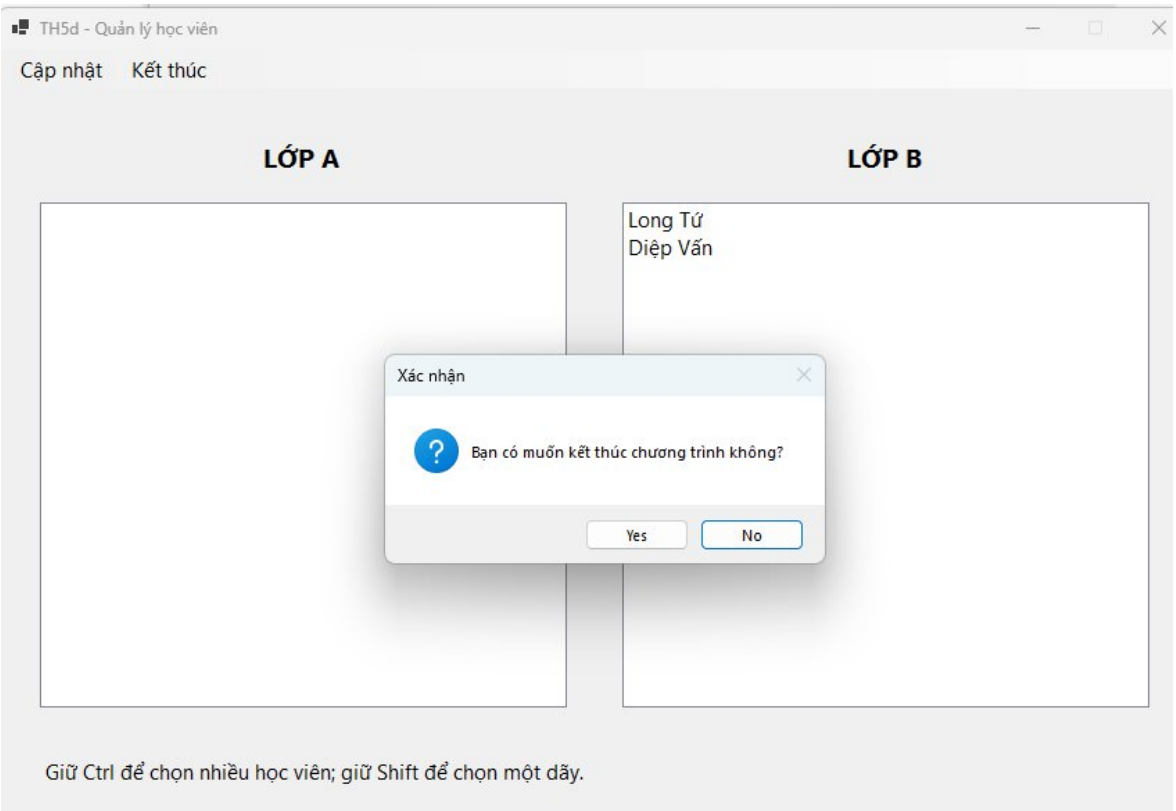
Hình 3. Học viên được thêm vào các lớp tương ứng.



Hình 4. Chuyển học viên giữa hai lớp.



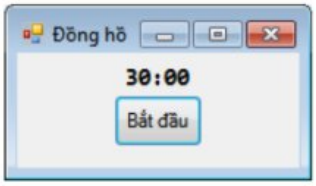
Hình 5. Yêu cầu chọn học viên trước khi chuyển lớp.



Hình 6. Xác nhận kết thúc chương trình.

Bài Tập Về Nhà - Bài 1:

Bài 1. Thiết kế form có giao diện thực hiện đếm ngược thời gian:



1. Yêu cầu bài tập

Thiết kế Form thực hiện đếm ngược thời gian theo giao diện trong đề. Đồng hồ ban đầu hiển thị 30:00, tương ứng với 30 phút.

Khi người dùng nhấn Bắt đầu, thời gian còn lại giảm dần. Khi về 00:00, chương trình dừng đếm và thông báo hết giờ.

2. Tổ chức chương trình

Bài tập được đặt trong project VeNha01_DemNguoc, thuộc thư mục BaiTapWinform\TH5d.

File Program.cs khởi động chương trình và mở Form frmDemNguoc. File frmDemNguoc.cs chứa giao diện, Timer và các phương thức xử lý đồng hồ.

Hằng số THOI_GIAN_BAN_DAU lưu thời lượng của một lượt đếm. Giá trị 30 nhân 60 tương ứng với 1800 giây.

3. Thiết kế giao diện

Giao diện gồm Label hiển thị thời gian và Button Bắt đầu, phù hợp với hình minh họa trong đề.

Label sử dụng font Consolas, kích thước lớn và căn giữa để dễ đọc. Thời gian được hiển thị theo dạng phút và giây, ngăn cách bằng dấu hai chấm.

Button Bắt đầu được gán cho thuộc tính AcceptButton để người dùng có thể nhấn Enter khi nút đang được bật.

4. Hiển thị thời gian

Phương thức HienThiThoiGian nhận tổng số giây còn lại. Số phút được tính bằng phép chia nguyên cho 60, còn số giây được tính bằng phép chia dư cho 60.

Định dạng D2 giúp mỗi phần luôn có ít nhất hai chữ số. Ví dụ, 65 giây được hiển thị thành 01:05.

Khi khởi tạo Form, chương trình gọi phương thức này với giá trị 1800 để hiển thị 30:00.

5. Bắt đầu đếm ngược

Khi nhấn Bắt đầu, chương trình đưa màn hình về thời gian ban đầu, vô hiệu hóa nút và đổi nội dung thành “Đang đếm...”.

Stopwatch được khởi động lại bằng Restart để đo thời gian của lượt hiện tại. Timer được bật bằng Start để cập nhật giao diện.

Việc vô hiệu hóa nút ngăn người dùng khởi động lại đồng hồ trong lúc một lượt đếm đang chạy.

6. Sử dụng Timer và Stopwatch

Timer của Windows Forms được đặt Interval bằng 200 mili giây. Mỗi lần phát sinh sự kiện Tick, chương trình lấy thời gian đã trôi qua từ Stopwatch.

Số giây còn lại được tính bằng thời gian ban đầu trừ số giây đã qua. Hàm Math.Max giữ kết quả không nhỏ hơn 0.

Timer có nhiệm vụ cập nhật giao diện định kỳ. Stopwatch có nhiệm vụ đo khoảng thời gian đã trôi qua. Vì vậy, chương trình không giảm một giây sau mỗi Tick mà tính lại thời gian còn lại dựa trên khoảng thời gian đo được.

Cách xử lý này hạn chế sai lệch tích lũy khi giao diện xử lý sự kiện Tick muộn.

7. Xử lý khi hết thời gian

Khi số giây còn lại bằng 0, Label hiển thị 00:00. Chương trình dừng Timer và Stopwatch trước khi mở MessageBox thông báo hết giờ.

Việc dừng Timer trước khi thông báo giúp hộp thoại chỉ xuất hiện một lần.

Nút được bật trở lại và đổi thành Bắt đầu lại. Khi nhấn nút này, chương trình khởi động một lượt đếm mới từ 30 phút.

8. Giải phóng tài nguyên

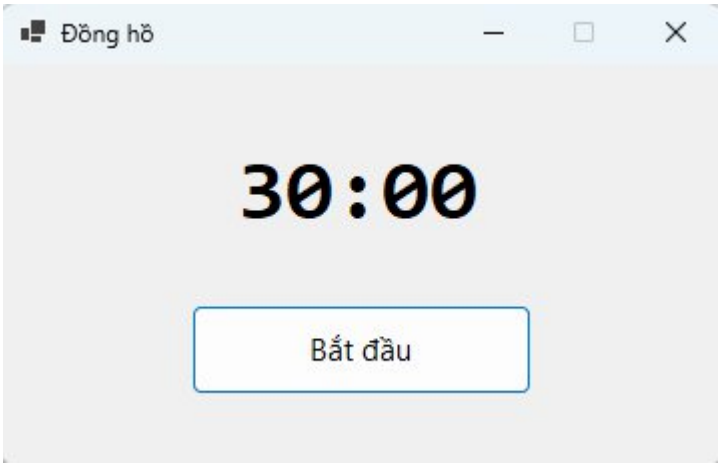
Timer được tạo trực tiếp bằng code nên chương trình giải phóng nó trong phương thức Dispose của Form.

Khi Form được giải phóng, Timer được dừng và gọi Dispose. Stopwatch cũng được dừng. Việc này tránh để bộ định thời tiếp tục hoạt động sau khi cửa sổ đã đóng.

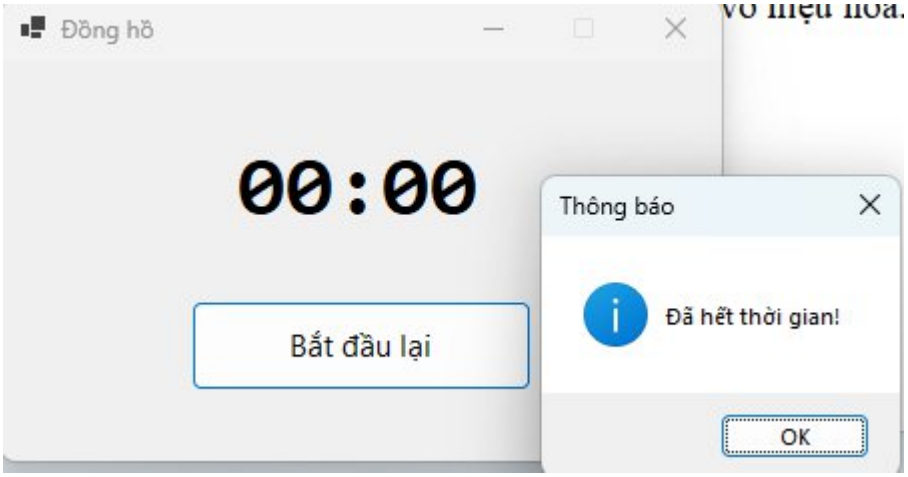
9. Hình ảnh kết quả thực hiện

Chèn ảnh giao diện ban đầu hiển thị 30:00.

Hình 1. Trạng thái ban đầu của đồng hồ đếm ngược.



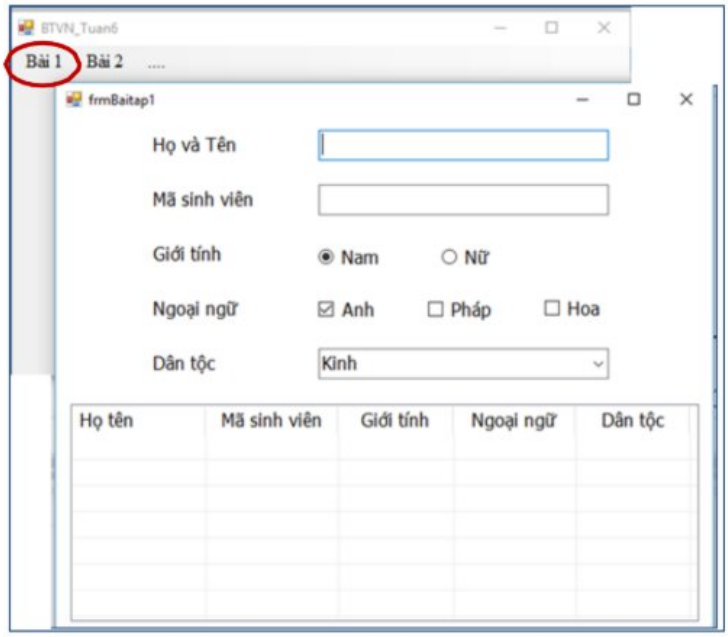
Hình 2. Đồng hồ đang đếm ngược và nút bắt đầu bị vô hiệu hóa.



Hình 3. Thông báo khi kết thúc lượt đếm.

Bài Tập Về Nhà - Bài 2:

Bài 2. Thiết kế form main có menustrip, gắn tất cả bài tập trong tuần này và hiển thị các form khi chọn menu tướng ứng.



1. Yêu cầu bài tập

Thiết kế Form chính có MenuStrip để mở các bài tập đã thực hiện trong tuần. Khi người dùng chọn một mục menu, chương trình hiển thị Form tương ứng.

Chương trình tổng hợp sáu bài TH5c và bốn bài TH5d đã làm, không bao gồm các bài mẫu. Các Form bài tập được hiển thị bên trong Form chính.

2. Tổ chức chương trình

Bài tập được đặt trong project VeNha02_MenuTongHop, thuộc thư mục BaiTapWinform\TH5d.

File Program.cs khởi động chương trình và mở Form frmMain. File frmMain.cs chứa MenuStrip, phương thức mở Form và xử lý kết thúc chương trình.

Project tổng hợp tham chiếu đến các project bài tập bằng ProjectReference. Nhờ đó, chương trình sử dụng lại các lớp Form đã viết mà không cần sao chép code sang project mới.

Các project bài tập vẫn có thể chạy riêng bằng lệnh dotnet run với đường dẫn tương ứng.

3. Thiết kế giao diện chính

Form chính được đặt IsMdiContainer bằng true để chứa các Form con. Khi mở chương trình, cửa sổ được phóng lớn để có đủ không gian hiển thị các bài tập.

MenuStrip gồm các mục Thực hành 5c, Thực hành 5d, Cửa sổ và Kết thúc.

Hai menu thực hành chứa các bài theo nhóm tại lớp, nâng cao và về nhà. Các đường phân cách giúp người dùng nhận biết từng nhóm.

Menu Cửa sổ có các chức năng xếp chồng, xếp ngang và xếp dọc. Thuộc tính MdiWindowListItem cho phép menu hiển thị danh sách các Form con đang mở.

4. Liên kết các project bài tập

Các đường dẫn ProjectReference được khai báo trong file csproj của project tổng hợp. Đường dẫn được tính tương đối từ thư mục project menu đến từng project bài tập.

Khi build chương trình tổng hợp, các project được tham chiếu cũng được build. Vì vậy, mỗi bài cần có code hợp lệ trước khi sử dụng trong menu.

Các lớp Form phải được khai báo public để project tổng hợp có thể truy cập. Namespace và tên lớp được ghi đầy đủ khi tạo Form để phân biệt các bài.

Việc tham chiếu chỉ đi từ project tổng hợp đến các bài tập, không tạo tham chiếu ngược.

5. Mở Form theo mục menu

Mỗi mục menu được gắn sự kiện Click. Khi người dùng chọn mục, chương trình gọi phương thức MoForm với lớp Form tương ứng.

Phương thức này tạo đối tượng Form, gán MdiParent là Form chính và gọi Show. Nhờ đó, Form bài tập xuất hiện trong vùng làm việc của cửa sổ tổng hợp.

Các Form được mở theo cách không chặn Form chính. Người dùng có thể tiếp tục chọn menu để mở một bài khác trong khi bài trước vẫn đang hoạt động.

6. Quản lý các Form đang mở

Chương trình sử dụng Dictionary với khóa là kiểu Form và giá trị là đối tượng Form đang mở.

Trước khi tạo cửa sổ mới, chương trình kiểm tra Form của bài đó đã tồn tại hay chưa. Nếu đã mở, chương trình gọi Activate để đưa cửa sổ lên phía trước. Nếu đang thu nhỏ, cửa sổ được đưa về trạng thái bình thường.

Nếu chưa mở, chương trình tạo Form mới và lưu vào Dictionary. Khi Form con đóng, sự kiện FormClosed loại đối tượng khỏi Dictionary để người dùng có thể mở lại bài đó.

Cách xử lý này tránh việc mở nhiều cửa sổ của cùng một bài do nhấn menu nhiều lần.

7. Sắp xếp và sử dụng các cửa sổ con

Menu Cửa sổ gọi phương thức LayoutMdi để sắp xếp các Form con. Chế độ Cascade xếp chồng, TileHorizontal xếp ngang và TileVertical xếp dọc.

Các chức năng bên trong Form bài tập vẫn sử dụng code đã viết trước đó. Chẳng hạn, Form ước số vẫn tính các ước, Form ngân hàng vẫn thêm và xóa tài khoản, còn Form đếm ngược vẫn cập nhật thời gian bằng Timer.

Dữ liệu của từng Form được giữ trong bộ nhớ trong thời gian cửa sổ đó đang mở. Khi đóng và mở lại Form, một đối tượng mới được tạo nên dữ liệu trở về trạng thái khởi tạo.

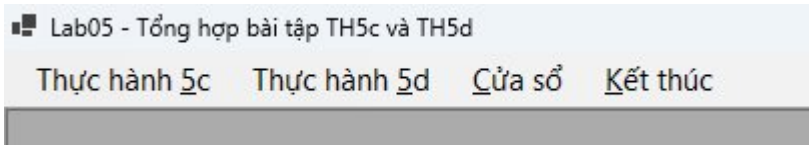
8. Kết thúc chương trình

Mục Kết thúc gọi Close của Form chính. Sự kiện FormClosing hiển thị hộp thoại xác nhận.

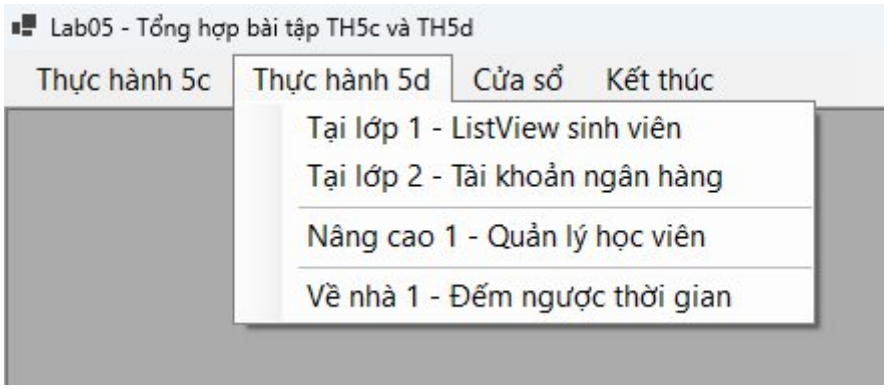
Nếu người dùng chọn No, chương trình hủy thao tác đóng. Nếu chọn Yes và các Form con cho phép đóng, chương trình kết thúc.

Các Form bài tập giữ nguyên xử lý xác nhận đóng đã được viết trước đó. Vì vậy, khi còn nhiều Form con đang mở, người dùng có thể nhận thêm hộp thoại xác nhận của các cửa sổ này.

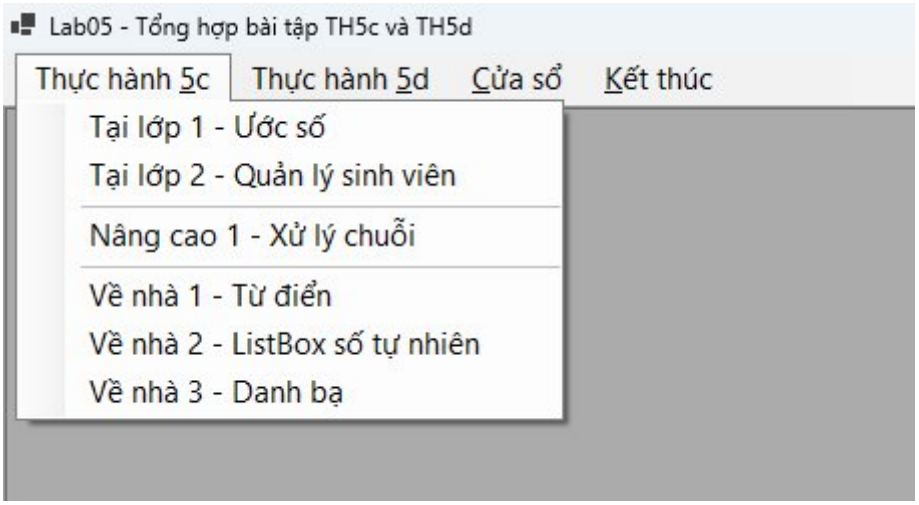
9. Hình ảnh kết quả thực hiện



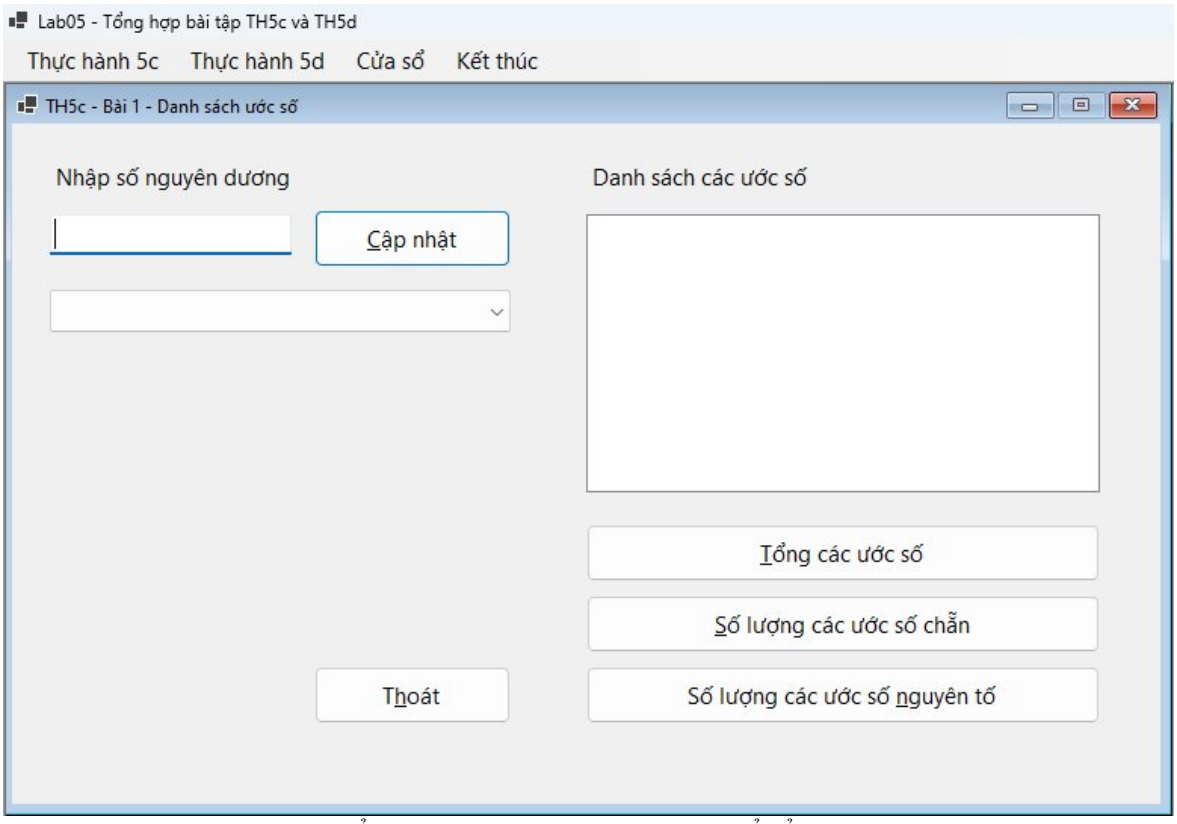
Hình 1. Giao diện chương trình tổng hợp bài tập.



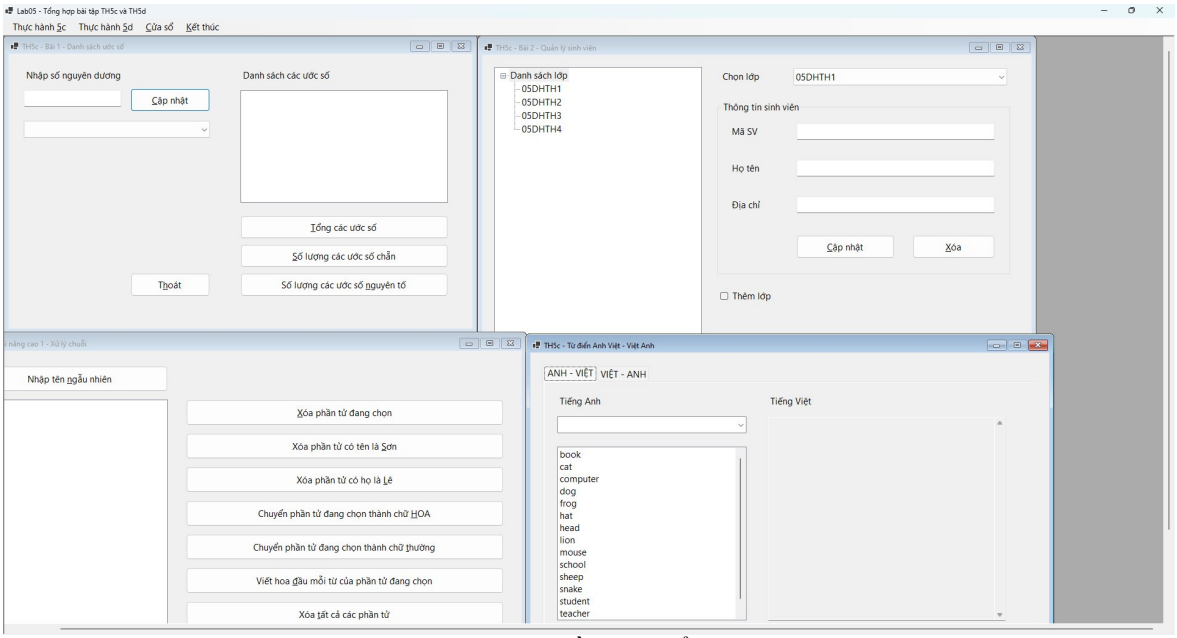
Hình 2. Danh sách bài tập TH5c.



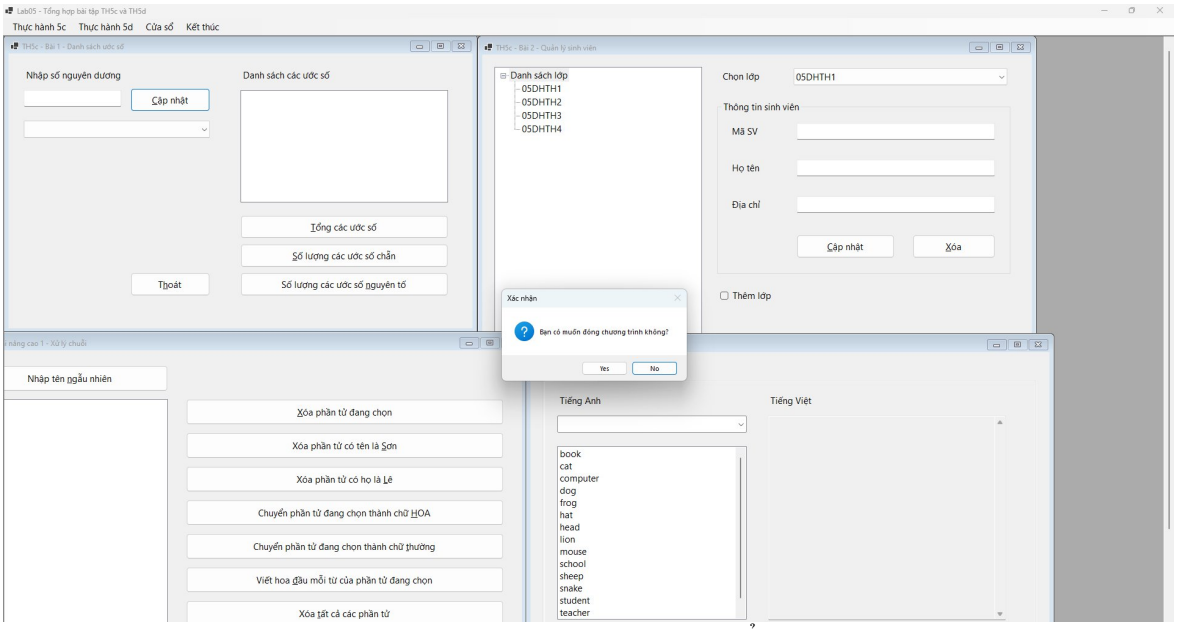
Hình 3. Danh sách bài tập TH5d.



Hình 4. Hiển thị Form bài tập trong cửa sổ tổng hợp.



Hình 5. Quản lý nhiều cửa sổ bài tập.



Hình 6. Xác nhận đóng chương trình tổng hợp.